AUTHBC

Feasibility Boundaries for Authenticated UAV Telemetry

An Exclusion Bound and a Capacity Envelope

Mohamed A. Farouk

A thesis submitted for the degree of

[DEGREE — TO BE COMPLETED]

[DEPARTMENT, INSTITUTION — TO BE COMPLETED]

October 25, 2026

Draft.

Abstract

Flying ad-hoc networks (FANETs) stream telemetry records so small that a cryptographic signature (64–96 B) exceeds the record it signs: compactly encoded, a record is 24.0 B, and 9.0 B as a difference from its predecessor. Once the payload has been compressed, authentication — not data — becomes the dominant on-air cost, and the operational question stops being *how many bytes does a scheme save* and becomes *which configurations can run at all*.

This thesis answers the second question. It searches four design axes jointly — record *encoding*, authentication *placement*, signature *scheme* and *batch* size — against a byte model, an NS-3-validated IEEE 802.11 broadcast channel model, and energy measured on Raspberry Pi 4 hardware. Three results follow, ordered by how durable they are.

An exclusion. A frame that verifies alone must carry a header, a chain link, a signature and one record that decodes without its predecessor. A link whose payload cannot hold those admits no such telemetry at any batch size, and packet loss forecloses fragmentation as an escape whenever the verifiability target is no looser than the loss rate. On LoRaWAN this removes eight of the twelve EU863-870 data rates: five because a 64 B signature alone overflows a 51 B payload, and three because this format's header, link and signature fill a 115 B payload before any record. The conditions under which that holds, and what each relaxation buys, are stated with it.

Bytes. Built as one frame format with a sender and a receiver, the design puts 43.25 B per record on air where per-record signing puts 145.77 B (70.3% fewer), at an operating point compliant with 3GPP TS 22.125. Every frame carries one self-contained record: a frame that depends on its predecessor does not meet the verifiability target at its own loss rate.

Capacity. Simulated directly, with a bootstrap interval, one 802.11a collision domain of 124 nodes still delivers 95% of frames with the design, against 35 nodes with per-record signing.

In the byte model the axes were first searched with, only placement and batching interact, by exactly $g_a(1-1/b)$; in the frame as built, delta coding is conditional on batching too, since a frame that must decode alone starts with a full record. The signature-scheme axis is byte-neutral among 64 B schemes. AUTHBC introduces no new cryptographic primi-

tive. Its contribution is a reproducible co-design, its quantified trade-offs, the boundary beyond which no batch size helps, with the conditions under which it holds and what each relaxation buys — and a methodological record of the defects found while establishing it, including three results this thesis had to correct after an external review.

Declaration

▶ *TO BE COMPLETED by the candidate. Institutional wording required.*

Acknowledgements

► *TO BE COMPLETED.*

Nomenclature

Symbols

Symbol	Meaning	Value
N	nodes in one collision domain (a *neighbourhood* of N; each node hears $N-1$)	varied
Λ_i	record rate of one UAV, records per second	50 (adopted), 20
Λ	rate at which a receiver must verify, $\Lambda_i(N-1)$	derived
$D_{\max}$	freshness bound on the oldest record of a frame	100 ms, 250 ms
$D(b)$	delay of the oldest record in a batch of b	$\leq D_{\max}$
b	records per frame (batch)	decision variable
r	reference interval: one frame in r decodes alone	1 in the design
s	bytes of one encoded record (first format: with its chain link)	measured
s_k, s_δ	record that decodes alone (keyframe); record coded against its predecessor	measured
g_a	bytes of the authentication object	64, 64, 96
H_f	bytes of the frame header	44 first format, 23 lean
L	bytes of the chain link carried once per frame (lean format)	35
M	largest payload the link carries in one frame	1500 (802.11)
$s_{\max}$	largest record a single frame admits, $M - H_f - L - g_a$	derived
p	probability that a frame is lost	0.02, 0.05, 0.10
ε	verifiability slack: $V \geq 1-\varepsilon$ is required	$= p$
V	probability that a generated record is verified at a given receiver	$\geq 1-\varepsilon$
R	PHY data rate	6 Mb/s
f	frames per second sent by one node, Λ_i/b	derived
T	time one frame holds the medium (airtime plus DIFS)	derived
W	contention window in slots	16
ρ	share of time the other nodes hold the medium	derived
U	offered load over saturation throughput	derived
N_{sat}	largest N whose offered load is within saturation throughput	derived
$N_{\max}$	largest N with mean delivery at least $1-\varepsilon$	simulated
N_{cpu}	largest N whose frames one receiver core can verify	derived
t_v, t_h	time to verify one signature; to hash one record	measured
P_c, P_r	power drawn by the processor; by the radio	measured

A note on H_f. It is a *range*, not a constant. Canonical CBOR encodes small integers in one byte and large ones in five, so the measured frame header moves by six bytes with the magnitudes of the sender identifier and sequence number (Table 3.4).

Abbreviations

ARQ	automatic repeat request (retransmission on loss)
BLS	Boneh–Lynn–Shacham pairing-based signature
CBOR	Concise Binary Object Representation (RFC 8949)
CLAS	certificateless aggregate signature
DCF	distributed coordination function, the 802.11 contention access rule
DIFS	DCF inter-frame space
DR	LoRaWAN data rate (DR0–DR11 in EU863-870)
ECDSA	elliptic-curve digital signature algorithm
ECQV	elliptic-curve Qu–Vanstone implicit certificate
FANET	flying ad-hoc network
LR-FHSS	long-range frequency-hopping spread spectrum
MAC	medium access control; in a cryptographic context, message authentication code
MTU	maximum transmission unit
PHY	physical layer
SF	LoRa spreading factor
SITL	software in the loop
TESLA	timed efficient stream loss-tolerant authentication
UAV, UAS	unmanned aerial vehicle; unmanned aircraft system

Contents

Abstract **1**

Declaration **3**

Acknowledgements **4**

Nomenclature **5**

1 Introduction **19**

1.1 The problem, in one paragraph 19

1.2 Why this is not simply a compression problem 19

1.3 The question this thesis asks 20

1.4 Research questions 20

1.5 Contributions 21

1.6 Explicit non-goals 22

1.7 Structure of this thesis 23

2 Background and Related Work **24**

2.1 Flying ad-hoc networks and UAV telemetry 24

2.1.1 The workload, read at source 25

2.1.2 How such networks are usually evaluated 26

2.2 Tamper-evident telemetry and hash-chained ledgers 26

2.2.1 What the chain gives and what the signature gives 26

2.3 Signatures for constrained authentication 27

2.4 Authenticating messages one at a time, and signing streams 28

2.4.1 Per-message practice 28

2.4.2 Amortising one signature over many packets 28

2.4.3 A standard that faces the same disproportion . . . 29
2.4.4 Certificates . . . 29
2.5 Batch and aggregate verification . . . 30
2.6 Compact serialisation and delta coding . . . 31
2.7 Channel models for IEEE 802.11 . . . 31
2.7.1 Frame aggregation . . . 32
2.8 LoRa and LoRaWAN capacity . . . 32
2.8.1 Limits that are rules, not physics . . . 32
2.9 Where this thesis sits . . . 33

3 System Model and Threat Model 34
3.1 Network . . . 34
3.1.1 Why broadcast, and why it matters . . . 34
3.2 Ledger . . . 35
3.3 Traffic and the two arrival rates . . . 36
3.3.1 The adopted operating point . . . 36
3.3.2 The floor of the region, which is a real constraint on the application 36
3.4 Threat model . . . 37
3.4.1 The adversary . . . 37
3.4.2 What is claimed, and where each property is enforced . . . 38
3.4.3 Formal analysis, and the evidence for each property . . . 39
3.4.4 What is not claimed . . . 39
3.4.5 Behaviour under loss . . . 41
3.5 The decision space . . . 41
3.5.1 Authentication placements . . . 41
3.5.2 Schemes and encodings . . . 42
3.6 Two frame formats . . . 42
3.6.1 The lean frame . . . 42
3.7 The frame header, which is a range . . . 44

4 Theoretical Framework 45
4.1 T1 — the overhead-dominance threshold . . . 45
4.2 T2 — frame-level batching and the amplification law . . . 45

4.3 T2a — which ceiling binds, and what compression is therefore worth 46
4.4 T3 — the loss-robustness frontier 47
4.5 T3' — a frame coded against its predecessor is not self-verifiable 47
4.6 T4 — scheme selection 48
4.7 T5 — an empirical separability result 49
4.8 T6 — the authentication-exclusion threshold 49
4.8.1 Four tiers, because each names a different escape route 50
4.8.2 Scope 50
4.9 A withdrawn theorem 50
4.10 Freshness, and a convention that had to be derived 51
4.10.1 The peak age under loss 52
4.10.2 A window on GNSS time 52
4.10.3 A loss budget with a margin 52
4.10.4 The design map 53

5 Implementation 54

5.1 Design rule: one home per fact 54
5.2 The library 54
5.3 Wire format 56
5.3.1 One frame, byte by byte 56
5.4 Sender and receiver 57
5.5 Before the receiver: admission, and a window on time 58
5.6 A compiled codec 59
5.7 Tests 59
5.8 The two-layer data pipeline 60

6 Experimental Methodology 61

6.1 Binding statistical rules 61
6.1.1 Why 30 seeds, and what it does not buy 61
6.1.2 Intervals that cover what they say 62
6.1.3 Thresholds must be applied to distributions 62
6.2 Pre-registration 62
6.3 Experiment matrix 63

6.4 Telemetry . . . 64
6.5 Simulation campaign . . . 64
6.6 Hardware campaign . . . 65
6.6.1 Boards and timing . . . 65
6.6.2 The energy rig . . . 65
6.6.3 The compiled sender, and floods against one core . . . 66
6.6.4 The model the meter is compared with . . . 66
6.6.5 Uncertainty of the energy figures . . . 67
6.6.6 Two radios . . . 68

7 Results I — Bytes, Placement and Loss 69
7.1 E1 — authentication dominates after compression . . . 69
7.2 E2 — batching is the cure, and which ceiling binds . . . 70
7.3 E3 — the loss frontier . . . 70
7.4 E10 — loss with the decoder in the loop . . . 71
7.4.1 When loss grows with frame length . . . 72
7.4.2 The loss the simulator makes, and a budget with a margin . . . 73
7.5 E9 — what a frame is made of . . . 74

8 Results II — Co-design and Capacity 78
8.1 E5 — the end-to-end configuration, and a correction to it . . . 78
8.2 The factorial ablation, which narrows the claim . . . 79
8.3 E4 — scheme selection, and the receiver's CPU . . . 79
8.4 E17 — the classical stream-signing schemes, placed . . . 82
8.5 Capacity, by direct search . . . 83
8.5.1 A rule for the crossing, and the test it was put to . . . 84
8.5.2 How the traffic source was chosen . . . 87
8.5.3 Why the crossing is where it is . . . 89
8.5.4 Nodes spread out, and capture . . . 91
8.5.5 A spread, moving swarm . . . 93
8.5.6 A batch chosen at run time . . . 93
8.5.7 Other PHYs, and a post-quantum signature . . . 96
8.5.8 Flooding . . . 96

8.6 Freshness, with every term . . . 97
8.7 Certificates . . . 98
8.8 Energy . . . 99

9 Model Validation and Hardware 101

9.1 Three independent implementations of the same physics . . . 101
9.2 The mechanism, confirmed by removing it . . . 101
9.3 A non-monotonicity that is real . . . 102
9.4 Hardware . . . 102
9.4.1 The link . . . 103
9.4.2 How one radio spaces its own frames . . . 103
9.4.3 Two radios at a set load: the registered experiment . . . 104
9.4.4 Two saturated radios . . . 105
9.4.5 What the hardware supports, and what it does not . . . 106
9.5 The utilisation ceiling, and a flaw in the author's own pre-registration . . . 106
9.6 The other half: invariance in N, tested, and it does not hold . . . 107
9.7 The access rule away from its operating point . . . 108

10 The Low-Rate Regime and the Exclusion Bound 109

10.1 Why the low-rate arm is a different regime, not a slower one . . . 109
10.2 The exclusion . . . 110
10.2.1 The floor of the format . . . 110
10.2.2 All twelve data rates . . . 110
10.2.3 Scope . . . 111
10.2.4 What each relaxation buys . . . 112
10.2.5 How this result reached its present form . . . 113
10.3 Per-frame chaining, and what it is worth . . . 114
10.4 Capacity . . . 114
10.4.1 A topology caveat . . . 115
10.4.2 The result, and how it must be quoted . . . 115
10.4.3 Why DR5 and not DR6 . . . 116
10.4.4 What the configuration models, and what it does not . . . 116
10.4.5 The result is range-limited, and that limit binds first . . . 117

10.4.6 Mobility . . . 117
10.4.7 What infrastructure collection would buy . . . 118
10.4.8 External corroboration, quoted as a curve . . . 118
10.5 The same accounting on other broadcast links . . . 119
10.6 What the regime is for . . . 120
10.7 Scope of this chapter . . . 120

11 Reproducibility, Defects and Research Integrity 121

11.1 Why this chapter exists . . . 121
11.2 The defect taxonomy . . . 122
11.3 October 2026: five errors that survived two audits . . . 122
11.3.1 The capacity ceiling, and a prediction that failed . . . 123
11.4 A simulator artifact large enough to change what a reader should believe . 124
11.4.1 A claim that was withdrawn . . . 124
11.5 An automated staleness gate, and what it does not cover . . . 125
11.6 Practices that earned their place . . . 125
11.7 Two defects worth stating individually . . . 126
11.7.1 A test that asserted a bug . . . 126
11.7.2 A criterion that could not fail . . . 126
11.8 What a reader should take from this . . . 126
11.9 How citations are checked . . . 127
11.10 Where this sits . . . 127

12 Conclusions, Limitations and Future Work 129

12.1 What this thesis establishes . . . 129
12.2 The research questions, answered . . . 130
12.3 Limitations . . . 131
12.3.1 Modelling scope . . . 131
12.3.2 Validation scope . . . 131
12.3.3 The low-rate arm . . . 132
12.3.4 The boundary itself . . . 132
12.3.5 Security scope . . . 132
12.4 Future work . . . 133

12.4.1 Small 133
12.4.2 Substantial 133
12.4.3 Speculative 133
12.5 Closing 134

A Reproducing the Results **135**
A.1 Four paths, by cost 135
A.2 Path A 136
A.3 What is where 136
A.4 Paths B to D 136
A.5 Software 137

B Predictions Registered Before Their Data **138**

C Results That Were Corrected **144**
C.1 The configuration as first reported 144
C.2 The capacity envelope from one load ceiling 145
C.3 Per-frame chaining as first computed 146

List of Figures

3.1 System overview: the sender's pipeline, the four-record frame with the size of each field in bytes, the shared channel, the receiver's pipeline, and the five constraints that select the design. 35

3.2 The lean frame, to scale (bytes below). Per-record signing repeats header, chain link and signature with every record; the design carries them once per four records, and the three records after the first are coded as differences (δ). 42

5.1 The packages of the library 55

7.1 Authentication fraction across encodings. Every byte saved on the payload raises the fraction spent on cryptography. 70

7.2 Per-record cost against batch size. The curve flattens where the freshness ceiling takes over from the frame-size ceiling. 71

7.3 The loss-robustness frontier. Block-level authentication trades verifiability for bytes at a rate the constraint does not permit. 71

7.4 Records verified by the design ($b = 4$, $r = 1$) against frame loss under five loss processes, with 95% intervals; for loss that grows with length, p is the loss of a one-record frame. Shaded: the losses the margin rule admits at $\varepsilon = 0.05$. 73

7.5 Bytes per record against batch size, by where the chain link sits. Records are delta-coded in all three and a frame starts with a keyframe, so at $b = 1$ the first-format curves start below row 1 of Table 8.1, whose records are plain CBOR. The dotted line is the batch that freshness allows at the adopted operating point. 75

8.1 The scheme crossover. BLS becomes worthwhile only in the cross-signer regime. 80

8.2 Delivered fraction of frames against neighbourhood size, lean format: per-record signing and the design. Mean of thirty runs; bars span the runs. . . 84

8.3 The neighbourhood the lean design serves at 50 records/s, by batch size and the smallest deadline that admits it: the five simulated crossings of 0.95 delivery (95% intervals, smaller than the markers) and the airtime line through the batch sizes that were not simulated. At 40 ms the batch is one record and nothing is gained. 86

8.4 Nodes served at 95% delivery as the swarm spreads over a disc (top), and the design's ratio to per-record signing with its 95% interval (bottom). Under each radius, the share of spoilers hidden from the sender; hollow markers, the swarm at 1 km moving at 0, 5 and 15 m/s. 94

8.5 A swarm growing in flight, mean of ten seeds: the batch each policy chooses, the share of records verified in each second, and the peak age of information. 95

8.6 Honest delivery on one Raspberry Pi 4 receiver core with 41 senders, against the rate of forged frames, for each defence and attacker. The dotted vertical line is the most one 802.11a channel carries. 97

8.7 The design map: the batch b^* each record rate and deadline admit, and the capacity it buys over per-record signing by the access-rule model. Outlined: cells simulated in ns-3, for per-record signing, the design or both; hatched: no batch meets the deadline. 98

9.1 Analytic models against simulation, both modes. 102

10.1 Payload limit of each data rate against the floor of the lean format and the frames this telemetry produces. 111

C.1 The configuration as first reported 144

C.2 The envelope as previously reported (saturation threshold). Superseded for the $V \geq 0.95$ threshold by direct simulation. 145

C.3 Per-frame chaining as first computed 146

List of Tables

3.1 The two operating points, lean format. Bytes are identical; the neighbourhood is not. $N_{\max}$ is the largest neighbourhood with mean delivery ≥ 0.95, found by direct simulation (Chapter 8). 36

3.2 Security properties and threats, the mechanism for each, and the evidence: proofs in Tamarin, simulated clocks, and floods against one Raspberry Pi 4 receiver core. 40

3.3 Byte budget of a frame, field by field, for a sender with a five-digit identifier one hour into a 50 Hz flight (smaller values earlier in a flight in parentheses). 43

3.4 H_f is not a constant. Canonical CBOR encodes integers 0–23 in one byte and those ≥ 65536 in five, so the measured header moves with field magnitude. 44

5.2 A lean frame of two records, byte by byte 57

6.1 Primitive timings on the Raspberry Pi 4, medians. 65

7.1 Measured record size and authentication fraction. Compression moves the bottleneck. 69

7.2 Verifiability at frame loss $p = 0.05$ with one self-contained frame in r ($b = 4$). Mean and 95% bootstrap interval over thirty seeds, against Theorem 4.6. . 72

7.3 Record sizes on real flight logs against the generator, at matched spacing. The logs carry position at 5 Hz, so nothing finer than 0.2 s can be measured from them. 74

7.4 Records from 30 flights of the autopilot's simulator, ten per airframe, at 50 records/s: keyframe and per-airframe delta in bytes, the bytes the design saves, and the nodes it serves over per-record signing by the access-rule model. 76

8.1 The design ladder at 50 records/s and $D_{\max} = 100\,\text{ms}$. N_{sat}: offered load within the saturation throughput of the broadcast model. $N_{\max}$: largest neighbourhood with mean delivery ≥ 0.95 in simulation, with its 95% bootstrap interval. CPU: share of one receiver core spent verifying at $N_{\max}$. † size is a sum of measured parts; rows 5, 6 and 8 are emitted frames. . . . 79

8.2 Time to receive a four-record frame on one Raspberry Pi 4 core, and the nodes one core serves at 50 records/s; all four timed in one session. 82

8.3 Stream-signing schemes at 50 records/s. Sizes are modelled; $N_{\max}$ is simulated for a frame of that size, with its 95% bootstrap interval. † no non-repudiation: a third party cannot be shown who sent a record; ‡ a packet does not verify alone, so $N_{\max}$ — frames delivered — is an upper bound on records verified; the last column counts a record only if it verifies within $D_{\max}$. § disclosure delay of two intervals of at most 20 ms. 82

8.4 $N_{\max}$ of baseline and design at both operating points, with 95% bootstrap intervals. 84

8.5 The seven held-out configurations: the crossing the line predicted, the crossing simulated (interpolated, thirty seeds per node count), and the deviation of the simulation from the line and from the single load ceiling. 85

8.6 The lean design at three other batch sizes, 50 records/s: the crossing of 0.95 delivery predicted by the airtime line (slope as calibrated before, band ±6%) and by the access-rule model (nothing fitted, band ±3%), and as simulated (30 seeds, 95% interval). All six predictions were committed before the first run. 86

8.7 The lean baseline (146 B at 50 frames/s) and the design (173 B at 12.5) with the nodes uniform in a disc and free-space loss: the crossing of 0.95 delivery as predicted and as simulated (30 seeds, each with its own placement; 95% interval). 92

8.8 Nodes served at 95% delivery in ns-3 on other PHYs and with a post-quantum signature, and the access-rule model's error (negative: optimistic; the larger of the two rows). Frame: the design's, on air. 96

8.9 Delay of the oldest record in a frame, $b = 4$, 50 records/s. Channel delay is simulated at the heaviest load that still delivers 95% of frames. 97

8.10 Sender energy per record on a Raspberry Pi 4: metered (median and range of five repetitions) against time multiplied by power. The first-format rows are of July 2026; the lean rows are of October, on the second board through the second sensor, where a control that repeats the first row gives 116.2 µJ; the compiled rows are of 22 October, on that board, beside a control giving 118.0 µJ. 99

9.1 Saturation throughput (Mb/s), 1400 B broadcast frames. The slot-exact simulator was written *before* the published model was located and shares none of its assumptions. 101

9.2 Time from one frame of a saturated sender to its next, read at the receiver. The frame itself is 1976 µs on air. 103

9.3 Frames lost between two radios, in percent. Three sessions of twelve windows on one night; the last column is all 36 windows over the model's prediction. 104

9.4 Two radios, each offered 400 frames/s where the medium carries about 490. Frames on air are read at the receiver. Eight values per column. 105

10.1 EU863-870 data rates against one self-contained, signed, chained frame (g_a = 64 B). The verdict is the same in both frame formats and under both of the standard's payload tables. 110

10.2 The lean format under four authentication choices. *Floor*: every field zero. *Range*: frames of this telemetry over a flight. 112

10.3 The low-rate capacity by two independent methods: frame loss at N transmitters. 119

10.4 Records per frame each broadcast system admits with one signature per frame, by deadline, under the freshness bound; —: no signature fits. The last row is this thesis's design, by the same rule. 120

B.2 Registrations of October 2026 and their outcomes. 141

C.1 The envelope as previously reported, from the single load ceiling. **Superseded** by Tables 8.1 and 8.4; kept because the comparison is the finding. . 145

Chapter 1

Introduction

1.1 The problem, in one paragraph

A flying ad-hoc network of unmanned aerial vehicles must do two things at once that pull against each other. It must broadcast telemetry often enough to be useful — position, velocity, battery, mode, tens of times a second — and it must make that telemetry *tamper-evident*, so that a record cannot be forged, replayed or silently altered by a participant or an outsider. The second requirement costs bytes. A digital signature at the 128-bit security level is 64 B for Ed25519 or ECDSA-P256, and 96 B for the BLS12-381 variant actually shipped by the reference implementation. A telemetry record, once compressed, is 45 B — and 32 of those are not telemetry either: they are the hash that links the record to its predecessor. The data are 13 B.

Those are the sizes of the first frame format this work built, in which every record carries its own chain link, and they are where the argument starts. The lean format of Chapter 3 carries the link once per frame and codes the same record in 24.0 B standing alone and 9.0 B as a difference from its predecessor; the results of this thesis are stated for both formats, and the inequality holds in each.

That inequality is the whole thesis. **The signature is larger than the thing it signs.**

1.2 Why this is not simply a compression problem

The obvious response — compress harder — fails, and it fails for a reason worth stating precisely. Payload compression was the subject of prior work; this thesis begins where that work ends. Once a record is squeezed from 191 B of JSON to 45 B of delta-encoded CBOR, the signature has gone from 25% of the on-air cost to 59%; counted with the chain link inside the record, signature and link are 96 of 109 B. Every further byte saved on the payload increases the fraction spent on cryptography. Compression does not solve the

problem; *compression is what creates it.*

This thesis calls the system it builds AUTHBC — authenticated telemetry for a blockchain-style, hash-chained ledger: a frame format, a sender and a receiver, with the four design choices below made together. The design space has four axes, and they are searched jointly:

- **Encoding** e — how a record is serialised (JSON, CBOR, MessagePack, delta-CBOR).
- **Placement** — *where* the signature sits relative to the records it covers: inline per record, self-batched over b of one sender's records, cross-signer aggregated, or block-level across multiple frames.
- **Scheme** σ — which signature algorithm, and therefore how many bytes the authentication object costs and how much CPU verification demands.
- **Batch** b — how many records share one authentication object, bounded by the frame size on one side and by a freshness deadline on the other.

1.3 The question this thesis asks

The literature on efficient authentication asks *how many bytes does my scheme save.* That is a useful question and it has been answered many times. This thesis asks a different one, because for the deployments that motivate it the byte count is not the binding constraint:

> **Which configurations can run at all?**

That reframing — from optimisation to feasibility — is the thesis's organising decision, and Chapter 11 records why it was made partway through the work rather than at the start. Briefly: as a co-design optimisation the result is mid-tier, because the optimisation is largely closed-form and a factorial ablation showed only one genuine interaction among the four axes. As a *boundary* result the same evidence is stronger, because an impossibility cannot drift as models improve — and four of this project's own performance numbers did drift, and had to be corrected.

1.4 Research questions

RQ1 How large is authentication overhead after compression, and where is the payload-size threshold below which it dominates?

RQ2 Which authentication placement minimises on-air bytes while keeping records verifiable under loss, and what does self-batching change relative to cross-signer aggregation?

RQ3 When does an aggregatable scheme beat a fast conventional one, as a function of relay fraction, arrival rate and the radio/CPU power ratio?

RQ4 What is the end-to-end optimal configuration, how much does it beat the naive and compression-only baselines, and do the analytical models survive contention simulation and real hardware?

RQ5 *(added during the work)* Where is the boundary beyond which a signed, hash-chained frame cannot be sent at all, whatever the batch — under which conditions does it hold, and what does each relaxation (a shorter authenticator, no chain link on air, loss recovery) buy?

RQ5 was not in the original charter. It emerged from the low-rate arm and became the thesis's most durable result; Chapter 10 develops it, and records the two occasions on which this thesis stated it wrongly.

1.5 Contributions

1. **An overhead-dominance characterisation** for compressed authenticated FANET ledgers, with the threshold stated in closed form and measured across four encodings (Chapter 4, Chapter 7).

2. **A placement theory** distinguishing self-batching from cross-signer aggregation, with a loss-robustness frontier proving that frame-level batching Pareto-dominates block-level under any stationary loss process — not merely under the independent-loss assumption the emulator implements (Chapter 4).

3. **An exclusion bound instantiated on a real band, with its scope.** Eight of the twelve EU863-870 data rates cannot carry one signed, hash-chained frame: five because a 64 B signature will not fit a 51 B payload, three because header, chain link and signature fill a 115 B payload before any record, in this frame format. The five conditions under which that holds, and what each relaxation buys, are stated with it (Chapter 10).

4. **The design built as one object, and its capacity measured directly.** A frame format with a sender and a receiver, in which the sizes of the design and of its baselines are lengths of emitted frames, and a size that is a sum of measured parts is marked as one; and the largest neighbourhood each configuration sustains, found by simulating each at its own frame size and rate with a bootstrap interval: 124 nodes for the design against 35 for per-record signing (Chapter 3, Chapter 8).

5. **Validated models with their gaps stated**, including a broadcast channel model validated to $\leq 0.08\%$ against an independent slot-exact simulator and $\leq 0.51\%$

against NS-3, plus measurements on two radios of the link's loss and of the access rule under saturation, and a registered prediction of their loss below saturation that failed at one load of three (Chapter 9).

6. **The gain under the conditions a reviewer would vary.** Spread over discs of up to 1 km with fading, capture and hidden stations, moving on independent paths, carried by six 802.11 PHYs and rates, or signed with the post-quantum Falcon-512, the design serves between 3.39 and 3.80 times the nodes of per-record signing; and a design map over record rate and deadline gives the batch each pair admits, checked in ns-3 at 15 cells within 1.05% (Chapter 8).

7. **A formal model and two defences.** The protocol's properties are proven in the Tamarin prover, in three models; a window on GNSS time refuses frames an attacker holds back; and an 8 B group tag stops a flood from outside the swarm, while a flood from a member is shown not to be stopped (Chapters 3 and 8).

8. **The prototype's anomalies traced to the interpreter.** Receiver and sender rewritten in a compiled language, byte-identical on 10 000 frames: one core of the board then serves 289 nodes, so the channel binds again, and the design's sender spends 17.1 µJ per record against the first format's 58.4 (Chapters 5 and 8).

9. **An ablation that narrows our own claim**, showing that in the byte model only placement and batching interact, that an apparent encoding interaction was a ratio-scale artifact, and that the one real coupling of encoding to batching appears only when the frame is built (Chapter 8).

10. **A reproducible testbed and a defect taxonomy.** Every model-derived artifact is re-derived byte-identically by an automated gate, and the classes of defect that survived that gate are catalogued — including the ones found in this work (Chapter 11).

1.6 Explicit non-goals

This thesis introduces no new cryptographic primitive, no new consensus protocol and no new post-quantum scheme: Falcon-512 is taken from a library, timed on the board and simulated in the frame, and signatures that do not fit one frame remain projections. Key compromise, multi-hop routing research and finality semantics are out of scope. The low-rate arm is a *modelling* chapter: no LoRa radio was metered and no energy column exists for it.

1.7 Structure of this thesis

Chapter 2 places the work in its literature. Chapter 3 fixes the system and threat models and the notation. Chapter 4 develops the theory. Chapter 5 describes what was built and Chapter 6 how it was measured. Chapter 7 and Chapter 8 give the 802.11 results; Chapter 9 the channel-model validation and hardware; Chapter 10 the low-rate regime and the exclusion bound. Chapter 11 is unusual for a thesis and deliberate: it reports the errors found in this project's own results, how they were found, and what class of defect resisted the safeguards. Chapter 12 concludes.

Chapter 2

Background and Related Work

This chapter places the work: the networks and the telemetry it concerns, what a hash chain and a signature each provide, how messages are authenticated in practice and in the stream-signing literature, how batching and aggregation differ, and the channel models the evaluation rests on. Every source cited was read at the source and not through a summary of it. Where a figure reaches this thesis through another paper, that is said at the point of use; where only part of a long document bears on this work, the register of sources kept with the repository records which part was read. Section 11.9 describes how the bibliography is checked.

2.1 Flying ad-hoc networks and UAV telemetry

A flying ad-hoc network is a set of unmanned aircraft that exchange information directly, without a base station between them. Chen, Tang and Lao's review of swarm communication distinguishes the architectures by where information is exchanged — between the aircraft, or between the aircraft and a central control centre — and treats the routing protocols that each needs [1]. This thesis is concerned with the simplest exchange the decentralised architectures share, and the one all the others rest on: every aircraft periodically *broadcasts its own state* to whichever neighbours can hear it. There is no route, no acknowledgement and no retransmission; a neighbourhood is a set of receivers within range of one another, and Chapter 3 models it as one collision domain.

Surveys of the field treat such networks as a class of their own. Gupta, Jain and Vaszkun set UAV networks beside mobile and vehicular ad hoc networks and find that results for those two do not carry over unchanged: the nodes may be anything from stationary to fast, the topology changes and links come and go, a node that fails or leaves can partition the network, and a small aircraft has little energy to spend [2]. They organise the open problems around architecture, routing, handover between nodes and energy. Authentication of what the nodes broadcast is not among them, and the word

does not occur in the survey. The literature on these networks concentrates on the layers above the single-hop broadcast; this thesis is about the cost of making that broadcast attributable, which every one of those layers takes as given.

Two strands of that literature meet this thesis directly. Bujari *et al.* survey the application scenarios of these networks and the mobility models used for them [3]; the moving swarm of Section 8.5.5 uses one of those models, Gauss–Markov, with parameters in the range that literature reports. And at the medium access layer, FMAC adapts the contention window of a flocking swarm, predicting the flock's density from the motion its neighbours announce [4]. Acting on a predicted state is the precedent of the batch controller of Section 8.5.6, which changes the batch where FMAC changes the window.

2.1.1 The workload, read at source

What such a broadcast carries, and how often, is taken here from autopilot source code and from a standard, not from secondary summaries.

PX4. The MAVLink module configures its streams by link mode. For a link to a companion computer (`MAVLINK_MODE_ONBOARD`) the global position is streamed at 50 Hz, attitude at 100 Hz, the local position at 30 Hz and battery status at 0.5 Hz [5]; these four rates were read again in the source of release v1.17.0 when it was built for Section 7.5. The 50 records per second of the adopted operating point is that position rate.

ArduPilot. Its default stream rates are set per vehicle type, and for the multicopter firmware they default to zero [6]. An earlier draft of this work said ArduPilot "defaults to 1 Hz"; reading the parameter table corrected it. No single telemetry rate describes "an autopilot", which is why the operating point is treated as a variable in Chapter 8 and not as a constant.

The standard. 3GPP specifies direct UAV-to-UAV local broadcast as a service, with requirements that bound the same quantities this thesis optimises [7]: at least ten messages per second, an end-to-end latency of at most 100 ms, and a payload of 50–1500 B "not including security-related message component(s)". The last clause matters here. The standard itself separates the bytes of authentication from the payload it sizes, and says nothing about how large they may be; that silence is the gap this thesis measures.

How small the messages are. Rotta and Mykytyn, designing authenticated telemetry broadcast for a swarm, observe that MAVLink telemetry messages are "much smaller than 256 bytes" [8]. The record of Chapter 3 — time, position, velocity, battery, mode — is 24.0 B standing alone and 9.0 B as a difference from its predecessor.

2.1.2 How such networks are usually evaluated

Simulation in ns-3 is the common instrument, and a recent swarm protocol stack shows what a typical scenario states: 20–100 nodes in a $400 \times 400 \times 1000$ m volume, Random Waypoint mobility, Nakagami fading and an 802.11ah MAC [9]. It does not state the speed range or the pause time of its mobility model, the two parameters that define what that model did. The observation is not a criticism of one paper. Kurkowski, Camp and Colagrosso surveyed the 151 papers of one ad hoc networking conference over six years: 114 used simulation, none of those said its code was available, and 87.5% of those with plots showed no confidence intervals; of the papers with mobility, half stated the pause time. They concluded that fewer than 15% were repeatable [10]. A decade later Cavalcanti *et al.* found vehicular-network research little better [11]. Chapter 11 is this thesis's answer to that record: every result is regenerated from stored runs, every simulated capacity carries an interval, and the parameters that define each scenario are in the result files' headers.

Two things follow for the design of this study. Capacity is reported for *one collision domain with static nodes*, stated as such, because the contention model has no position term and a mobility model would add parameters without adding a mechanism (Chapter 9); a separate scenario, with every parameter stated, then spreads and moves the nodes to test what that choice hides (Section 8.5.5). And the telemetry itself is synthetic, checked against public flight logs and against the autopilot run in simulation (Section 7.5), because a public corpus of 50 Hz swarm telemetry was not found.

2.2 Tamper-evident telemetry and hash-chained ledgers

Linking each record to the hash of its predecessor makes an append-only log tamper-evident: altering a record invalidates every subsequent link. The construction is Haber and Stornetta's [12]; Bitcoin made it the spine of a blockchain [13]. UAV-blockchain systems, surveyed in [14], place telemetry or attestations on such a chain, gaining integrity at a per-record overhead cost. This thesis targets exactly that substrate cost and is agnostic to the consensus layer above it; finality and witnessing semantics are out of scope.

2.2.1 What the chain gives and what the signature gives

The two mechanisms answer different questions, and the threat model of Chapter 3 needs both.

A *hash chain* fixes order and content relative to one value. Whoever holds the hash of the latest record can detect any change to, removal of, or insertion among the records before it; they need trust nothing but that one hash. The chain says nothing about *who* wrote the records: anyone can build a consistent chain over data of their choosing.

A *signature* fixes origin. A record signed by a vehicle's key was produced by a holder

of that key, and a third party can be shown so. A signature on a single record says nothing about the records around it: a signer can sign two different records for the same position in the sequence, and each verifies.

Together they give what neither gives alone. A signed record that names its predecessor's hash commits its signer to *one history*; two validly signed records that claim the same sequence number are then evidence of equivocation that anyone can check, because both carry the signer's own signature. That property — transferable evidence of inconsistency — is why this thesis requires a public-key signature and does not accept a shared-key tag in its place, and it is the property lost by the cheaper per-message schemes of Section 2.4.1.

It also explains a choice in the lean frame (Chapter 3). Within one frame the records are signed together, so their order and content already rest on the signature; the link each record would carry to its predecessor *inside the same frame* can be recomputed by the receiver and need not be sent. Only the link out of the frame, to the last record of the frame before, carries information the signature does not. The stored ledger is unchanged.

Whether a protocol delivers such properties against an attacker who controls the network is a question for symbolic verification. The Tamarin prover [15] takes the attacker to be the network itself, able to read, block and inject any message, and proves properties of every trace over an unbounded number of protocol runs. Section 3.4.3 uses it for the frames of this thesis.

2.3 Signatures for constrained authentication

ECDSA [16] yields 64 B P-256 signatures at 128-bit security but is randomness-sensitive. Ed25519 [17, 18] uses a twisted-Edwards curve with deterministic nonces for fast, misuse-resistant, batchable verification. BLS [19] gives the shortest signatures via pairings, and [20] aggregates n signatures into one at the cost of expensive pairing verification.

AUTHBC introduces *no new primitive*. It quantifies *where and when* each scheme wins once placement and batching are fixed (Chapter 8), finding Ed25519 dominant for self-batching and BLS worthwhile only for cross-signer relay traffic.

Independent work on UAV swarms adopts precisely the design this thesis treats as its naive baseline: Rotta and Mykytyn attach a HMAC-256 signature with a timestamp to *each* broadcast telemetry message [8] — corroborating the baseline from outside this work. Their construction is a symmetric MAC under a group key, so it provides integrity and group authenticity but *not* non-repudiation, and therefore cannot underwrite a ledger attributable to a specific UAV. That is a different trust model, not a cheaper solution to this one.

2.4 Authenticating messages one at a time, and signing streams

2.4.1 Per-message practice

MAVLink 2 signing appends a link identifier, a 48-bit timestamp and a 48-bit truncated SHA-256 tag computed with a shared secret [21]: thirteen bytes per message. It is cheap and it authenticates a *link* — any holder of the key can produce any message — so a record cannot be attributed to one vehicle afterwards. Vehicular broadcast uses public-key signatures: an ECDSA signature on each message, with the signer's certificate in some messages and an eight-octet digest of it in the others [22, 23]. That is the trust model adopted here, and the per-message cost this thesis starts from.

2.4.2 Amortising one signature over many packets

This is a classical problem and the closest prior art to batching. Gennaro and Rohatgi sign the first packet of a stream and chain the rest by hashes [24]; one lost packet breaks the chain. Wong and Lam sign a hash tree over a block and send in every packet the hashes needed to reach the root [25], so each packet verifies alone — the *signing cost* is amortised, the bytes are not. EMSS tolerates loss by carrying several hashes per packet with a periodic signature packet, and TESLA replaces the signature by a MAC whose key is disclosed later [26, 27]; TBRD applies TESLA to UAS Remote ID and reports about half the overhead of signatures [28].

A further family recovers from loss with erasure codes. Pannetrat and Molva hash every packet of a block, sign the block, and spread the hashes and the signature over the block's own packets with two erasure codes, so that the block verifies whenever no more than a chosen share of its packets is lost [29]. The overhead is small where blocks are long: with a 128 B signature and 16 B hashes, 2 B per packet for blocks of 128 packets that may lose 5% of them, and 10 B for blocks of sixteen. The price is a latency of one block at the receiver, or a buffer of one block at the sender, and a packet that cannot be verified without others of its block. They compare with an earlier scheme of the same family, SAIDA, which was not obtained for this thesis.

All of these spread *one authenticator across packets*, and so make packets depend on each other or on a later one. This thesis does the opposite: it puts *several records in one packet*, which is possible only because a telemetry record is a small fraction of a frame. Each frame then stays self-verifying and the cost moves to latency, at the sender. TESLA instead delays verification at the receiver, and a disclosed MAC key gives no non-repudiation, which a ledger needs. The lesson of the stream-signing literature that does transfer is the one this thesis learned late: a unit that depends on its predecessor inherits its predecessor's loss (Theorem 4.6). Section 8.4 places each of these schemes at

this thesis's operating point, by authenticator bytes and by frames per second.

2.4.3 A standard that faces the same disproportion

Broadcast Remote ID is the nearest standardised case of a signature that does not fit its frame. Over Bluetooth 4 a Remote ID frame carries 25 octets, and the IETF's authentication formats for it use a 64-octet signature [30]. The standard resolves the mismatch in the two ways this thesis sets aside. It *pages* an authentication message over several frames, 23 octets of it to a page and at most nine pages of signed content, and adds one parity page from which a single lost page can be rebuilt. And it *defers*: a signed manifest carries short hashes of up to eleven messages already sent, with the hash of the previous manifest among them, so that a stream is authenticated after the fact for less than its own size again in overhead, and the manifests form a chain.

Both choices suit their setting. Remote ID messages are sent about once a second to observers on the ground, a signed unit a few seconds late is still useful, and one parity page is cheap insurance. They are also exactly the relaxations under which the exclusion of Chapter 10 does not hold: a signed unit spread over frames, and recovery from loss. The difference in regime is the point. At fifty records a second under a deadline of a tenth of a second there is no time in which to collect nine pages, and a record that is verified only when a later manifest arrives is, for that deadline, a record not yet verified.

Civil aviation has the same disproportion in a harsher form. An ADS-B message on 1090 MHz is 112 bits, of which 56 are free for data and are normally occupied by position. Strohmeier, Lenders and Martinovic survey the proposals to secure it: a public-key scheme must distribute one signature over several messages, which receivers buffer until the last has arrived, or change the message format [31]. That is once more a signed unit spread over frames.

2.4.4 Certificates

A signature is checked against a public key, and a receiver must learn that key from somewhere. Vehicular practice attaches the signer's certificate to some messages and a short digest of it to the rest [22, 23]; Chapter 8 charges that policy to every frame. IEEE 1609.2 defines two kinds of certificate. An *explicit* certificate carries the verification key and the issuer's signature over it. An *implicit* certificate (ECQV) carries only a reconstruction value from which the key is derived using the issuer's root certificate, and is shorter: a signed message is at most 226 B with one against 330 B with an explicit certificate [23]. This thesis prices explicit certificates only. The certificate figures of Chapter 8 are therefore upper bounds, for the baseline and the design alike.

2.5 Batch and aggregate verification

Zhang *et al.* [32] let a roadside unit verify multiple received signatures simultaneously, motivated by the same pressure from the opposite side: a hard deadline — DSRC's 300 ms — within which sequential verification does not scale. Their batching is *receiver-side* and buys verification time; the batching in this thesis is *sender-side* and buys airtime. The two are complementary rather than competing. The nearest FANET work is of the same kind: Rajasekaran *et al.* [33] authenticate a group of drones to a user in one batch check of a pairing-based scheme, and report the communication cost of n authentications as n times that of one ($1184\,n$ bits) — the verifier's work is shared, the bytes are not. This work generalises from receiver batching to authentication *placement*, co-designs it with encoding and scheme, and adds a loss-robustness frontier that pure aggregation does not address.

Certificateless aggregate signature (CLAS) schemes for vehicular broadcast are the closest external comparator. Li *et al.* [34] tabulate the per-message size of their own scheme and four others, for a 67 B traffic message: 583 B (theirs, and Cahyadi *et al.*), 596 B (Xu *et al.*), 735 B (Liang *et al.*) and 859 B (Wang *et al.*). Those totals are Li *et al.*'s accounting of what one signer sends with each message. The four other papers were then read for what they say of size themselves. Cahyadi *et al.* give the same figure as the table, 583 B for one message and $583\,n$ for n [35]. Liang and Liu, like Wang *et al.*, work with group elements of 128 B [36]. Xu *et al.* state no size in bytes, so their 596 B is Li *et al.*'s count [37].

What a signer sends for n messages is n times what it sends for one, since each message carries its own identity, key and signature components. Aggregation acts after that. In the scheme of Wang *et al.* a vehicle signs, a roadside unit aggregates the signatures it has received into one, and a traffic management centre verifies the aggregate; a single signature is three group elements and a timestamp, 388 B, and the aggregate of a hundred is 784 B [38]. So aggregation compresses the verifier's work and the link *from* the aggregator; it does not touch the link on which each signer first sends. **The finding is the axis, not a ratio**: batching compresses what one sender puts on the air for its own records, and aggregation compresses what an intermediary passes on for many senders — placements B and C of Chapter 3. A FANET in which every node broadcasts its own state to its neighbours has the first link and no aggregator on it. No claim is made to beat those schemes. They provide conditional privacy, which this work does not, their group elements are 128 B against a 64 B signature here, and their figures are for a different message.

What these schemes are for is worth stating, since it is not what this thesis is for. In a certificateless scheme a key-generation centre issues part of each signer's private key and the signer adds a secret of its own, which removes certificates without giving the centre the whole key; the aggregate property lets a verifier check many signatures at once. Li *et al.* show that one such scheme, by Cahyadi *et al.*, falls to a public-key replacement attack

and to a malicious key-generation centre, and repair it [34]. That is the literature's centre of gravity: constructions, attacks on them and proofs, with communication cost reported as a per-message size and never as what a channel can then carry.

2.6 Compact serialisation and delta coding

CBOR [39] and MessagePack [40] give compact binary encodings. Delta coding of telemetry time series is established practice, most prominently in Gorilla [41], which stores delta-of-delta timestamps and XOR-folded values for roughly a 10× reduction.

The variant used here differs in one respect the link forces. Gorilla is a database and can chain deltas indefinitely. On a lossy broadcast link a record coded against one the receiver never got cannot be decoded, and a frame that cannot be decoded cannot be verified (Theorem 4.6). The first design of this work re-anchored on a self-contained record every $K = 16$ records, and missed its own verifiability target for that reason; the design reported here starts *every frame* with a self-contained record and codes only the records behind it in the same frame (Section 7.4).

2.7 Channel models for IEEE 802.11

Bianchi's fixed-point analysis of the DCF [42] is the standard model for saturated unicast throughput. Broadcast is *not* that model with the acknowledgement removed — a point Ma and Chen make explicitly, and which this project rediscovered expensively. Their broadcast model [43, 44] accounts for the backoff-counter Consecutive Freeze Process, which dominates when the contention window is small relative to the station count. Tinnirello, Bianchi and Xiao [45] supply the anomalous-slot refinement that explains a residual small-frame bias observed in Chapter 9.

Both are models of a saturated channel, in which every station always has a frame to send, and Ma and Chen name non-saturated traffic as future work [44]. The capacities of this thesis lie well below saturation: each node offers one frame per period and the question is how many of them are lost. That regime has been analysed for vehicular safety messages. Ma, Chen and Refai extend the broadcast model to non-saturated traffic on a highway — messages arriving at random at each vehicle, a queue per vehicle, hidden terminals, fading and vehicles that move out of range — and obtain the share of the receivers in range that get a packet; one of its loss terms is the simultaneous transmission of stations within range [46]. Cao *et al.* treat the periodic case in a fully connected network: a packet is sent at once if it finds the medium idle and after a backoff in a fixed window if it does not, and a fixed point gives the delivery ratio [47]. Both check the analysis against a simulation the authors wrote. Section 8.5.3 builds on the same mechanism and adds a second, which neither has: the few microseconds in which a transmission that has begun cannot yet be

sensed. Two further analyses that Cao *et al.* cite were not obtained.

When stations do not all hear one another, a further loss appears: a receiver can be reached by two senders that cannot sense each other. Hu and Dai analyse prioritised safety broadcasts with such hidden terminals, Nakagami fading and distinct transmission, carrier-sensing and interference ranges [48]. The spread swarm of Section 8.5.5 loses capacity, counted in neighbours, by the mechanism their model contains: a receiver contends with every node within its carrier-sensing range, not only with those it counts as neighbours.

2.7.1 Frame aggregation

That every frame pays a fixed cost in preamble, inter-frame space and backoff, and that the cost is best paid once for several payloads, is the principle of frame aggregation in IEEE 802.11n. Skordoulis *et al.* describe its two forms and simulate a point-to-point link on which aggregation raises the saturated throughput about four and a half times for packets of 1000 B [49]. Both forms carry frames for one receiver. The subframes of an A-MSDU must share the receiver's address, and the authors say in terms that broadcasting or multicasting is not allowed; the frames of an A-MPDU must be addressed to the same receiver as well. A broadcast sender cannot use either. Batching records at the application is how this thesis obtains the same effect for broadcast frames, and Chapter 8 shows it to be the larger part of the capacity gain: sharing a frame alone takes the capacity from 35 to 76 nodes before any signature is shared. One detail of the standard is this thesis's freshness constraint in another setting: an A-MSDU is closed when it is full or when the delay of its oldest packet reaches a set value.

2.8 LoRa and LoRaWAN capacity

Haxhibeqiri *et al.* [50] provide the only published LoRa capacity model found that is grounded in hardware interference measurements rather than an analytical collision assumption, and state it in closed form so it can be evaluated at this work's operating point rather than merely quoted at theirs. Zirak *et al.* [51] supply the only air-to-air LoRa PDR-versus-range field data located, which range-limits the capacity result of Chapter 10.

2.8.1 Limits that are rules, not physics

Two of the limits that shape Chapter 10 are set by documents and not by the radio. The LoRa physical layer carries up to 255 B in a frame [52]; the payload limits of 51, 115, 222 or 242 B that the exclusion bound is computed against are those of LoRaWAN's regional parameters for the EU863–870 band, one per data rate [53]. A system that used the LoRa radio without the LoRaWAN link layer would not be bound by them, and the exclusion is stated with that condition. The second is the duty cycle: the same document limits

a device's transmit time in the band to less than 1%, which fixes how many records per second a node may send whatever its neighbourhood. The link layer itself is specified in [54], which also requires an end device to vary its transmit times pseudo-randomly "to prevent systematic synchronization of populations of end-device transmissions" — the standard's own statement of the artifact that strictly periodic simulated senders produce (Chapter 11).

2.9 Where this thesis sits

The adjacent literature almost uniformly proposes a *new construction* and evaluates it against its predecessors. This thesis deliberately proposes none. Its contribution is a boundary: the links on which a frame that verifies alone cannot be sent at any batch size, stated with the conditions under which that holds. For a given authenticator the result is arithmetic rather than empirical, so unlike a performance figure it does not move as models or hardware improve. It does move with the authenticator — a 48 B signature or a 13 B tag fits where a 64 B signature does not — and Chapter 10 gives the matrix. It is invisible to the byte models the literature optimises against.

Chapter 3

System Model and Threat Model

This chapter fixes what the rest of the thesis reasons about: the network, the ledger, the traffic and its two operating points, the adversary, the design choices, and the two frame formats. A frame as it is sent, byte by byte, is in Section 5.3.1.

Figure 3.1 shows the system as a whole. Each UAV codes its records, chains them and signs them into frames, and broadcasts the frames on a shared channel; each receiver decodes, rebuilds, verifies and stores the frames of its neighbours; and five constraints, one per equation of Chapter 4 and Chapter 8, select the batch, the placement of the signature, the scheme and the keyframe interval.

3.1 Network

N UAVs share an IEEE 802.11 broadcast channel and all hear one another: one collision domain, called here a *neighbourhood* of N nodes, in which each node has $N-1$ neighbours. (OFDM, base rate $R = 6\,\mathrm{Mbit\,s^{-1}}$ for robustness; application MTU budget $M = 1500\,\mathrm{B}$). Frames are lost independently with probability p, an abstraction of fading, collisions and mobility, with $p \in \{0.02, 0.05, 0.10\}$ as a declared sensitivity range. Contention is modelled by the DCF (Chapter 4) and validated against NS-3 (Chapter 9). There is no ground control station in the model: the substrate is peer-to-peer broadcast.

Every capacity in Chapter 8 is computed under this assumption except one family: Sections 8.5.4 and 8.5.5 place the nodes over discs of up to 1 km, with capture, fading and hidden stations, and move them, to see what the assumption hides.

3.1.1 Why broadcast, and why it matters

IEEE 802.11 broadcast frames carry no acknowledgement and are never retransmitted. Three consequences follow, and each shapes a later chapter. A broadcast receiver sees the *raw* channel error rate with none of the ARQ that hides loss on a unicast link, which

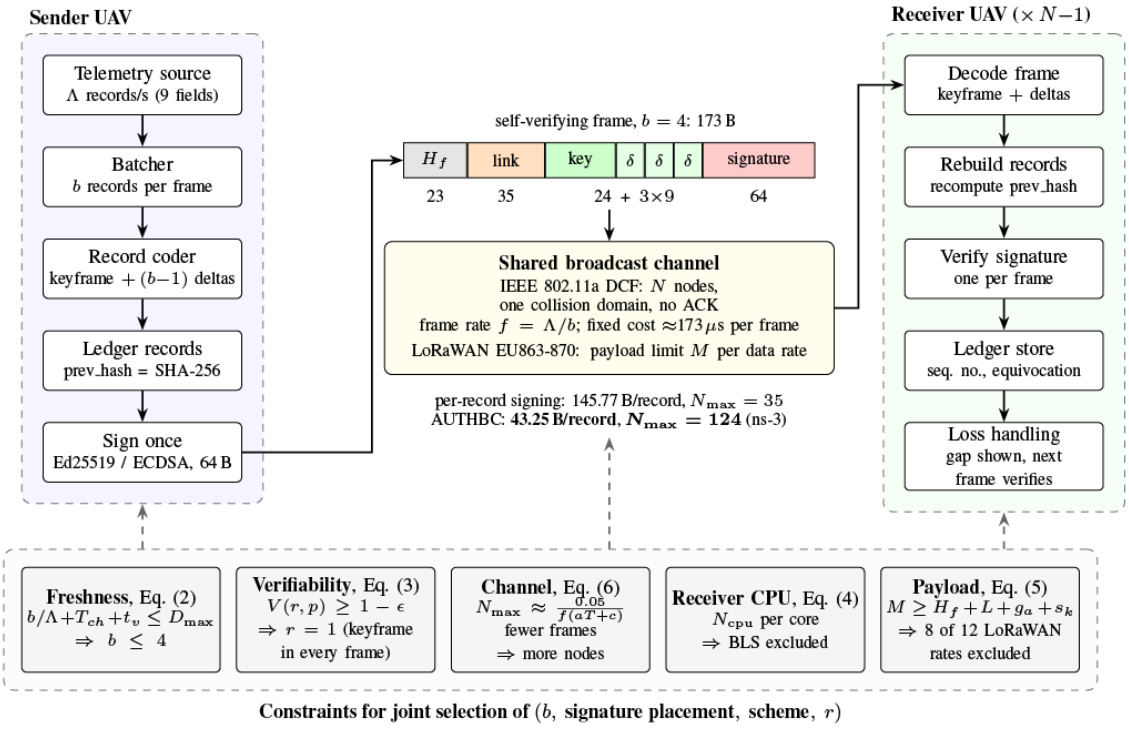


Figure 3.1: System overview: the sender's pipeline, the four-record frame with the size of each field in bytes, the shared channel, the receiver's pipeline, and the five constraints that select the design.

is why the loss grid is one to two orders of magnitude more pessimistic than the 99.9% reliability TS 22.125 specifies for managed command-and-control links *with* retransmission. Overload degrades *delivery*, never latency, because excess offered load is dropped rather than queued. And the channel model is not Bianchi's — Chapter 9 records that using the unicast model with the acknowledgement removed under-predicts simulation by 16×.

3.2 Ledger

Each UAV maintains an append-only hash chain of its own telemetry records and stores verified records overheard from others. A record is

$$\texttt{rec} = \{\texttt{src}, \texttt{seq}, \texttt{ts}, \texttt{prev_hash}, \texttt{payload}\},$$
$$\texttt{prev_hash} = \text{SHA-256}(\text{previous record bytes}).$$

Only the owner appends to its own chain — an owned-object model — so tampering or equivocation on a $(\texttt{src}, \texttt{seq})$ pair is detectable by chain verification. Finality and witnessing semantics are out of scope; the deliverable is *authenticated dissemination*, not consensus.

A ledger is audited from the uploads alone. One pass over the frames of a mission, with

Table 3.1: The two operating points, lean format. Bytes are identical; the neighbourhood is not. $N_{\max}$ is the largest neighbourhood with mean delivery ≥ 0.95, found by direct simulation (Chapter 8).

	Λ_i	$D_{\max}$	b	B/rec	$N_{\max}$, per-record signing	$N_{\max}$, design	TS 22.125
adopted	50	100 ms	4	**43.25**	35	**124**	yes
alternative	20	250 ms	4	43.25	85	306	no

the public keys of the senders and nothing else, finds every equivocation and every frame that does not verify, and the auditor signs a Merkle root of what it accepted. For three UAVs over a flight hour, 391 495 frames, a planted equivocation and a planted forgery were both found, with no false finding, in 6.4 minutes of one core. Keeping the frames costs 42.9 B per record.

3.3 Traffic and the two arrival rates

Records are generated at rate Λ_i per UAV. The aggregate arrival rate in a neighbourhood of N nodes is $\Lambda = \Lambda_i\,(N-1)$ — the rate a *receiver* must verify.

Remark. These are different quantities and conflating them is a real defect, recorded as such. A node batches its *own* records, so freshness uses Λ_i; it verifies *everyone's*, so the verification-throughput constraint uses Λ. Using Λ_i for both silently models a one-sender network and makes cross-signer aggregation look feasible at fleet sizes where the receiver cannot keep up.

3.3.1 The adopted operating point

$\Lambda_i = 50$ rec/s at $D_{\max} = 100$ ms is adopted. It is PX4's `MAVLINK_MODE_ONBOARD` companion-computer rate, read from autopilot source rather than from a secondary summary, and it is fully compliant with 3GPP TS 22.125, clause 5.2.2, which specifies at least 10 messages per second and at most 100 ms end-to-end latency for direct UAV-to-UAV local broadcast.

The alternative (20 rec/s, 250 ms) point is retained and reported because it gives *identical bytes*. The batch is bounded by the fill time, $b/\Lambda_i \leq D_{\max}$, so only the product $\Lambda_i D_{\max}$ matters, and it is 5 at both points. Five records would fill in exactly $D_{\max}$ and leave no time for the channel or for verification, so the largest batch that meets the deadline is $b = 4$ at both (Section 4.10), and the frame is the same.

3.3.2 The floor of the region, which is a real constraint on the application

Holding $b \geq 4$ under a 100 ms deadline requires $\Lambda_i > 40$ Hz. At 20 or 10 Hz the same deadline forces $b = 1$ and nothing is left of the saving but the choice of encoding. **The**

batching benefit is not available at every compliant operating point: it requires a telemetry rate fast enough to fill a batch inside the deadline. More structurally, any batching at all ($b \geq 2$) needs $\Lambda_i > 2/D_{\max}$ — at the standard's 100 ms bound, twice the standard's own minimum message rate. At an ArduPilot Copter default of 0 Hz, where streams are requested by a ground station rather than pushed, this thesis's mechanism does not apply at all.

3.4 Threat model

3.4.1 The adversary

The adversary controls the radio. It can read, inject, replay, modify, delay and drop frames, and may operate any number of radios of its own. It does not hold any honest UAV's private key; key compromise is out of scope by charter.

3.4.2 What is claimed, and where each property is enforced

property	meaning	mechanism
origin authentication	a stored record was produced by the UAV named in `src`	public-key signature over the canonical records, checked before anything is stored
integrity	no field of a stored record was altered in transit	the same signature: every one of 1 392 single-bit flips of a design frame is refused
replay rejection	a frame delivered twice is stored once	`seq` strictly increasing per sender
equivocation evidence	two validly signed records for one (`src`, `seq`) are detected and the two frames that carried them are kept	each frame verifies alone under the sender's public key, so the pair is proof to a third party
continuity	omission or re-ordering within what is held is detectable	each record's `prev_hash`; one link per frame commits to the record before it
non-repudiation	a sender cannot later deny a record	follows from a public-key signature, for whoever holds the frame; a symmetric tag would not give it
bounded-delay freshness	while GNSS time is held, a frame whose signed timestamp is more than Δ before the receiver's clock is refused	a window on the signed timestamp (Section 4.10.2)
outsiders' floods	the frame of a node outside the swarm is refused before it is decoded	an 8 B tag under a key all members hold, checked first

Remark (What the receiver keeps, and what it did not). The last two rows are properties of the frame on air. They become evidence only where the frame is kept: one signature covers a frame, so a record cannot be checked without the other records of its frame and the signature. Until 2026-10-09 the receiver of this prototype kept verified records and discarded the frame; the pair it retained on an equivocation was a pair of records, and nothing it held could be checked by anyone else. A review of the paper found it, and a test that had passed throughout: it counted the pairs and never asked whether one could be checked.

The receiver now keeps every accepted frame as it arrived, and both frames of an equivocation. Two functions are what a third party runs: one returns the records of a frame that decodes and verifies under the sender's public key with nothing else, the other

says whether two such frames give one sequence number two different records. Neither reads any state of the receiver. The cost is storage: a frame's length on air for every b records held, 43.25 B per record in the design, beside the record itself. A frame that is coded against its predecessor cannot be shown alone; the design sends a keyframe in every frame, so every frame can.

Fixing this found a second defect. The receiver offered a frame's records to the store in order and let the first refusal decide the frame. A frame whose first record repeated one already held was therefore refused as a replay, and a different record signed for a later sequence number in the same frame was never seen. Every record of a verified frame is now compared with what is held before any is stored.

3.4.3 Formal analysis, and the evidence for each property

The frame protocol is modelled in the Tamarin prover [15]: a sender that signs frames with increasing sequence numbers and a link to its previous record, a receiver that accepts in the order of Section 5.4, and an attacker that is the network. Origin and integrity, the rejection of replays, their combination as injective agreement, the evidence of an equivocation (which implies a revealed key) and the detection of a skipped frame are proven for unboundedly many frames in a model without the chain and the clock. Continuity is proven in a model with the chain. The time window is shown in a model with the clock: without it, a frame held back by the attacker is accepted, and with it no such trace exists.

Three models, not one. A single model with the chain and the clock did not terminate within a gigabyte of memory per lemma. The registered model settled three of its nine lemmas; the counters were then keyed by a session identifier and the receiver given a start rule, and both changes are recorded with the registration (Appendix B). That the properties carry from the smaller models to the full protocol is an argument — the smaller model's receiver checks less and so accepts a superset — and not a proof. Canonical encoding is assumed injective.

Table 3.2 sets each property beside its evidence, the proofs with the simulations and measurements of Chapter 8.

3.4.4 What is not claimed

Each of these is outside the design, not an oversight in it.

Confidentiality. Telemetry is broadcast in clear.

Availability. Jamming removes frames, and nothing here recovers them: there is no retransmission and no coding across frames. Loss appears as the *robustness* measure V, not as a security property. A flood is resisted only from outside the swarm: a

Table 3.2: Security properties and threats, the mechanism for each, and the evidence: proofs in Tamarin, simulated clocks, and floods against one Raspberry Pi 4 receiver core.

property or threat	mechanism	evidence	result
origin and integrity	signature over the frame	Tamarin, model without chain and clock	proven
replay	increasing sequence numbers	same model	proven
both together	both	same model	injective agreement proven
equivocation	both frames kept	same model	proven: it implies a revealed key
a gap in a sender's frames	sequence numbers	same model	detection proven
continuity	chain link per frame	Tamarin, model with the chain	proven
delayed frame	GNSS-time window Δ	Tamarin, model with the clock simulated clocks, 72 000 honest frames per setting	accepted without the window, refused with it with $\Delta = 150$ ms no honest frame refused at clock errors up to 10 ms; every frame held 250 ms or more refused; a 250 ms window outlasts 75 min without GNSS
flooding by outsiders	8 B group tag	UDP flood, 41 senders on one core	$V = 0.984$ at 3 000 forged frames/s; the tag costs 3.1% of capacity
flooding by a member	none	same	$V < 0.95$ from 129 forged frames/s
ledger audit	signed frames uploaded	three UAVs, one flight hour, 391 495 frames	both planted faults found, no false finding, in 6.4 min of one core

member holding the group key can still make a receiver verify forged frames, and at 129 of them a second one core no longer keeps up (Section 8.5.8).

Timeliness without GNSS time. The window refuses a held-back frame only while sender and receiver share GNSS time. A frame delayed by less than Δ is accepted, authentic and in order, and so is a later one once GNSS time has been lost for longer than the window allows: two clocks part by 144 ms an hour (Section 4.10.2).

Truthfulness. A UAV holding a valid key can sign false telemetry. Only *inconsistent* falsehood — two records for one sequence number — is detectable.

Key distribution, revocation and admission. Receivers are assumed to hold each sender's public key. The on-air cost of delivering certificates is charged separately (Chapter 8); who may join and how a key is withdrawn is out of scope, and so is Sybil resistance.

Privacy. Senders carry a stable identifier. The certificateless aggregate schemes of the vehicular literature buy conditional privacy that this design does not offer.

3.4.5 Behaviour under loss

Every frame of the design decodes alone (Theorem 4.6), so a lost frame costs its own records and nothing else. The receiver then holds a chain with a gap: the next frame's link commits to a record it does not have, which it records and cannot check. Continuity across the gap is therefore *provable later*, if the missing records are ever obtained, and *unknown until then* — neither violated nor verified.

Remark. Availability under loss is a *robustness* metric in this thesis, not a security claim. The quantity V, the probability that a record is verifiable at a receiver, is treated throughout as a design constraint, not as a guarantee against an adversary who controls the channel.

3.5 The decision space

3.5.1 Authentication placements

A — inline per record. Every record carries its own signature. The baseline.

B — self-batch. One UAV packs b of its *own* records into a frame and signs the frame once. Per-record authentication cost $(g_a + H_f)/b$. Self-contained, hence loss-local.

C — cross-signer aggregate. A frame carries b records from *different* originators, each originally signed; the signatures aggregate into one. Required only when forwarding others' records or combining attestations.

D — block-level aggregate. One signature over b records spanning $n > 1$ frames. All-or-nothing under loss; Chapter 4 shows when this is dominated.

3.5.2 Schemes and encodings

Signature schemes are ECDSA-P256 (64 B, legacy baseline), Ed25519 (64 B, fast) and BLS12-381. The BLS object is 96 B, not the 48 B minimum-signature variant originally specified: the available library ships only the minimum-pubkey scheme. That deviation is recorded and applied consistently, and it makes BLS lose the byte comparison outside multi-signer aggregation.

Encodings are JSON (baseline), canonical CBOR, MessagePack, and delta-CBOR. In the first format the delta encoder emits a keyframe every $K = 16$ records, a value that was fixed and never optimised. The lean format makes the interval a parameter — one self-contained frame in every r — and Section 7.4 evaluates it; the design uses $r = 1$.

3.6 Two frame formats

The thesis reports two formats side by side, and the reason is historical. The *first* format is the one implemented at the start: canonical CBOR with text keys, and in every record its own chain link, sender identifier and sequence number. It is frozen, and every artifact of chapters 7–9 that predates October 2026 was produced with it. The *lean* format was built after an external review pointed out that the design the results headlined — delta records under one signature in a short header — had never existed as one object: it was a sum of sizes measured in three places (Chapter 11). The lean format is that design as code, with a sender and a receiver. The sizes reported for its design and its baselines are lengths of frames that were emitted, decoded and verified; the one lean size that is a sum of measured parts (the design without delta coding) is marked in Table 8.1.

3.6.1 The lean frame

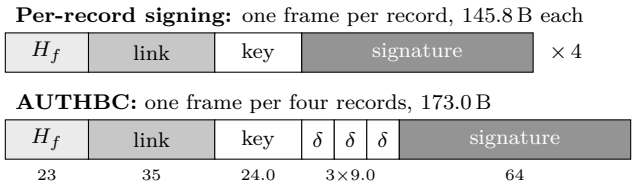


Figure 3.2: The lean frame, to scale (bytes below). Per-record signing repeats header, chain link and signature with every record; the design carries them once per four records, and the three records after the first are coded as differences (δ).

Figure 3.2 draws the frame to scale. A frame is one canonical CBOR map with integer keys. `src` and `base_seq` name the sender and the sequence number of the first record;

Table 3.3: Byte budget of a frame, field by field, for a sender with a five-digit identifier one hour into a 50 Hz flight (smaller values earlier in a flight in parentheses).

field	content	first	lean
(map)	CBOR map header	1	1
`v`, `t`	version; frame type	6	4
`src`	sender identifier, 16 bit	7 (5)	4 (2)
`base_seq`	sequence number of the first record	14 (10)	6 (2)
`n`	records in the frame	3	2
`recs`	key and length of the records	6	3
`auth`	key and length of the signature	7	3
header H_f		**44** (38)	**23** (17)
chain link		32 per record	35 per frame
signature g_a (Ed25519, ECDSA)		64	64
record that decodes alone (keyframe)		59.9*	24.0
record coded against its predecessor, 50 ms apart		44.0*	9.0

* includes that record's own 32 B chain link.

`link` is the hash of the record before it; `recs` is the record stream and `auth` one signature. A record in the stream is nine variable-length integers — timestamp, position, velocity, battery, mode. The first record of a frame is a *keyframe* carrying absolute values; each later one is a *delta* from the record before it. Sender and sequence number are not repeated per record.

What is signed is not the stream. It is the canonical ledger records, each with its own `prev_hash`, exactly as in the first format. The receiver rebuilds them — the first `prev_hash` is `link`, each later one the hash of the record just rebuilt — and only then checks the signature. A frame that cannot be decoded therefore cannot be verified, and Theorem 4.6 counts that. The state against which a later frame would be decoded advances only after a frame has been verified *and* stored, so a replayed frame cannot desynchronise a stream.

What one link per frame gives up. In the first format every record carries its own `prev_hash`; in the lean format only the first record of a frame does, and the rest are derived. Inside a frame, order and content then rest on the frame signature alone, where the first format has a signature and an independent hash. A frame is signed and accepted as a unit, so no strength is lost — but one mechanism remains where there were two. The decision of July 2026 had scoped per-frame chaining to the low-rate arm; the lean format extends it to both, and that extension is recorded as a decision, not presented as a neutral encoding choice.

Table 3.4: H_f is not a constant. Canonical CBOR encodes integers 0–23 in one byte and those ≥ 65536 in five, so the measured header moves with field magnitude.

`src`	`base_seq`	first format	lean format
0	0	**38 B**	**17 B**
24	24	40 B	19 B
256	256	42 B	21 B
40 000	180 000	**44 B**	**23 B**

3.7 The frame header, which is a range

H_f feeds the batching theory, the exclusion bound, the batch bounds, the channel constraint and the energy model. It is measured from the implemented wire formats:

$$H_f = \text{len(frame)} - \text{len(records)} - \text{len(link field)} - \text{len(signature)},$$

the link term being zero in the first format, where the link lives inside each record.

The values 44 and 23 B are used throughout; they are the steady-state values for any realistic flight and the conservative end for every byte comparison.

Remark (A claim this section used to make). Earlier versions said the range decides the exclusion bound of Chapter 10: that one data rate was excluded only because the top of the range was used. That was withdrawn in October 2026. The bound it rested on charged neither the chain link nor a record that decodes alone, and with both charged the rate is excluded at every header size, including zero (Section 10.2.5).

Chapter 4

Theoretical Framework

This chapter develops the closed-form results. Each carries its empirical validation hook, and where a result is *applied* rather than novel, that is stated at the point of statement rather than buried in related work.

4.1 T1 — the overhead-dominance threshold

Theorem 4.1 (Overhead dominance)**.** *With inline per-record authentication, the authentication byte fraction is $\varphi(s, g_a) = g_a/(s + g_a)$, and $\varphi \geq \alpha \iff s \leq g_a(1-\alpha)/\alpha$. In particular authentication dominates ($\alpha = \frac{1}{2}$) exactly when $s \leq g_a$.*

Proof. $\varphi \geq \alpha \iff g_a \geq \alpha(s + g_a) \iff g_a(1-\alpha) \geq \alpha s$. □

The threshold is trivial; its *instantiation* is the motivating result. At $g_a = 64\,\mathrm{B}$ and measured record sizes, JSON at 191 B gives $\varphi = 25.1\%$, CBOR at 66 B gives 49.1%, and delta-CBOR at 45 B gives 58.7%. Compression moved the bottleneck from data to cryptography. Validated by E1 (Chapter 7), where a remark states what these record sizes contain: s includes the record's own 32 B chain link, so φ understates the point.

4.2 T2 — frame-level batching and the amplification law

Placement B or C packs b records, one authentication object of g_a bytes and a header of H_f bytes into one frame, subject to $b \leq b_{\max}(s) = \lfloor (M - H_f - g_a)/s \rfloor$.

Proposition 4.2. *Per-record authentication overhead $\omega(b) = (g_a + H_f)/b$ is strictly decreasing, so the byte-minimal feasible batch is $b = b_{\max}(s)$. Relaxing integrality, per-record on-air bytes are $C(s) = s \cdot A$ with amplification $A = M/(M - H_f - g_a) \geq 1$.*

Proof. $\omega'(b) < 0$ gives the first claim. For the second, substitute $b = (M - H_f - g_a)/s$ into $s + \omega(b)$ and factor. □

Worked example. Take the delta encoding of the first format, $s = 45\,\mathrm{B}$, with $H_f = 44\,\mathrm{B}$, $g_a = 64\,\mathrm{B}$ and $M = 1500\,\mathrm{B}$. A frame holds $b_{\max} = \lfloor 1392/45 \rfloor = 30$ records, the overhead is $108/30 = 3.6\,\mathrm{B}$ per record, and a record costs $48.6\,\mathrm{B}$ on air. The amplification is $A = 1500/1392 = 1.078$: a byte saved by the encoder saves $1.078\,\mathrm{B}$ on air, because a smaller record also lets more records share the fixed $108\,\mathrm{B}$. Under the deadline of the next section the batch is 4 and not 30; the overhead is then $108/4 = 27\,\mathrm{B}$ per record, the cost $72.0\,\mathrm{B}$, and a byte saved by the encoder saves exactly one.

So every payload byte saved by compression saves A on-air bytes — *in the MTU-limited regime.* This is MTU-efficiency algebra, not a new law, and Theorem 4.3 shows the regime is never reached on 802.11.

4.3 T2a — which ceiling binds, and what compression is therefore worth

Theorem 4.3 (Binding constraint)**.** *Let* $b^\star = \min(b_{\mathrm{MTU}}, b_{\mathrm{fresh}})$ *with* $b_{\mathrm{MTU}} = \lfloor (M - H_f - g_a)/s \rfloor$ *and* $b_{\mathrm{fresh}} = \lfloor \Lambda_i D_{\max} \rfloor$*. Then*

$$\frac{\mathrm{d}C}{\mathrm{d}s} = \begin{cases} A = \dfrac{M}{M - H_f - g_a}, & \textit{MTU-limited,} \\ 1 \textit{ exactly,} & \textit{freshness-limited.} \end{cases}$$

Freshness binds exactly when $s < (M - H_f - g_a)/(\lfloor \Lambda_i D_{\max} \rfloor + 1)$.

Proof. Differentiate C in each regime; in the second, b is constant in s. □

The consequence is sharp and negative for the 802.11 arm. At $M = 1500$, $H_f = 44$, $g_a = 64$, $\Lambda_i D_{\max} = 5$, the boundary is $s < 1392/6 = 232.0\,\mathrm{B}$. Every encoding studied lies below it — delta 45, MessagePack 65, CBOR 66, JSON 191 — so **freshness binds for all of them and A is never operative on 802.11**. Compression pays exactly $1\times$, and the residual authentication cost is a floor $(g_a + H_f)/b^\star$ that compression cannot touch.

One refinement matters for every number that follows. $\lfloor \Lambda_i D_{\max} \rfloor$ bounds the batch by its fill time alone. When $\Lambda_i D_{\max}$ is an integer, a batch of that size fills in exactly $D_{\max}$ and leaves nothing for channel access, airtime and verification, so the largest batch that meets the deadline is one less. Both operating points of this thesis are of that kind, with $\Lambda_i D_{\max} = 5$: the design uses $b = 4$ (Section 4.10 gives the convention behind the fill time). With $b_{\mathrm{fresh}} = 4$ the boundary rises to $1392/5 = 278.4\,\mathrm{B}$ and the conclusion is unchanged.

On a low-rate link ($M = 242$) the same boundary falls to $s < 134/6 = 22.3\,\mathrm{B}$, below every encoding studied, so there the frame size binds and $A = 242/134 = 1.81$ is operative. **The leverage that motivates compression is exclusive to the low-rate arm**, which strengthens that chapter's motivation.

4.4 T3 — the loss-robustness frontier

Frame-level placements are self-verifiable, so $V_B = 1 - p$ and a loss costs b records. Block-level placement D spans $n(b) = \lceil (bs + g_a + H_f)/M \rceil$ frames under one signature, so $V_D = (1-p)^n$.

Theorem 4.4 (Fragmentation is foreclosed)**.** *The constraint $V \geq 1 - \varepsilon$ admits at most $n_{\max} = \lfloor \ln(1-\varepsilon)/\ln(1-p) \rfloor$ frames. If $\varepsilon \leq p$ then $n_{\max} = 1$: block-level authentication is infeasible beyond a single frame.*

Proof. Solve $(1-p)^n \geq 1 - \varepsilon$ for n. At $\varepsilon = p$ the right-hand side is $1 - p$ and $n \leq 1$. □

Theorem 4.5 (Robustness to correlated loss)**.** *For* any *stationary loss process of mean rate p, whatever its correlation structure,*

$$V_D(n) = P(\textit{all } n \textit{ frames arrive}) \leq P(\textit{one given frame arrives}) = 1 - p = V_B,$$

with equality at $n = 1$ and, for $n > 1$, only in the degenerate case in which the n frames are lost together or not at all.

Proof. A joint probability cannot exceed a marginal. □

This matters more than it first appears. The $(1-p)^n$ form assumes independent loss, which the emulator also implements — so agreement between measured and theoretical V is a *consistency check, not a validation.* Theorem 4.5 rescues the ordering without the assumption: whatever the correlation, D cannot overtake B. Among bursty processes, in which a lost frame makes the next loss more likely, independence is the worst case for D, and correlation narrows the gap without inverting it. Gilbert–Elliott simulation at matched mean $p = 0.05$ confirms it: $V_D(n{=}2)$ rises from 0.9025 (independent) to 0.9498 at a mean burst of 160 frames — asymptotically $1 - p$, never past it, and never reaching the feasibility threshold.

4.5 T3' — a frame coded against its predecessor is not self-verifiable

Theorem 4.4 rests on the premise that each frame verifies alone. **That premise fails for a delta encoding whose keyframes are rarer than its frames**, and the design this thesis first headlined was one: a keyframe every $K = 16$ records with $b = 4$, so that only one frame in four could be decoded without the frame before it. A receiver cannot verify what it cannot rebuild — the signature is over the records, not over the bytes on air — so a lost frame takes every later frame with it until the next self-contained one.

Theorem 4.6 (Verifiability with dependent frames)**.** *Let one frame in r be self-contained. Frame j of a group ($j = 1, \ldots, r$) verifies iff it and the $j-1$ frames before it in the group all arrive, so under independent frame loss p*

$$V(r,p) = \frac{1}{r}\sum_{j=1}^{r}(1-p)^j.$$

$V(1,p) = 1-p$, and $V(r,p) < 1-p$ for every $r > 1$.

Corollary 4.7. *If $\varepsilon \le p$, the requirement $V \ge 1-\varepsilon$ admits $r = 1$ only: every frame must decode alone.*

$V(4, 0.05) = 0.881$. The design as first published therefore missed $V \ge 0.95$ at the loss rate it was specified for. Its byte figure had amortised one keyframe over four frames; its verifiability figure had treated the four as independent; and the two had never been evaluated on one object, because no such frame had been built (Chapter 11). Measured with the encoder and decoder in the loop, $r = 4$ verifies 0.876 of records (Chapter 7).

For bursty loss the same counting gives, for a two-state process with mean loss $\bar{p}$ and mean burst length B frames,

$$V = \frac{1-\bar{p}}{r}\sum_{j=1}^{r}(1-a)^{j-1}, \qquad a = \frac{\bar{p}}{B\,(1-\bar{p})},$$

which reduces to Theorem 4.6 for memoryless loss and is *larger* for longer bursts: losses that arrive together waste fewer good frames. It never exceeds $1-\bar{p}$, with equality only at $r = 1$, so bursts soften the penalty of a dependent design and never remove it.

The corrected design uses $r = 1$. What a longer interval would buy is small — 3.5 B per record between $r = 1$ and $r = 16$ — and a cleaner link would be needed to afford it.

4.6 T4 — scheme selection

For a UAV's *own* records under self-batching, one ordinary signature covers b records, so Ed25519 and BLS are near-equivalent in bytes while Ed25519's verification is far cheaper than a pairing. Ed25519 wins self-batching outright.

For *relayed* records from distinct signers, Ed25519 must carry $b \cdot 64$ B while BLS aggregates. BLS saves $\Delta(b)$ bytes per record but costs Δt extra CPU. BLS is energy-optimal iff

$$P_r \cdot \frac{8\Delta(b)}{R} > P_c \cdot \Delta t,$$

and throughput-feasible iff $t_{\text{agg-verify}}(b)/b \le 1/\Lambda$. At $R = 6\,\text{Mbit}\,\text{s}^{-1}$ the radio side is $\approx P_r \cdot 75\,\mu\text{s}$ against a CPU side of $P_c \cdot (1\text{–}3)\,\text{ms}$, so **Ed25519 wins on 802.11 and BLS's regime is low-rate links and attestation compression** — a falsifiable, regime-dependent rule.

Worked example. Four relayed records share a frame, each from a different signer. Ed25519 carries their four signatures, 256 B; BLS carries one aggregate of 96 B, so $\Delta = 64 - 96/4 = 40$ B per record, which at 6 Mb/s is 53 µs of airtime. On the measured board, verifying the aggregate costs 3.7 ms more processor time per record than verifying an Ed25519 signature. BLS is the cheaper in energy only if $53\,\mu\text{s} \cdot P_r > 3.7\,\text{ms} \cdot P_c$, that is if the radio draws more than 69 times the power of the processor. Measured, the ratio is 0.29 (Chapter 8).

4.7 T5 — an empirical separability result

Remark (Not a theorem). T5 is often written as a co-design theorem. It is not one. The joint optimum is *empirically separable*, verified by exhaustive search over the full grid of 4 encodings × 3 schemes × 4 placements × 32 batches = 1536 points. A non-separable optimum would have been found had one existed; none was. That is evidence, not proof, and the distinction is kept.

The recipe: pick the smallest deterministic encoding $e^\star$, placement B for own traffic or C for relay, batch $b^\star = \min(b_{\max}(s), \lfloor \Lambda_i D_{\max} \rfloor)$ — on 802.11 the second term binds, less one when $\Lambda_i D_{\max}$ is an integer (Theorem 4.3) — and scheme per T4.

A factorial ablation (Chapter 8) narrows this further, and against the project's own earlier claim: only placement and batching genuinely interact, by exactly $g_a(1 - 1/b)$, hence not at all at $b = 1$. Encoding is perfectly separable *in that byte model*, where a record's size does not depend on its neighbours, and the scheme axis is byte-degenerate. In the frame as built the delta saving is $(1 - 1/b)(k - \delta)$ per record for a keyframe of k and a delta of δ bytes — conditional on batching in the same way (Chapter 8).

4.8 T6 — the authentication-exclusion threshold

Remark (Prior art, checked before publication). The inequality below is *established*, not novel. Gündoğan *et al.* compute exactly $M - H_f - g_a = s_{\max}$ for 802.15.4/NDN, and the post-quantum literature reports NIST signatures as incompatible with 5G SIB1's 372 B limit. That a fragmented unit delivers with $(1-p)^n$ is likewise standard in 6LoWPAN. T6 is therefore a *synthesis* of two known facts, presented as an applied threshold. What has not been found stated elsewhere is the *composition*: that the sliced-signature workaround the literature proposes is foreclosed whenever $\varepsilon \leq p$. This check was run *before* publication precisely because an earlier novelty claim in this project was not.

Theorem 4.8 (Exclusion)**.** *A frame that verifies alone must carry a header H_f, one chain link L, an authentication object g_a and at least one record that decodes alone, s_k. A link*

of maximum payload M therefore admits authenticated, hash-chained telemetry iff

$$M \;\geq\; H_f + L + g_a + s_k,$$

each term taken at its smallest, and no choice of batch size, encoding of later records or aggregation changes this: those decide only how the space above *the floor is used.*

Fragmentation does not escape it, by Theorem 4.4: at $\varepsilon \leq p$ the entire verifiability budget is spent on the first frame, so a signed unit may not span two.

Remark (The statement this replaces). Until October 2026 the theorem read $s_{\max}(M) = M - H_f - g_a \geq s_{\min}$, with $s_{\min}$ the smallest record. It charged neither the chain link, which per-frame chaining moves into the frame, nor the requirement of Theorem 4.6 that the record decode alone; $s_{\min}$ had been taken as a mean delta record less its link. The consequences, including a claim this thesis had to withdraw twice, are in Section 10.2.5.

4.8.1 Four tiers, because each names a different escape route

tier	condition	what could fix it
signature	$M < g_a$	a smaller authentication object *only*
header	$g_a \leq M < g_a + H_f$	a leaner frame header
chain link	$g_a + H_f \leq M < g_a + H_f + L$	taking the link off the air
record	$g_a + H_f + L \leq M < g_a + H_f + L + s_k$	a smaller self-contained record

The tier structure is what makes the bound *constructive*: Chapter 10 evaluates it for all twelve EU863-870 data rates and tabulates what each relaxation buys.

4.8.2 Scope

The theorem is conditional, and Chapter 10 states its four conditions in full: the payload limit is that of the link layer in use; every frame carries a public-key signature; there is no loss recovery; and $\varepsilon \leq p$. A fifth belongs to the instantiation and not to the theorem: how many data rates it excludes depends on the header of the frame format, for three of the eight.

4.9 A withdrawn theorem

An earlier version of this framework contained a seventh result, T7, claiming that channel capacity excludes configurations at $U \geq 1$, where U is offered load over *saturation* throughput. **It was withdrawn on the day it was written**, by its own validation experiment.

Saturation throughput is what the channel carries when every node is backlogged; a lightly loaded channel collides far less and carries substantially more. Simulation at $N = 50$ shows 98.8% of frames still delivered at $U = 1.00$, with the verifiability crossing near $U \approx 2.4$. The claimed exclusion does not exist, and a "50% ceiling" derived from it was an artifact of the wrong threshold.

A second structural error compounded the first: the theorem assumed overload manifests as delay. It does not. Broadcast has no ARQ and no queue buildup, so excess load is dropped, not queued — mean delay at nine times saturation is still under 3 ms. **Any capacity argument here must be made in V, not in D.**

What survives is a cost rather than an exclusion: because $b \leq \lfloor \Lambda_i D_{\max} \rfloor$, tightening the freshness deadline shrinks the batch and raises the frame rate the medium must carry. Meeting the standard's deadline costs swarm size — 124 nodes where the relaxed point carries 306 — and *nothing* in bytes. That is the measured statement, and it is a better result than the theorem it replaces.

Remark (The lesson, recorded). T7 was written from a model quantity treated as a physical limit, and published into the documentation before the experiment that tested it existed — an experiment already scheduled. Running it first would have prevented the claim.

4.10 Freshness, and a convention that had to be derived

The freshness of the oldest record in a batch is modelled as $D(b) = b/\Lambda_i + T_{\text{air}} + W_q$ and enforced as a hard constraint.

Remark (Which quantity b/Λ_i is). With periodic arrivals, b/Λ_i is the *batch-window duration* — open to transmit — while the *age of the oldest record* at transmission is $(b-1)/\Lambda_i$, because it waits out the remaining $b-1$ gaps. Both were confirmed exactly by discrete-event simulation under deterministic and Poisson arrivals. The formulation above names the second and computes the first.

It is retained as the *worst case*, and it is exactly that: a record timestamped at its sample instant may describe state up to $1/\Lambda_i$ older, and $(b-1)/\Lambda_i + 1/\Lambda_i = b/\Lambda_i$. So it bounds end-to-end latency including sampling quantisation. The cost of the choice is stated: the tight reading admits $b = 5$ at both operating points (36.4 B per record, −75.0%) where the worst case admits $b = 4$ (43.25 B, −70.3%). **This thesis under-reports its own saving by about 4.7 percentage points.**

The retained quantity has a name in the literature on timeliness. The *age of information* at a receiver is the time since the newest update it holds was generated, and its *peak*, reached the instant before the next update arrives, is the delay of the previous update plus the interval between deliveries [55]. With one frame every b/Λ_i and a frame's newest record delayed by $T_{\text{air}} + W_q$, that peak is $b/\Lambda_i + T_{\text{air}} + W_q$: the $D(b)$ above, exactly, when no frame is lost. The constraint of this thesis is therefore a bound on peak age, where

that literature mostly minimises an average; the survey lists the probability that the peak age exceeds a threshold as a metric of its own. This reading was found on 2026-10-09, after the convention had been chosen. It does not change the convention; it says what the convention bounds.

4.10.1 The peak age under loss

The bound holds for a frame that arrives. A lost frame adds one frame period, $P = b/\Lambda_i$, to the age of the update after it: after k consecutive losses the peak age is $(k+1)P$ plus the delay of the frame that does arrive. With independent loss p the number of periods between deliveries is geometric, and the mean peak age is $P/(1-p) + T_{\text{air}} + W_q$. At the design's $b = 4$ and 50 records/s, simulated frame by frame through the codec at $p = 0.05$, the mean peak age is 87.7 ms, within $D_{\max}$, but its 99th percentile is 169.8 ms: one lost frame is enough to break the deadline, and 4.7% of updates do, the share of frames lost. The deadline therefore holds for a share $1 - p$ of updates, the same share that V counts.

4.10.2 A window on GNSS time

A sequence number refuses a frame delivered twice and accepts a frame delivered late. Every record carries its sampling time, signed with it, and every UAV keeps GNSS time, so a receiver can refuse a frame whose newest record is too old:

$$t_{\text{local}} - t_{\text{newest}} \;\le\; \Delta = \frac{b}{\Lambda_i} + T_{\text{air}} + W_q + t_v + 2\sigma_{\text{clk}} + g, \tag{4.1}$$

with σ_{clk} the clock error between the two nodes and g a guard. The check reads the timestamp before the signature; a frame whose timestamp was altered to look fresh then fails its signature, and one altered to look stale does what dropping it would. A stale frame therefore costs a receiver a decode, not a verification, and the rule costs no byte on air.

While GNSS time is lost each node's crystal drifts, by up to 20 parts per million, so two clocks part by up to 144 ms an hour. A window widened by that drift keeps working, its bound growing at the same rate; a fixed window of 250 ms outlasts 75 min of outage (Table 3.2). Beyond a set limit the receiver stops applying the rule and marks what it accepts as unverified in time, falling back on sequence numbers alone.

4.10.3 A loss budget with a margin

With one self-contained frame in every frame, $V = 1 - p$, so a target $V \ge 1 - \epsilon$ is met with no margin at $\epsilon = p$: half of all runs fall below it by chance. The deployment rule of this thesis sets the budget at least 0.02 above the expected loss, $\epsilon \ge p + 0.02$. Figure 7.4 tests it under five loss processes of one mean.

4.10.4 The design map

The operating point fixes Λ_i and $D_{\max}$; the freshness bound then fixes the batch. Over any pair, the largest batch the deadline admits is

$$b^*(\Lambda_i, D_{\max}) = \max\{\, b : b/\Lambda_i + T_{\text{air}} + W_q + t_v \le D_{\max} \,\}, \tag{4.2}$$

and none where even one record misses the deadline. Figure 8.7 evaluates it over seven record rates and eight deadlines, with the capacity it buys over per-record signing from the access-rule model of Section 8.5.3.

Chapter 5

Implementation

The system is a Python library of about eight and a half thousand lines, the scripts that run it, two simulation scenarios and a hardware harness. This chapter describes what was built and the rules it was built to; Appendix A says how to run it.

5.1 Design rule: one home per fact

The implementation follows one architectural rule that is worth stating because it is what makes the reproducibility claims of Chapter 11 possible. **Every quantity has exactly one definition, and everything else refers to it.** There is one function that produces the canonical bytes a signature covers; one airtime implementation shared by the channel model, the energy model and the simulation comparison; one place where experiment parameters live.

Where that rule was broken, defects followed. A signature-size constant duplicated between two modules produced a stale result that survived a re-freeze. An airtime approximation that existed in two forms produced a 12% discrepancy on acknowledgement frames. And the largest instance was not a duplicated constant but a *missing object*: for three months the design the results headlined existed only as a sum of sizes measured in three modules, with no frame, no sender and no receiver behind it — so the byte model, the verifiability model and the exclusion test each evaluated a slightly different thing and none could be checked against the others (Chapter 11).

5.2 The library

Figure 5.1 shows the packages and which of them use which. Four of them use none of the others — the signature schemes, the ledger, the analytical models and the simulators — and that is deliberate: a model that imported the encoder it describes could not be used to check it.

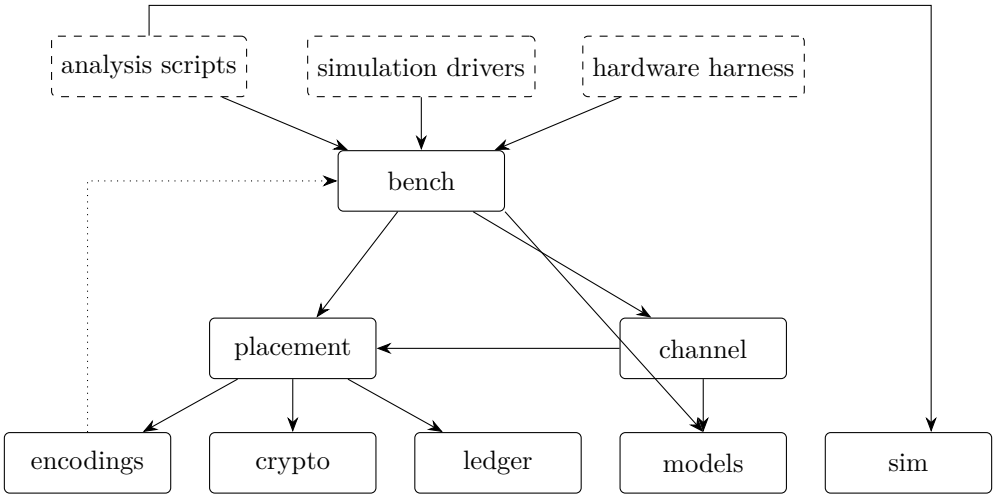


Figure 5.1: The packages of the library (solid) and what runs them (dashed). An arrow points from a package to one it uses; arrows that follow from others are omitted. The dotted arrow is the one upward reference: the encoders take the record type from the telemetry generator.

`encodings/` The e axis: JSON, canonical CBOR (RFC 8949, Section 4.2), MessagePack, and delta-CBOR with keyframes every $K = 16$. Delta encoders are *stateful*; a fresh instance per record emits all keyframes and silently inflates sizes — a trap that cost real time.

`crypto/` The σ axis: Ed25519 and ECDSA-P256 (both 64 B), BLS12-381 (96 B). Known-answer tests from RFC 8032, NIST CAVP and the IETF BLS draft are vendored and must pass before any timing work.

`ledger/` The data structure being authenticated: the canonical record, the hash chain, persistence, and verification with replay and equivocation detection.

`placement/` The placement axis. `wire.py` is the first frame format and the one place that decides *what bytes a signature covers* — the audit boundary; it is frozen. `framer.py` holds its header constant and the batch bounds. `wire_v2.py` is the lean format of Section 3.6: integer keys, one chain link per frame, a variable-length record stream, and the same signed bytes as the first format. `session_v2.py` is the protocol around it — a sender that emits one self-contained frame per reference interval, and a receiver that decodes, verifies, stores and reports what it did with each frame (accepted, desynchronised, malformed, unknown sender, bad signature, replay, equivocation, tampered).

`models/` The analytical layer: the DCF fixed point with exact OFDM-symbol airtime, the published broadcast model, the energy and freshness models, the Pareto search, the scheme crossover, and the low-rate PHY with every constant transcribed from primary specifications. `frame.py` holds the *one* definition of a frame from which the

byte, verifiability and exclusion results are all computed, and whose arithmetic is held equal to emitted frames by test. `phy.py` makes the OFDM timing a parameter.

`sim/` Two simulators written for this work, neither of which shares code with ns-3. A slot-exact backoff simulator of the saturated channel, written *before* the published broadcast model was located and retained as a cross-check that shares none of that model's assumptions (Chapter 9). And an event simulator of the access rule for a lightly loaded channel — arrivals, carrier sense, backoff counters and nothing else — which is the model of Section 8.5.3.

`channel/` Airtime of a frame from its length, and the loss processes of the verifiability experiments: independent, bursty, and growing with frame length.

`bench/` The experiments. Each reads a configuration file, runs the library and writes one result file with a header that records the environment and a hash of the configuration. The telemetry generator of Section 6.4 and the timing rules of Section 6.1 live here.

5.3 Wire format

$$\text{Frame} := \{v, t, \texttt{src}, \texttt{base_seq}, n, \texttt{recs}[\,], \texttt{auth}\}$$

in canonical CBOR, with the authentication block shaped by placement. The signature input is the canonical bytes of the covered region, and test vectors are frozen. The lean format keeps that signature input and changes only how the records travel; its field table is Table 3.3.

5.3.1 One frame, byte by byte

Table 5.2 is a frame as the sender emitted it: the second and third records of one generated flight, an hour in, from a sender whose identifier needs two bytes. It was encoded, decoded and verified by the library, and the table is cut from those bytes by a script, not typed.

bytes on the wire (hex)	field	content	B
`a8`	(map)	a CBOR map of eight pairs	1
`00 02`	v	format version: 2	2
`01 01`	t	frame type: one signature over the frame, decodes alone: 1	2
`02 19 9c 40`	src	sender identifier: 40000	4
`03 1a 00 02 bf 21`	base_seq	sequence number of the first record: 180001	6
`04 02`	n	records in the frame: 2	2
`05 58 20`	link	key, and a byte string of 32	3
`ee 02 8a 23 c1 61 ... 5b fb`		hash of the record before the first one	32

bytes on the wire (hex)	field	content	B
`06 58 22`	recs	key, and a byte string of 34	3
`b5 dd db 01`	keyframe: ts	3600053 ms	4
`bf 9f c8 a3 02`	keyframe: lat	$-305727456\ 10^{-7}$ deg	5
`86 8d b6 b3 02`	keyframe: lon	$322356035\ 10^{-7}$ deg	5
`f4 9d 36`	keyframe: alt	444282 cm	3
`8e 05`	keyframe: vel_x	327 cm/s	2
`b6 04`	keyframe: vel_y	283 cm/s	2
`d5 02`	keyframe: vel_z	−171 cm/s	2
`64`	keyframe: battery	100 %	1
`04`	keyframe: mode	4	1
`66`	delta: ts	+51 ms (now 3600104)	1
`24`	delta: lat	$+18\ 10^{-7}$ deg (now −305727438)	1
`20`	delta: lon	$+16\ 10^{-7}$ deg (now 322356051)	1
`0f`	delta: alt	−8 cm (now 444274)	1
`13`	delta: vel_x	−10 cm/s (now 317)	1
`04`	delta: vel_y	+2 cm/s (now 285)	1
`06`	delta: vel_z	+3 cm/s (now −168)	1
`00`	delta: battery	+0 % (now 100)	1
`00`	delta: mode	+0 (now 4)	1
`07 58 40`	auth	key, and a byte string of 64	3
`c3 d2 b6 a9 fc aa ... a5 05`		Ed25519 signature over the canonical records	64
the frame			156

Table 5.2: A lean frame of two records, byte by byte. The link and the signature are abbreviated to their first six and last two bytes.

Reading it against Table 3.3: the map header, the five scalar fields and the two three-byte prefixes of the record stream and the signature are the 23 B header; the link and its prefix are 35 B; the signature is 64 B. The first record carries absolute values and takes 25 B here — five bytes each for latitude and longitude, which is where a keyframe's size goes — and the second carries nine differences in nine bytes. A third and a fourth record would add nine bytes each, which is the whole of the design's saving: $23 + 35 + 64 = 122$ B of the frame are paid once however many records follow.

The values are variable-length integers, seven bits to a byte, signed ones folded so that small magnitudes of either sign stay short. Nothing in the stream says where a record ends: a record is nine integers, and the count n says how many records there are.

5.4 Sender and receiver

The sender takes its own records in order, b at a time, and emits one frame per batch. One frame in every r starts with a keyframe and decodes alone; with $r > 1$ the others

start with a delta against the last record of the frame before. The design uses $r = 1$.

The receiver does five things to a frame, in an order that matters. It reads the sender identifier and decodes the record stream — against the last record it *accepted* from that sender, if the frame is a dependent one. It rebuilds the canonical ledger records, taking the first record's link from the frame and computing each later one as the hash of the record just rebuilt. It verifies the signature over those records. It offers each record to the store, which accepts only a sequence number newer than the last one stored for that sender. And only if every record was stored does it advance the state against which a later dependent frame would be decoded. A frame is signed as a unit, so it is accepted or refused as a unit.

outcome	when
accepted	decoded, verified, every record stored
desynchronised	well formed, but coded against a record this receiver does not hold
malformed	not a frame of this format
unknown sender	no public key held for the sender identifier
bad signature	the rebuilt records do not verify under the sender's key
replay	a sequence number not newer than the last one stored; nothing is stored
equivocation	a validly signed record for a sequence number already stored with different content; both are kept, as evidence
tampered	verified, but its chain link contradicts a record the store holds

Two properties follow from the order and are held by tests. Nothing unverified is ever stored, because storing comes after verifying. And a replayed or forged frame cannot move the decoding state, because that state advances only on acceptance; were it to advance on a frame that merely decoded, replaying an old self-contained frame would rewind it and desynchronise an honest sender's stream.

5.5 Before the receiver: admission, and a window on time

A forged frame that names a real sender and a newer sequence number costs a receiver a decode, a rebuild and a verification, as much as an honest one. Three steps can stand in front of the receiver, each cheaper than the next and each optional:

A group tag. An 8 B truncated HMAC-SHA256 under a key every member of the swarm holds, wrapped around the frame. An outsider's frame is refused at the cost of one HMAC. The frame format does not change; the tag is an envelope around it.

Cheap checks. The frame's length lies in the range a lean frame can have, its map parses with the expected keys, it names a known sender, and its sequence number is newer than the last accepted and not too far ahead — all before any record is rebuilt.

A per-sender rate cap. A token bucket at the sender's own frame rate with some slack,

so that an impostor can crowd out only the sender it names.

The window on GNSS time of Section 4.10.2 is a further layer: it reads the newest record's timestamp before the signature is checked. Section 8.5.8 measures what each step buys.

5.6 A compiled codec

The prototype is interpreted, and its receiver serves too few nodes per core for the channel to bind (Chapter 8). The lean frame was therefore written a second time, in Rust: encode, decode, rebuild and verify, with Ed25519 from the `ed25519-dalek` crate and its batch verification. On 10 000 frames the compiled encoder emits the same bytes as the interpreted one, and each verifies the other's frames.

The compiled receiver has a second, lazy path. A first tier checks the group tag and decodes and rebuilds the records, which are then usable at once; a second tier verifies their signatures in batches of 64 before the records enter the ledger. Records are acted on before they are evidence, and the window between the two is a stated cost: a member holding the group key could have records acted on that are later purged. The sender has a compiled path too, which encodes, chains and signs back to back; the energy meter of Chapter 8 reads it.

5.7 Tests

The tests are in layers, and the layers catch different things.

Published vectors. Each signature scheme must reproduce the test vectors of its specification before it is timed or used; the frame formats have vectors of their own, frozen when each format was fixed.

Properties. Round trips for every encoding over generated records; every single-bit change to a frame must make verification fail, tried at every bit position of a design frame; a frame that cannot be decoded must not be stored.

The reproduction gate. Every derived result file is recomputed from the committed raw data and compared row by row (Section 5.8).

Text against data. The numbers in the paper and in this thesis are macros written by a script from the result files. What is typed — a figure quoted from a frozen artifact, a sentence that states a verdict — is held to its artifact by a test, and a sentence the data no longer support stops the build.

The environment. A test reads every import in the code and requires the package to be declared, added after the gate passed on the author's machine and failed on a clean one.

What the layers do not catch is the subject of Chapter 11: a green suite is a statement about the questions the tests ask.

5.8 The two-layer data pipeline

The pipeline splits deliberately. **Measured** artifacts — hardware timings, simulation outputs — are non-deterministic, so they are measured once under controlled conditions, committed with environment and configuration-hash headers, and never silently re-rolled. **Derived** artifacts are byte-stable functions of those, so an automated gate re-computes every one of them on every run and fails if a single row differs.

This buys reproducibility at the price of *silent staleness* risk: a frozen artifact can go stale when an upstream decision changes and the artifact is not re-run. That is not hypothetical — it is exactly what produced the project's first recorded defect. The gate exists because the risk is real.

Chapter 6

Experimental Methodology

This chapter states how every result of Chapter 7–Chapter 10 was obtained: the statistical rules, the telemetry the measurements run on, the two simulation scenarios, and the hardware with its uncertainty.

6.1 Binding statistical rules

Every experiment obeys the following, and where a result predates a rule that is recorded rather than retrofitted:

- At least 30 seeded repetitions per configuration, with seeds logged in the artifact.
- Report a 95% interval with every estimate: a Student-t interval for a mean over seeds, and an expanded percentile bootstrap interval over 10^4 resamples for an interpolated crossing (Section 6.1.2).
- Microbenchmarks: $\geq 10^4$ iterations after 10^3 warm-up, garbage collection disabled, runs spanning a suspend discarded.
- Comparisons state effect size; never bare means.
- Raw artifacts are immutable once frozen.
- Any model-versus-measurement deviation above 10% must be investigated and either explained in writing or the model corrected. **Silent tolerance widening is forbidden.**

6.1.1 Why 30 seeds, and what it does not buy

The 30-seed rule is not arbitrary and is not original policy — it was imposed *after* four headline numbers were found to have been distorted by small-sample means evaluated against thresholds. None was a modelling error; every one was sampling.

Remark (The limit of the rule). More seeds fixes exactly one class of defect: a small-sample mean landing on the wrong side of a threshold. It does nothing for a threshold applied to a mean rather than a distribution, for an unverified constant on the measurement path, for a configuration change that perturbs the random realisation, or for a claim wider than the experiment supports. Chapter 11 catalogues instances of each, several found in this project's own work *after* the 30-seed rule was in force.

6.1.2 Intervals that cover what they say

Until October 2026 every interval in this thesis was a plain percentile bootstrap. A test with a known truth and thirty seeds found that it covered 92–94% of the time, not 95%: with thirty values the bootstrap distribution is too narrow. An interval of a mean is now a Student-t interval, and an interval of a crossing an expanded percentile bootstrap interval [56]; in the same test both covered 94–95%. The printed intervals widened slightly and no estimate moved.

6.1.3 Thresholds must be applied to distributions

A criterion of the form $V \geq 0.95$ is a *reliability* requirement. Applying it to a mean across seeds answers a throughput question instead. Where this thesis reports a threshold crossing it reports the distribution alongside — minimum, maximum, standard deviation, and the count of seeds that individually fail — and where the mean and per-realisation readings disagree, both are given.

6.2 Pre-registration

Where a run tests a prediction, the prediction is committed to version control *before the data exist*, as its own commit, so the ordering is third-party checkable in the repository history.

This practice is enforced rather than aspirational, and it has cost the project a claim: one pre-registration assertion was *withdrawn* rather than reconstructed, because writing an expectations file after results are known manufactures evidence. It has also exposed a subtler defect in this work's own practice, recorded in Chapter 9: a pre-registered *directional* prediction that the experiment had no statistical power to resolve.

Appendix B lists every registration with its commit, what it predicted and what happened. Of its twenty-two entries, eight failed in whole or in part, one could not be scored and one was met by a criterion that could not have failed; one has been run with two boards of the two to five it covers. They are in the table with the rest. The experiments added in October 2026 are in a second table of their own (Table B.2): seventeen, each registered before it ran, with the plan's statements and finer ones added in the same reg-

istration. In fifteen of the seventeen at least one statement missed or failed, and each is reported as it came out.

6.3 Experiment matrix

Exp	Question	Primary output
E1	overhead dominance (T1)	bytes/record, φ per encoding
E2	batching cure (T2/T2a)	φ and bytes/rec vs b; binding-ceiling check
E3	loss frontier (T3)	measured V vs $(1-p)^n$; goodput; Pareto plot
E4	scheme selection (T4)	energy/record, verification wall, crossover
E5	co-design end-to-end (T5)	bytes/energy/goodput vs baselines
E6–E7	low-rate arm and its co-design	feasibility partition, Pareto front
E8	capacity envelope from one load ceiling (superseded)	channel utilisation
Added October 2026, on the frame built as one object		
E9	what a frame is made of	field-level byte budget, both formats
E10	loss with the decoder in the loop (T3')	measured V against keyframe interval, three loss processes
E11	the design ladder	bytes, $N_{\max}$, receiver CPU, energy per rung
E12	which link carries a signed, chained frame (T6)	exclusion matrix, twelve data rates
E13	freshness with every term	fill, channel, verification against $D_{\max}$
E14	the low-rate batch at its own record spacing	records per frame, sustainable rate
E15	other PHY rates (model only)	saturation bound against rate
E16	sender energy, metered against modelled	µJ per record
E17	classical stream-signing schemes (model only)	authenticator bytes, frames/s, saturation bound
—	direct search for $N_{\max}$ (simulation)	delivery against N, bootstrap interval
—	record sizes on public flight logs	keyframe and delta sizes against the generator

Baselines in E1–E8 are A+JSON (naive), A+CBOR (compression-only) and D over-aggregated. From E11 on the comparison is a ladder in which each rung changes one thing, because A+CBOR signs every record *and* sends one record per frame and so confounds two effects.

6.4 Telemetry

A record holds nine integers: a millisecond timestamp; latitude and longitude in units of 10^{-7} degree; altitude in centimetres; three velocity components in centimetres per second; battery level in percent; and a flight mode. Integers and not floating point, because a canonical encoding must give the same bytes on every platform.

Records come from a seeded generator. A vehicle starts at a random position with a horizontal speed of up to 5 m/s in each axis; at every step its velocity changes by at most 0.1 m/s per axis, its position integrates that velocity with a jitter of two units, its battery falls by one percent with probability 0.02 and its mode changes with probability 0.01; the timestamp advances by 50 ± 3 ms. Every size in this thesis that is not marked otherwise is measured over thirty seeds of one thousand such records.

Two properties of the generator limit what its sizes mean, and both are checked. *Its records are* 50 *ms apart*, the spacing of the 20 records/s operating point it was written for, while the adopted point is 50 records/s. For the lean format that changes nothing: the delta record is already at the floor of the format at 50 ms, nine bytes for nine fields, and it is the same nine bytes when the generator is run at 20 ms (Section 7.5). *It is synthetic.* Section 7.5 sets it against twelve public flight logs at the spacing those logs allow, and against an autopilot run in simulation at 20 ms.

6.5 Simulation campaign

Two scenarios are used, for two different questions. The *saturation* scenario validates the channel models: IEEE 802.11a ad-hoc at 6 Mbit s^{-1}, RTS/CTS off, with N saturated senders in *both* unicast and broadcast modes, frame sizes taken from the real framer output. The *delivery* scenario measures capacity: the same channel with each node offering one frame per period, so the network runs at a chosen load and the delivered fraction is read at the receivers. The capacities of Chapter 8 come from the second, with the send instant redrawn in every period for the reasons given in Section 8.5. Transport is a packet socket rather than a flow monitor, because MAC-level goodput with no addressing artifacts is what the models predict.

Five further scenarios were added in October 2026, each a separate program so that the published runs stay as they were. Nodes spread over discs and squares of up to 1 km with log-distance path loss, Nakagami fading and capture, still or moving on independent Gauss–Markov paths (Section 8.5.5); a swarm that grows or merges while a controller chooses its batch (Section 8.5.6); the design map's cells at other record rates and deadlines (Section 4.10.4); 802.11a at 12 and 24 Mb/s and the 10 MHz channel of 802.11p; and the stream-signing schemes of Section 8.4, each record counted only if it verifies in time. Collision loss was also traced link by link at the capacity crossing (Section 7.4).

How a capacity is read. A configuration is a frame size and a frame rate per node. It is simulated over a grid of neighbourhood sizes N, thirty seeds per point, twenty seconds per run, and the delivered fraction of a run is the number of frames received over the number sent times $N - 1$. $N_{\max}$ is the largest simulated N whose mean delivered fraction is at least 0.95 with every smaller simulated N also passing. Its interval is the 2.5–97.5 percentile of that quantity over ten thousand bootstrap resamples of the seeds. A second reading is given beside it wherever the two differ: the largest N at which at least 95% of the individual runs reach 0.95.

Remark (The comparison rule). Each mode is compared *only* to its matching analytic variant: unicast to the acknowledged DCF model, broadcast to the published broadcast model. Mixing them is the single most effective way to fabricate a model gap, and this project did exactly that for several phases before catching it.

6.6 Hardware campaign

6.6.1 Boards and timing

The measured platform is the Raspberry Pi 4 (Cortex-A72), run headless with its frequency governor pinned to `performance`. Each primitive is timed by the rules of Section 6.1; Table 6.1 gives the medians used throughout. The ordering of the two 64 B schemes differs between this board and a desktop processor, which is why the timings of the deployment platform are the ones used.

Table 6.1: Primitive timings on the Raspberry Pi 4, medians.

	sign	verify
Ed25519	88 µs	259 µs
ECDSA P-256	136 µs	327 µs
BLS12-381	2.73 ms	8.61 ms
SHA-256 of one record		2.75 µs

6.6.2 The energy rig

Energy is measured outside the board. An INA219 current and voltage sensor with a 0.1 Ω shunt sits in the board's 5 V supply and is read fifty times a second by a microcontroller, which has no operating system to perturb the schedule and adds no load to the device under test. Each reading is one conversion of about half a millisecond — a sample, not an average — so the mean over a window is the mean power only if the load does not repeat in step with the sampling; a loop that runs back to back does not. The board marks the start and end of each measurement window on a GPIO line that the microcontroller

records with every sample, so the samples that belong to a window are known without aligning two clocks.

A measurement is a pair of sixty-second windows: one idle, one in which the board runs the operation in a tight loop and counts its completions n. With $\bar{P}$ the mean power of a window,

$$E_{\text{op}} = \frac{(\bar{P}_{\text{load}} - \bar{P}_{\text{idle}})\, t_{\text{load}}}{n}.$$

Each configuration is measured five times and the median is reported with the range. The board is checked for throttling before and after every window — by the firmware's flags and, since October 2026, by the kernel's count of under-voltage events, which still moves once the flags are set — and a flagged window is not used. Before a session the rig itself is put through a fixed sequence (`hw/rig_check.py`): the voltage reading must not move with the state of the marking line, the supply must hold with every core busy, and the board must log no under-voltage. The captures of July pass the meter's part of that test; a rig that fails is not measured on. The harness prints the value it *expects* before it reads the meter, so that a measurement cannot be quietly fitted to expectation.

6.6.3 The compiled sender, and floods against one core

In October 2026 the board metered the compiled sender of Section 5.6, as one process running back to back for each sixty-second window, beside a control that repeats a row of the first format; the rig check passed the same day. The sensor's gain was taken as calibrated. The same board received floods, with no radio in the path. A receiver process pinned to one core reads a local UDP socket; a sender process on another core offers the honest load of 41 senders, each at 12.5 frames/s — one core's share of the design's neighbourhood — and forged frames at a set rate. With each admission step of Section 5.5 switched on or off, honest V is the honest frames accepted over those sent; frames the kernel drops at a full socket count as lost, as they are for an overloaded receiver.

6.6.4 The model the meter is compared with

The model composes separately measured times with two measured powers: $P_c = 0.749\,\text{W}$, the incremental power of the processor while it computes, and $P_r = 0.218\,\text{W}$, that of the radio while it receives. Per record, for one signature over a frame of b records,

$$E = \underbrace{P_c\Big(t_{\text{enc}} + t_h + \frac{t_{\text{sg}}}{b}\Big)}_{\text{sender}} + \underbrace{P_c\Big(\frac{t_v}{b} + t_h\Big) + P_r\,\frac{T_{\text{air}}}{b}}_{\text{receiver}},$$

with t_{enc} the time to encode a record, t_h to hash it, t_{sg} and t_v to sign and verify, and T_{air} the airtime of the frame. Only the sender's term is metered (Table 8.10). The model reached this form through two corrections that the meter forced. It had no hashing term,

although every record is hashed once by the sender to extend the chain and once by the receiver to check it. And its processor power had been taken as the median over isolated primitives, 0.634 W; metered on four composed pipelines the power is 0.749 W, the four agreeing within 3.8%, so one constant is adequate; the composed value is 18% above the earlier one. Two of those four pipelines are the rows of Table 8.10. The power in the model is thus the median of the very runs it is set beside: what that table tests is the composition of the times, and it cannot test the power.

The radio's power is the weakest of the model's inputs and enters only the figures that are labelled as modelled. P_r is the power the *receiving board* added while a saturated unicast stream arrived on the 2.4 GHz band: two repetitions of thirty seconds, 0.205 and 0.231 W. It therefore includes the processor's share of receiving and the acknowledgements that radio sent, and it is applied here to broadcast frames on 5 GHz, which are not acknowledged. It has not been measured in the configuration it is used for.

6.6.5 Uncertainty of the energy figures

source	how it enters	size
sensor gain (shunt, converter)	multiplies every power reading	not calibrated against a reference load; cancels in every ratio
sensor offset	adds to every power reading	cancels in $\bar{P}_{\text{load}} - \bar{P}_{\text{idle}}$
idle drift	the idle window is not simultaneous with the load window	idle power moved by 5–9 mW across a session, about 1% of the difference
repeatability	five repetitions per configuration	half-range 0.6% and 1.1%
sampling	three thousand samples per window	small against the two above
time and count	a monotonic clock and an exact count	negligible

Three consequences follow. The *ratios* this thesis reports — the batched sender against the per-record one, and the meter against the model — do not depend on the sensor's gain, because the model's power constant was measured with the same sensor: they carry about one percent, from drift and repeatability. The gap between meter and model, 7.5 to 12.3%, is therefore several times the uncertainty of its measurement and is a property of the prototype (Chapter 8). The *absolute* values in microjoules carry the sensor's gain error in addition, and that was not measured: the calibration against a known load that the procedure prescribes was not recorded for these runs. They should be read as accurate to the sensor's factory tolerance and no better. One partial check exists. In October 2026 the baseline row was metered again on the second board through the second sensor and came out 2.2% from July's figure (Table 8.10). Two sensors with a gross error in one of them would not agree so; two sensors sharing an error — the same nominal shunt, the same calibration word — would, and this does not exclude it. The row that could not be

reported in July, with JSON records, was metered again there with ten repetitions and no window disturbed: 120.9 µJ [120.5, 122.5] per record.

One configuration is not reported. In three of its five repetitions the idle window read up to 0.5 W above the others — a background process on the board — which makes the difference meaningless. It is listed as unusable and is to be measured again.

6.6.6 Two radios

Two boards in ad hoc mode on a 5 GHz channel. With one sending broadcast frames and one counting them, they measure the loss of a link with no contention, and the time from one frame of a saturated sender to its next, which the receiver reads from the sequence numbers it spans and its own clock. With both sending and both counting they measure the loss among stations that contend, at a set load or saturated; each sender then draws its send instant within every period, as the simulations do. The chip's frame-burst mode is read from the firmware and logged before and after every session. Section 9.4 reports what these experiments do and do not support.

Chapter 7

Results I — Bytes, Placement and Loss

7.1 E1 — authentication dominates after compression

Thirty seeds of one thousand seeded telemetry records per encoding, inline placement, $g_a = 64\,\mathrm{B}$.

Table 7.1: Measured record size and authentication fraction. Compression moves the bottleneck.

encoding	mean bytes	φ at g_a=64	role
JSON	191.09	25.1%	naive baseline
CBOR	66.25	49.1%	compression-only baseline
MessagePack	65.16	49.6%	—
delta-CBOR	**45.00**	**58.7%**	smallest; the encoding the design uses

This is Theorem 4.1 instantiated: at delta encoding the signature is *larger* than the record it signs. The motivating claim of the thesis is a measurement, not a conjecture.

Remark (What s contains). Every size in Table 7.1 includes the record's own chain link: 32 B in the binary encodings, 64 hexadecimal characters in JSON. The delta record is 13 B of telemetry and 32 B of link, so the link is 71% of what the table calls the record. Counted with the signature, authentication material is 96 of 109 B, 88%, not 58.7%: φ as defined understates the point it was introduced to make. An external review made this observation, and it is what the lean format acts on (Section 7.5): the link is carried once per frame.

Remark (s is not a property of the encoding alone). The sizes above are measured over the first ∼50 seconds of each simulated flight. Sequence numbers and timestamps grow with the record index, and variable-length encodings charge for the extra digits, so JSON, CBOR and MessagePack all grow by 2–4 B over a ten-times-longer window while delta-

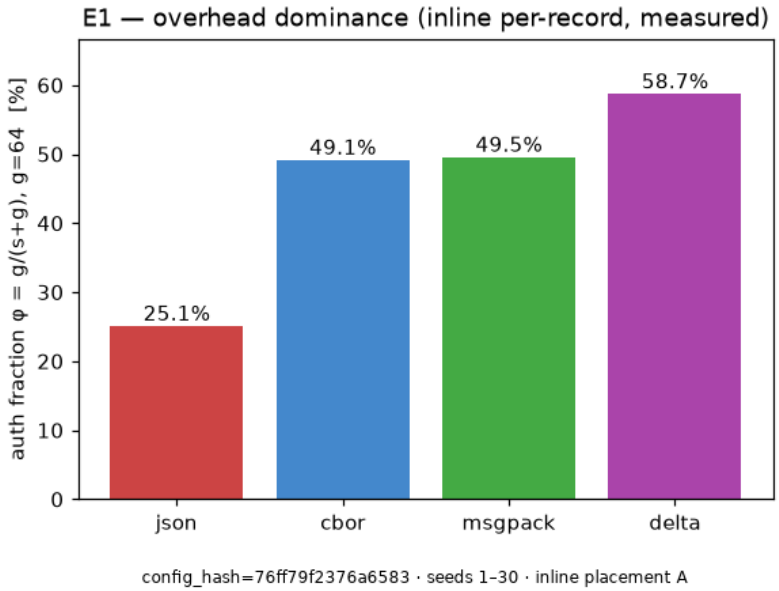


Figure 7.1: Authentication fraction across encodings. Every byte saved on the payload raises the fraction spent on cryptography.

CBOR stays flat — it encodes differences. The bootstrap interval quoted for CBOR is ± 0.02 B and describes *seed* variation; the systematic dependence on window length is ≈ 2.7 B, roughly 150× wider. The direction is conservative: a longer flight inflates the baseline and leaves the optimum alone, so the reported saving is the pessimistic end.

7.2 E2 — batching is the cure, and which ceiling binds

Sweeping placement A against B across encodings and MTU budgets confirms Proposition 4.2 qualitatively and Theorem 4.3 quantitatively. At $M = 256$ the MTU binds and amplification is operative. At $M = 1500$ freshness binds for every encoding, and at $M = 576$ for the three binary ones (JSON at 191 B is still limited by the frame there); where freshness binds the marginal value of compression is 1.0000 to twelve decimal places.

The practical consequence. On 802.11 the amplification law that motivates aggressive compression is never operative. Compression still pays — it pays exactly 1×.

7.3 E3 — the loss frontier

Measured verifiability against batch size and loss probability, placement B versus D. Frame-level batching Pareto-dominates block-level for all $V > (1-p)^2$, and at $\varepsilon = p$ block-level is infeasible beyond one frame, as Theorem 4.4 requires.

Remark (What this experiment does and does not validate). The emulator implements independent per-frame loss, which is also what the closed form assumes. Agreement

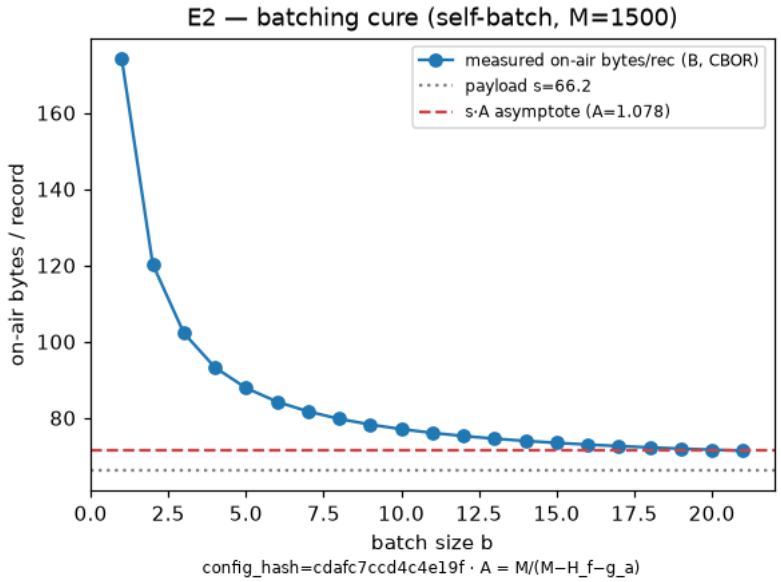


Figure 7.2: Per-record cost against batch size. The curve flattens where the freshness ceiling takes over from the frame-size ceiling.

is therefore a *consistency check*, not a validation of the loss model. What rescues the ordering is Theorem 4.5, which holds for any stationary loss process, plus Gilbert–Elliott simulation at matched mean rate showing V_D approaching but never reaching $1 - p$.

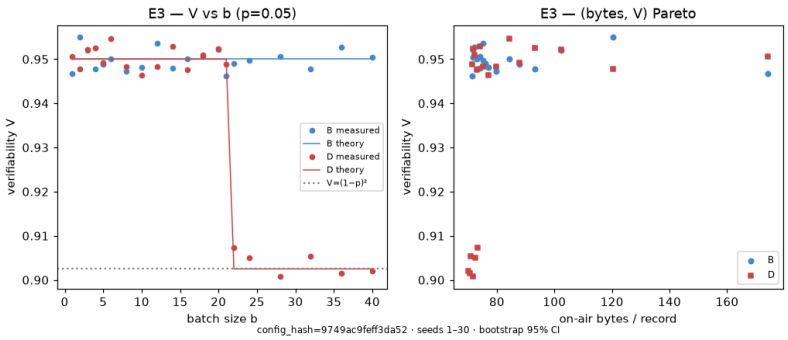


Figure 7.3: The loss-robustness frontier. Block-level authentication trades verifiability for bytes at a rate the constraint does not permit.

7.4 E10 — loss with the decoder in the loop

E3 compares one-frame placements with a block that spans frames. It does not exercise the case Theorem 4.6 describes, because in E3 every frame of placement B is assumed to verify alone. With delta coding that is true only of a frame that starts with a keyframe.

E10 removes the assumption by removing the model: records go through the lean sender, a loss process, and the lean receiver, and V is the fraction of generated records that the receiver decoded, verified and stored. Thirty seeds of 7 200 frames per cell in all.

Table 7.2: Verifiability at frame loss $p = 0.05$ with one self-contained frame in r ($b = 4$). Mean and 95% bootstrap interval over thirty seeds, against Theorem 4.6.

r	B/record	V measured	theorem	$V \geq 0.95$
1	43.25	0.952 [0.947, 0.957]	0.950	yes
2	41.38	0.921 [0.916, 0.927]	0.926	no
4	40.44	0.876 [0.863, 0.890]	0.881	no
16	39.73	0.655 [0.616, 0.693]	0.665	no

The design as first published is the $r = 4$ row: 0.876, not 0.95. The theorem predicts the measurement in every row, and at every other loss rate tried (p from 2.3×10^{-4} to 0.10). With bursty loss of the same mean — a Gilbert process with bursts of four frames — the $r = 4$ design verifies 0.929: better, and still short.

Remark (What this experiment validates, and what it does not). Unlike E3, agreement here is not a tautology: the closed form counts frames, while the experiment runs a codec and a receiver that know nothing of the formula, and they agree. What it does not validate is the loss process itself, which is synthetic at the frame level in both.

The corrected design uses $r = 1$. It costs 3.5 B per record more than $r = 16$ would, and it is the only interval that meets the target at the design's own loss rate.

7.4.1 When loss grows with frame length

Everything above gives every frame the same loss probability. At a fixed bit error rate a frame of L bytes with o bytes of MAC overhead survives with probability $(1 - \beta)^{8(L+o)}$, so two frames on one channel are related by

$$p(L) = 1 - (1 - p_{\text{ref}})^{(L+o)/(L_{\text{ref}}+o)},$$

and a batch, being a longer frame, is lost more often than the one-record frame it replaces. E10 has a third loss process for this: the bit error rate is set so that the *one-record* frame of the lean format is lost with probability p, and each frame is then lost by its own length.

On a channel that loses 5% of one-record frames, the design's frame is lost with probability 0.057, and 0.943 of its records are expected to verify. **That is below the target the design meets at equal frame loss**; the measurement, 0.947 [0.941, 0.953], agrees with the expectation and does not by itself separate it from the target. It reaches 0.95 only while one-record frames are lost at most 0.044 of the time. The loss budget $p = \varepsilon$ had no margin (Chapter 4); under this reading the batch spends margin it does not have. The byte and capacity comparisons of Chapter 8 are at equal frame loss and carry this qualification.

7.4.2 The loss the simulator makes, and a budget with a margin

The remark above names what E10 does not validate: the loss process. In October 2026 the simulator's own loss was traced, link by link, at the capacity crossing — 3 000 neighbour links in a spread and in a moving swarm. Consecutive frames of one link are lost independently: bursts average 1.06 frames, against 1.055 for independent loss at the same rate. Theorem 4.4 for independent loss is therefore the model of V at the crossing, and the pessimistic one wherever loss is bursty.

The budget then needs a margin. Figure 7.4 runs the design through five loss processes of one mean: independent, Gilbert with bursts of two, four and eight frames, and loss that grows with frame length. With $\varepsilon = 0.05$ and $p \leq \varepsilon - 0.02$ it meets the target, intervals included, under every one of them; the rule of Section 4.10.3 is that margin. Registered before the runs over every batch, loss rate and budget of equal frame loss, the rule failed by its interval in 5 of 192 cells, all of them bursty; every mean met its target (Table B.2). One of the five is the design itself: at $\varepsilon = 0.1$ and $p = 0.08$, under bursts of eight frames, its interval falls below the target, and under loss that grows with length it does too at the same point. A margin of 0.02 is smaller than the sampling half-width of such cells at 7 200 frames.

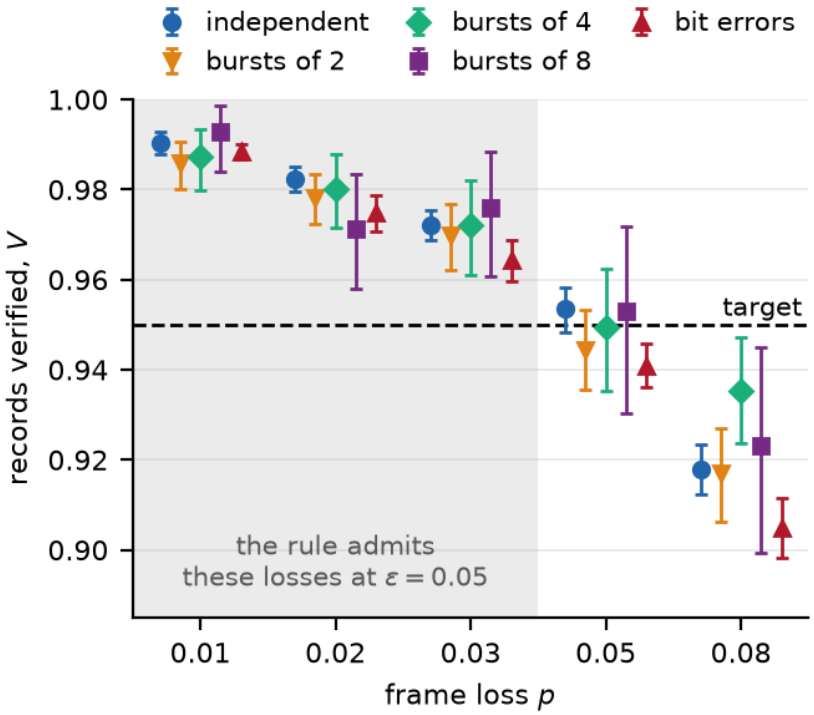


Figure 7.4: Records verified by the design ($b = 4$, $r = 1$) against frame loss under five loss processes, with 95% intervals; for loss that grows with length, p is the loss of a one-record frame. Shaded: the losses the margin rule admits at $\varepsilon = 0.05$.

7.5 E9 — what a frame is made of

Table 3.3 gives the byte budget of both formats. Three things in it matter for everything downstream, and none was visible while sizes were quoted as single numbers.

A delta is not a constant of the encoder. It encodes a difference, and a difference grows with the time it spans:

spacing between records	50 ms	0.1 s	1 s	5.5 s	16.5 s
lean delta record (B)	9.0	10.1	12.5	13.6	16.2

The generator's records are 50 ms apart, the spacing of the 20 records/s point it was written for. At the 20 ms of the adopted 50 records/s point the delta record is the same 9.0 B — it is already at the floor of the format, one byte per field, and a test in the repository runs the generator at 20 ms to confirm it — so the smallest value applies on the 802.11 arm. On the low-rate arm records are seconds apart, and using the 50 ms figure there overstated what a frame could carry (Section 10.3).

Where the chain link sits decides the floor. Figure 7.5 draws bytes per record against batch size for three layouts. With a link in every record, the first format cannot fall below its delta record however large the batch: at 12 records per frame it still costs 54.3 B each. Moving that link to the frame saves $32\,(1 - 1/b)$ B per record, 24 B at $b = 4$, and gives 51.0 B — less than a per-record link reaches at any batch drawn. The lean format's shorter header and record coding account for the remaining 7.7 B, to 43.25 B. The two first-format curves are sums of measured parts; the lean curve is emitted frames.

The generator's delta is the floor of the format. Every one of 29 970 delta records at 50 ms spacing is exactly 9.0 B: nine fields, one byte each. That looks synthetic, and it is. To find out whether it matters, the same encoder was run over twelve public PX4 flight logs — four quadrotors, two hexa- or octorotors, three fixed wings and three VTOLs, 29 092 in-flight records — chosen by a rule committed before any log was opened.

Table 7.3: Record sizes on real flight logs against the generator, at matched spacing. The logs carry position at 5 Hz, so nothing finer than 0.2 s can be measured from them.

	keyframe	delta, 0.2 s apart	B/record at b=4	saving
twelve flight logs (mean)	22.0	11.1	44.3	69.1%
generator	24.0	10.8	44.6	69.4%

At matched spacing the generator is within a quarter of a byte of real telemetry. The spread is by speed: the six multicopters, mostly hovering, average 10.0–10.5 B per delta,

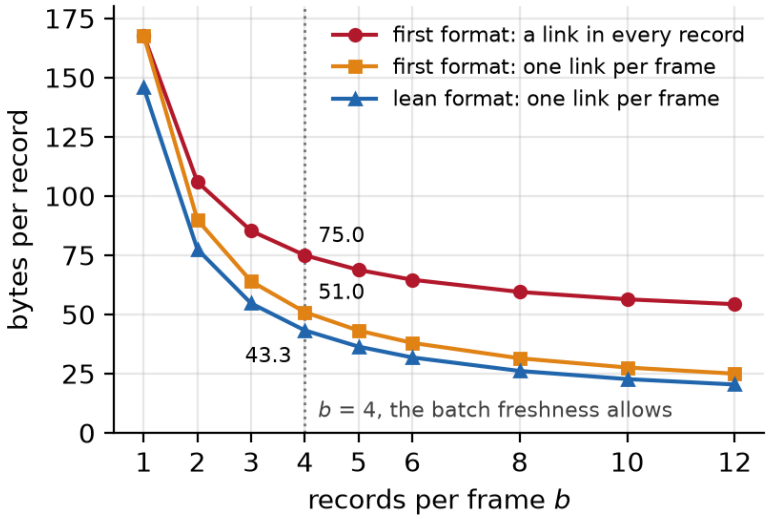


Figure 7.5: Bytes per record against batch size, by where the chain link sits. Records are delta-coded in all three and a frame starts with a keyframe, so at $b = 1$ the first-format curves start below row 1 of Table 8.1, whose records are plain CBOR. The dotted line is the batch that freshness allows at the adopted operating point.

and the three fixed wings up to 13.2 B, for which the saving is 68.2%. So the mean flatters a fast swarm by about a percentage point, and the 50 Hz spacing of the adopted operating point is not covered by any public log found.

At the operating rate: the autopilot in simulation. Because no public log reaches 50 Hz, PX4 v1.17.0 was run in software-in-the-loop — its own estimator, controllers, logger and MAVLink module, flying a simulated quadrotor — with the logger writing position every 20 ms, and the log was put through the same code as the public ones. The flight was fixed before it was flown: a climb, a 150 m square at a commanded 5 m/s, the same square at 12 m/s, a minute of position hold, a landing.

	records	keyframe (B)	delta (B)	B/record at b=4
PX4 in simulation, 20 ms grid	14 317	20.9	9.0	42.47
generator, as used throughout		24.0	9.0	43.25

Every one of the delta records is at the format's floor: over 20 ms nothing in a record changes by more than one byte's worth, at 9.1 m/s as at rest. The keyframe is three bytes smaller than the generator's, so with real autopilot records the design costs 42.47 B per record, 1.8% *less* than the figure this thesis reports. At the operating point the generator is slightly pessimistic, where at 0.2 s it was slightly optimistic.

Remark (Two predictions of this measurement that did not hold as written). Five ranges were committed before the flight. Four held. The fifth was a consistency check: resampled at 0.2 s, the simulated flight was predicted to fall among the six real multicopter logs,

10.0–10.5 B per delta; it gives 10.64 B. By phase it is 10.0 B in the climb, the hold and the descent and 11.0–11.2 B on the two squares: the real logs are mostly hover, and this flight spends two thirds of its time translating. That reading was made after the number was seen. It is the harder case for the 20 ms result and not the easier one, but the prediction as written was missed, by 0.04 B.

The commanded 12 m/s was not reached either: the fast square averaged 9.1 m/s. And the registered statistic for the stream's timing could not be scored at all, for a reason that has nothing to do with the autopilot (Section 8.5).

Richer records and other airframes. One quadrotor flight says little about a fixed wing, or about a record of more than nine fields. 30 further flights of the same autopilot's simulator were made at 50 Hz, ten each of a quadrotor, a fixed wing and a VTOL, the fixed wings at their planned 15–25 m/s, and records of 9, 15 and 25 fields were coded from each at their exact lengths (Table 7.4). Almost every field changes by less than its first byte holds, so a delta costs about a byte per field on every airframe, a fixed wing 0.2 B more than the quadrotor. With 25 fields — adding attitude, body rates, navigation quality, airspeed, wind, mission state and thrust — the design still saves 63.3% of the bytes and serves 3.04 times the nodes of per-record signing. The gain is set by the number of frames, not by the size of the record.

Table 7.4: Records from 30 flights of the autopilot's simulator, ten per airframe, at 50 records/s: keyframe and per-airframe delta in bytes, the bytes the design saves, and the nodes it serves over per-record signing by the access-rule model.

		delta (B)				
fields	keyframe (B)	quadrotor	fixed wing	VTOL	saved	ratio
9	22.8	9.0	9.2	9.2	70.1%	3.51
15	32.8	16.0	16.2	16.2	67.2%	3.31
25	45.5	26.0	26.3	26.2	63.3%	3.04

Remark (What the simulator could not fly, and what that changed). The simulator's airplane first flew no faster than about 14 m/s: its model brakes the aircraft with an artificial linear drag term. With that term removed, and the airplane's thrust raised to 12 N, the fixed wings flew the registered missions again at their drawn speeds, each within 1 m/s. The first flights also started their altitude controller from sea level, which put the first leg of every airplane flight near the ground; a reset before the mission fixed it. Neither changed a record size by more than a fraction of a byte. The simulator also holds its navigation-quality fields constant, so those fields cost less than they would in flight.

Remark (What was predicted, and what was not testable). The prediction committed in advance was a mean delta of at most 13 B and a saving of at least 68%, *at 0.1 s spacing*. The logs turned out to carry position at 5 Hz, so the prediction was applied, unchanged in its numbers, at 0.2 s — the stricter reading, since a difference over a longer interval is

never smaller — and holds there. The 0.1 s prediction itself is recorded as untested. One predicted range was missed: the keyframe, predicted at 22–25 B, measures 22.0 B, because multicopters fly slowly enough for one-byte velocities.

Chapter 8

Results II — Co-design and Capacity

8.1 E5 — the end-to-end configuration, and a correction to it

Exhaustive search over encoding, scheme, placement and batch returns delta coding, Ed25519, one signature per frame, $b = 4$. As first reported, that configuration cost 72.0 B per record against 174.3 B for the compression-only baseline: a 58.7% reduction.

Remark (That figure belonged to a design that did not meet its own constraint). The 72.0 B frame amortised one keyframe over four frames. By Theorem 4.6 such a design verifies 0.881 of its records at $p = 0.05$, not 0.95; measured with the decoder in the loop, 0.876 (Section 7.4). The search had admitted it because the verifiability model it consulted assumed every frame decodes alone. With that assumption replaced by the theorem, the search returns the same placement and batch with a keyframe in every frame.

Table 8.1 takes the design apart. A baseline that signs every record and sends it alone confounds two things — sharing a frame, and sharing a signature — so rows 2 and 6 separate them. The corrected figures are 74.96 B per record in the first format, 57.0% below its baseline, and 43.25 B in the lean format, 70.3% below its own.

Remark (Report the decomposition, never the bare authentication ratio). The cut in authentication bytes alone is 75.0%, and it is tempting to lead with it. It must not be led with, because it reduces algebraically to $1 - 1/b$: invariant to the header, the signature size, the encoding and the scheme. It is the definition of batching, not a result.

Table 8.1: The design ladder at 50 records/s and $D_{\max} = 100\,\text{ms}$. N_{sat}: offered load within the saturation throughput of the broadcast model. $N_{\max}$: largest neighbourhood with mean delivery ≥ 0.95 in simulation, with its 95% bootstrap interval. CPU: share of one receiver core spent verifying at $N_{\max}$. † size is a sum of measured parts; rows 5, 6 and 8 are emitted frames.

	configuration	b	B/record	with cert., one per 5 frames	with cert., one per 500 ms	N_{sat}	$N_{\max}$	CPU
First format: text keys, a chain link in every record†								
1	sign every record, one per frame	1	174.25	213.05	188.41	18	32 [32, 32]	41%
2	sign every record, share the frame	4	141.25	150.95	149.41	24	56 [56, 56]	72%
3	one signature per frame	4	93.25	102.95	101.41	30	76 [75, 76]	25%
4	… and delta records	4	74.96	84.66	83.12	34	88 [88, 89]	29%
Lean format: integer keys, one chain link per frame								
5	sign every record, one per frame	1	145.77	184.57	159.93	20	35 [35, 35]	45%
6	sign every record, share the frame	4	93.00	102.70	101.16	30	76 [76, 77]	98%
7	one signature per frame, no delta†	4	54.50	64.20	62.66	42	108 [107, 109]	36%
8	one signature per frame, delta **(design)**	4	**43.25**	52.95	51.41	52	**124** [124, 125]	42%

8.2 The factorial ablation, which narrows the claim

A full factorial ablation over the four axes asks whether "co-design" is doing work that single-axis optimisation would not.

- **Placement and batching couple exactly**, by $g_a(1 - 1/b)$ — hence *exactly zero* at $b = 1$, where placements A and B are byte-identical on a single-record frame.
- **Encoding is separable in the first format.** Every interaction term is exactly zero, because s enters additively and therefore *cannot* interact.
- **The scheme axis is byte-degenerate** among the 64 B schemes: they differ only in CPU cost.

An earlier claim that a smaller payload increases the value of batching was a **ratio-scale artifact**: the absolute saving is 81.0 B for every encoding. The four-axis co-design claim was overstated and is now precise.

Remark (One coupling the first-format ablation could not see)*.* In the lean format encoding and batching *do* interact, in one specific way: a record can be coded as a delta only if the record before it is in the same frame or is known to have arrived. Row 6 of Table 8.1 is smaller than four copies of row 5 partly for that reason. The separability result above is a property of the first format, where each record is encoded alone.

8.3 E4 — scheme selection, and the receiver's CPU

On ARM — the deployment platform — Ed25519 verification is 33.2× cheaper than a BLS pairing, which keeps Ed25519 optimal for self-batching at any plausible radio/CPU power

ratio. The ordering between Ed25519 and ECDSA-P256 *flips* between x86 and ARM, so the choice is platform-specific; the byte result is identical either way, since both are 64 B.

The crossover rule of Section 4.6 was evaluated over relay fractions of 0, 0.25, 0.5 and 1, batches of 2 to 32 and neighbourhood arrival rates of 50 to 2000 records per second: eighty cells. Ed25519 wins every one. For a node's own records BLS never breaks even, because its 96 B object costs bytes where it was meant to save them. When every record is relayed and thirty-two share a frame — the case most favourable to aggregation — BLS saves 61 B per record, worth 81 µs of airtime at 6 Mb/s, and costs 2.6 ms of extra verification; it would pay only if the radio drew 31.6 times the power of the CPU. On the board measured here the ratio is 0.29 (0.218 W against 0.749 W). And at 200 records per second and above one core cannot verify BLS fast enough at all. An aggregatable scheme beats a fast conventional one only where airtime is two orders of magnitude dearer than computation, which is the low-rate regime of Chapter 10 and not this one.

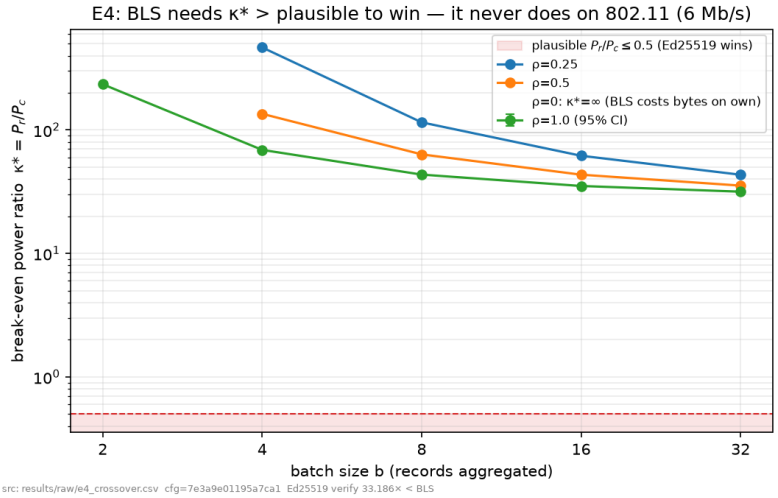


Figure 8.1: The scheme crossover. BLS becomes worthwhile only in the cross-signer regime.

A receiver verifies every neighbour's frames, and that is a second ceiling on the neighbourhood. With k signatures per frame, verification time t_v and one hash per record t_h, one neighbour costs $(\Lambda_i/b)\,k\,t_v + \Lambda_i\,t_h$ seconds of CPU per second, and one core serves

$$N_{\text{cpu}} = 1 + \left\lfloor \frac{1}{(\Lambda_i/b)\,k\,t_v + \Lambda_i\,t_h} \right\rfloor$$

nodes. On the Raspberry Pi 4 that is a neighbourhood of 296 nodes when each frame carries one Ed25519 signature and of 77 when every record is signed. At its $N_{\max}$ the design uses 42% of a core, so the channel binds first. Row 6 of Table 8.1 shows the constraint is not idle: with a signature on every record it reaches the channel's limit at 98% of a core. With BLS a verification takes 8.61 ms and one core serves 10 nodes: BLS is excluded by the CPU long before the channel is reached, in addition to its larger signature.

These figures verify one signature at a time. Ed25519 was designed for batch verification: its authors report under 134 000 cycles per signature in a batch of 64, against 273 364 for one [17]. Measured on the Raspberry Pi 4 with a public-domain implementation that has a batch interface, a signature costs 253 µs alone and 116 µs in a batch of 64: 0.46 of the cost, inside the 0.45–0.60 written down beforehand. A batch of four saves little (0.85). It is not used here — the cryptographic library this implementation is built on exposes no batch interface — so N_{cpu} is conservative for Ed25519 by about a factor of two where a receiver can afford to wait for 64 frames. ECDSA does not admit it.

Remark (What N_{cpu} does not charge). The expression above counts signature verification and hashing: the work of the cryptographic library. It does not count decoding a frame and rebuilding its records, which in this thesis's prototype is interpreted Python. Timed on the Raspberry Pi 4 as it runs, the lean receiver takes 1.43 ms for a four-record frame, 5.5 times its one verification, and the sender 0.87 ms (for a frame of one record: 0.66 ms and 0.38 ms). One core of the prototype therefore serves 57 nodes, not 296, and a neighbourhood of 124 needs, by that arithmetic, 2.2 of the board's four cores (measured, below: three). The sentence "the channel binds first" holds for a compiled receiver; for the prototype on one core the processor binds first. The receive times are of the receiver that keeps every accepted frame (Section 3.4): against a control of the earlier receiver timed in the same session, keeping the frame adds 10 µs to a four-record frame, 0.7%, where at most 2% had been written down beforehand (Appendix B). The node count is unchanged, by a margin of two parts in a thousand.

The board's four cores, measured. "2.2 of the board's four cores" is arithmetic from one core's time. It was put to the board: one receiver process pinned to each of one, two, three and four cores, each fed the stream the benchmark times one frame of, all started together and run for thirty seconds; what should come out was written down first (Appendix B). One process handles 692 frames a second and serves 56 nodes — one fewer than the count from the median frame time, which had two parts in a thousand to spare and does not keep them in a loop. Two processes together serve 106 nodes, three 150 and four 187: 1.90, 2.70 and 3.36 times one. The last two are below what had been written down (2.75 and 3.5 at least). At four, each process runs 16% slower than one alone, with no throttling recorded; what the processes share — the cache, the memory bus — was not examined. **The design's neighbourhood of 124 nodes therefore needs three of the board's four cores, not 2.2**, and that with no radio and no dispatcher in the path: a frame reached its process for free. N_{cpu} and the CPU column of Table 8.1 are ceilings set by cryptography alone.

Both times came in under what had been written down for them (1.0–2.5 ms and 2–5 ms). The ranges had been scaled from a desktop figure taken while that machine was busy with simulations, which was three to four times too high; the board is in fact six to seven times slower than the desktop on this interpreted code, against three times on a

signature.

A compiled receiver, and verification in two tiers. Compiled (Section 5.6), the same receive path takes 0.277 ms on the board, 5.1 times less, and one core serves 289 nodes; with signatures verified in batches of 64, 643 (Table 8.2). The channel binds again: the design's 124 nodes need less than half of one core. A receiver that uses records as soon as their tag and frame are checked, and verifies their signatures in batches before they enter the ledger, serves 628 nodes per core. A record is then evidence within 191 ms by the convention of Section 4.10; in that window a member holding the group key could have records acted on that are later purged.

Table 8.2: Time to receive a four-record frame on one Raspberry Pi 4 core, and the nodes one core serves at 50 records/s; all four timed in one session.

receiver	µs per frame	nodes per core
interpreted (Python)	1 409	57
compiled (Rust)	277	289
compiled, batches of 64	124	643
lazy: tag and decode, then batches	10.2 + 117.2	628

8.4 E17 — the classical stream-signing schemes, placed

Chapter 2 describes the schemes that amortise one signature over many packets. Table 8.3 places each at the adopted operating point as *the lean one-record frame with that scheme's authenticator in place of the signature* — the same header, chain link and record — with the authenticator sized from the scheme's own definition and this thesis's primitives (32 B digests, 64 B signatures). Two capacities are given: the saturation bound N_{sat}, and $N_{\max}$ from the direct search of Section 8.5 on a frame of that size at that rate — five more cells, whose crossings were predicted and committed before they were run (Section 8.5.1).

Table 8.3: Stream-signing schemes at 50 records/s. Sizes are modelled; $N_{\max}$ is simulated for a frame of that size, with its 95% bootstrap interval. † no non-repudiation: a third party cannot be shown who sent a record; ‡ a packet does not verify alone, so $N_{\max}$ — frames delivered — is an upper bound on records verified; the last column counts a record only if it verifies within $D_{\max}$. § disclosure delay of two intervals of at most 20 ms.

scheme	auth. B/record	frames/s	total B/record	N_{sat}	$N_{\max}$	in time
MAVLink 2 tag†	13	50.0	93.8	23	42 [41, 42]	42.1
TESLA†‡	24	50.0	105.8	22	40 [40, 40]	39.6–39.8§
Gennaro–Rohatgi‡	32	50.0	113.8	21	39 [39, 39]	none
EMSS‡	65	50.5	147.3	20	34 [34, 34]	none
Wong–Lam tree, block of 4	129	50.0	210.8	17	28 [28, 29]	29.1
a signature on every record	64	50.0	145.8	20	35 [35, 35]	35
this design, $b = 4$	16	12.5	43.3	52	124 [124, 125]	124

Every one of these schemes spreads an authenticator across packets and still sends a frame per record. A frame costs 173 µs of channel time before its first payload byte, so the weight of the authenticator hardly matters: replacing a 64 B signature by a 13 B tag — and giving up non-repudiation — moves N_{sat} from 20 to 23 and the simulated $N_{\max}$ from 35 to 42. Four records in a frame move them to 52 and 124. The stream-signing literature amortises the *cost of signing*; the cost that binds on a contended channel is the *frame*.

Three things the table does not show. Tree chaining is the one classical scheme that keeps both properties a ledger needs — every packet verifies alone and its origin can be shown to a third party — and it pays 129 B per packet for them. Batching has a cost no per-packet scheme has: a lost frame takes b records with it, and records wait for the batch to fill. And a scheme with a shorter frame is lost less often at a given bit error rate (Section 7.4), which this table does not credit.

The schemes run, against the deadline. Sized from their definitions, the schemes could still have been favoured by what sizing leaves out. In October 2026 each was implemented and run on simulated links, and a record was counted only if it verified within $D_{\max}$ (Section 6.5). The delivery crossings reproduce the table's within their intervals. Against a 100 ms deadline only three schemes reach 0.95: the MAVLink 2 tag at 42.1 nodes, Wong–Lam at 29.1, and TESLA with a disclosure delay of two intervals of at most 20 ms at 39.6–39.8. Gennaro–Rohatgi, every EMSS variant and TESLA with four records a frame never do: a packet that waits for a later one to verify misses the deadline. None of the three reaches the design's 124, and two of them give up non-repudiation.

8.5 Capacity, by direct search

Bytes matter because they are airtime, and airtime decides how many neighbours can be heard. The $N_{\max}$ column of Table 8.1 is the answer to the question an operator asks — *how many UAVs, at what telemetry rate?* — and it is found by simulating each configuration at its own frame size and frame rate over a grid of neighbourhood sizes, thirty seeds per point, and reading off where mean delivery leaves 0.95. The interval is the 2.5–97.5 percentile of that crossing over ten thousand bootstrap resamples of the seeds.

These figures hold the *mean* over runs to the 0.95 target, which is the headline criterion throughout this thesis. The stricter reading — that 95% of individual runs reach it — gives 34 and 120 in the lean format and 31 and 86 in the first: at most four nodes fewer, now that a run samples the channel and not one phase configuration. On the low-rate arm the two readings are far apart (Chapter 10).

Remark (Do not round this to one number). The advantage of the design is not one number. It hardly moves with the operating point — 3.5× and 3.6× in the lean format, 2.8× and 2.8× in the first — and it depends on the format, because the first format's

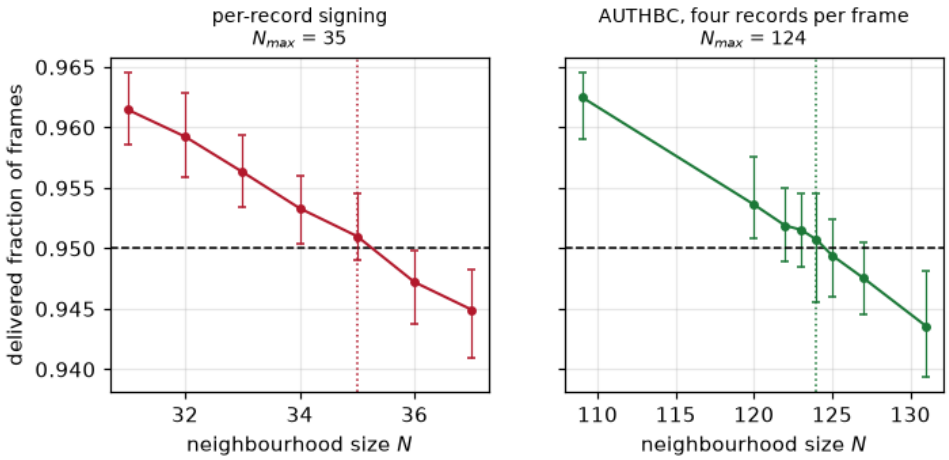


Figure 8.2: Delivered fraction of frames against neighbourhood size, lean format: per-record signing and the design. Mean of thirty runs; bars span the runs.

Table 8.4: $N_{\max}$ of baseline and design at both operating points, with 95% bootstrap intervals.

	per-record signing	design	ratio
50 records/s, 100 ms (compliant with TS 22.125)			
first format	32 [32, 32]	88 [88, 89]	2.8×
lean format	35 [35, 35]	124 [124, 125]	3.5×
20 records/s, 250 ms (relaxed)			
first format	78 [77, 78]	219 [215, 221]	2.8×
lean format	85 [85, 86]	306 [306, 308]	3.6×

records each carry a chain link that no batch amortises. Quote it with its format. An earlier draft said "≈ 3× holds across readings"; the four ratios it rested on came from the single load ceiling, ran from 1.9 to 3.2, and are superseded with it.

The larger step is sending a quarter as many frames. Row 6 of Table 8.1, which shares the frame and keeps a signature per record, already reaches 76 nodes (2.2× the baseline); sharing the signature as well adds 1.6×. Every frame pays a preamble, an inter-frame space, a backoff and MAC framing whatever its length — about 173 µs, 47% of what a baseline frame costs the channel — so removing three frames in four is worth more than their bytes. It is the cost that frame aggregation amortises in 802.11n for frames addressed to one receiver (Section 2.7.1); a broadcast sender has only the application to do it with.

8.5.1 A rule for the crossing, and the test it was put to

The envelope this chapter used to report applied one utilisation ceiling to every configuration (Appendix C.2), and Chapter 9 records how that failed. Looking at the six crossings it failed on suggested something simpler: at the 5% level, loss is a line in airtime. With

$f = \Lambda_i / b$ the frames per second a node sends and T the time one frame holds the medium (its airtime plus DIFS),

$$0.05 = N f\,(a\,T + c) \qquad \Longrightarrow \qquad N_{\max} \approx \frac{0.05}{f\,(a\,T + c)}. \tag{8.1}$$

NfT is the fraction of time the neighbourhood's frames occupy the medium, so the first term says loss grows in proportion to occupancy; the second, with $c = 8\,\mu s$ fixed at twice the simulator's preamble-detection time, is the chance that another frame starts inside the window in which a station cannot yet hear it. Only a is fitted.

A line found by looking at six points is a hypothesis. It was treated as one: the seven configurations that had not been simulated were named, the line's value for each was written down with a tolerance of ±6%, and both were committed before any of the seven was run. The slope is $a = 0.071$, the mean of the six per-configuration values, which reproduces those six within 3.8%.

Table 8.5: The seven held-out configurations: the crossing the line predicted, the crossing simulated (interpolated, thirty seeds per node count), and the deviation of the simulation from the line and from the single load ceiling.

configuration	frame, rate	line	simulated	vs line	vs ceiling
first, one signature per frame	373 B, 12.5/s	78.4	76.2	−2.7%	−3.5%
lean, one signature, no delta	218 B, 12.5/s	110.3	108.9	−1.3%	−11.5%
first, design as first specified	288 B, 12.5/s	93.5	91.3	−2.3%	−8.7%
relaxed: first, per-record signing	174 B, 20/s	78.1	78.4	+0.3%	−11.0%
relaxed: first, design	300 B, 5/s	227.6	219.8	−3.4%	+5.7%
relaxed: lean, per-record signing	146 B, 20/s	84.9	85.5	+0.7%	−15.3%
relaxed: lean, design	173 B, 5/s	312.4	308.1	−1.4%	+14.5%

The prediction held. All seven crossings lie within 3.4% of the line, and in each of the five configurations where the two rules disagree by more than a tenth the simulation is closer to the line than to the ceiling, which misses the thirteen simulated configurations by up to 15%.

The batch as a dimension. Every configuration above holds one record to a frame or four, because both operating points have $\Lambda D_{\max} = 5$. The line and the model had therefore been checked across frame size and frame rate and never across the batch itself; a review of the paper pointed this out on 2026-10-09. Three more cells were registered before any of them was run (Appendix B): the frame the lean codec emits with two, three and eight records at 50 records a second, which deadlines of 60, 80 and 180 ms would admit. Table 8.6 sets what was written down beside what the simulator returned.

All six held; Figure 8.3 sets them on the line. These are capacities of the same idealised scenario as every other in this chapter, and the three deadlines are hypothetical operating points: the freshness table, the loss experiment and the energy runs are all for four records,

Table 8.6: The lean design at three other batch sizes, 50 records/s: the crossing of 0.95 delivery predicted by the airtime line (slope as calibrated before, band ±6%) and by the access-rule model (nothing fitted, band ±3%), and as simulated (30 seeds, 95% interval). All six predictions were committed before the first run.

records	deadline	line	model	simulated	from the line	from the model
2	60 ms	66.0	66.6	66.7 [66.4, 67.0]	+1.1%	+0.2%
3	80 ms	96.3	96.0	96.5 [96.1, 96.8]	+0.2%	+0.6%
8	180 ms	225.9	220.6	222.3 [221.4, 223.3]	−1.6%	+0.8%

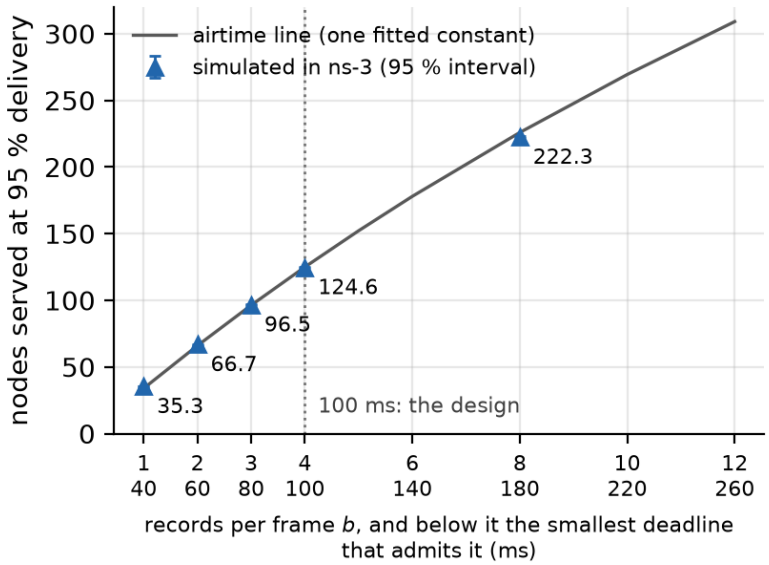


Figure 8.3: The neighbourhood the lean design serves at 50 records/s, by batch size and the smallest deadline that admits it: the five simulated crossings of 0.95 delivery (95% intervals, smaller than the markers) and the airtime line through the batch sizes that were not simulated. At 40 ms the batch is one record and nothing is gained.

and nothing but the channel was evaluated at two, three or eight.

A second test, on frames of another kind. The stream-signing baselines of Table 8.3 are five more frames, all shorter or longer than any the line had seen and all at one frame per record. The same procedure was followed: the line's crossing for each, with the slope unchanged, was committed before the runs.

scheme	frame, rate	line	simulated	vs line
MAVLink 2 tag	94 B, 50/s	40.6	42.2	+3.8%
TESLA	106 B, 50/s	38.8	40.4	+4.2%
Gennaro–Rohatgi	114 B, 50/s	38.0	39.4	+3.8%
EMSS	146 B, 50.5/s	33.6	34.8	+3.5%
Wong–Lam tree	211 B, 50/s	28.0	29.0	+3.5%

All five are inside the ±6% band, and all five are *above* the line, by 3.5–4.2%. That offset was written down beforehand as an expectation, not a prediction: the two calibration configurations at 50 frames per second sat 3.4 and 3.8% above the line, so if the residual belongs to the frame rate these five would too. They do. The line's error is therefore not noise at this rate; it is a small, repeatable bias, and it is the reason the remark below forbids quoting the line as a capacity. These cells do not separate the line from the ceiling — at this frame rate the two differ by a few percent — which the registration said in advance.

Remark (What this licenses, and what it does not). Equation (8.1) is an empirical fit, not a model. It holds at one loss level — loss is convex in load, so the line says nothing about a 1% or a 10% target — at one PHY rate, in one collision domain, for frames of 94–565 B, 5–50.5 frames per second and 28–306 nodes, and in a simulator. Its residual is ordered by frame rate, from +3.6% at 50 frames per second to −2.4% at 5. Every capacity quoted in this thesis is a simulated value; the line is what one would use to interpolate between them, and to see why a ceiling on *utilisation* could not work: it divides the offered load by the saturation throughput of N stations, and loss in a lightly loaded network has no reason to follow that yardstick.

8.5.2 How the traffic source was chosen

Each simulated node sends one frame per period at an instant drawn uniformly within the period. That choice was not the first one, and how it was reached is part of the result.

The scenario this project had used throughout sends frames *strictly* periodically: each node gets one random start offset and keeps it. In such a run the relative phase of any two nodes is frozen. Per-node output shows what that does: in the worst of thirty runs at $N = 29$, four nodes delivered *none* of their frames while 22 lost none at all. The simulator raises carrier sense only at the end of a 4 µs preamble-detection period, so two nodes whose phases fall within that window collide in every period of the run; and nodes whose phases put them in the same busy period contend with each other, and only each other, for all of it. A strictly periodic run is one draw from a lottery of phase configurations, and the spread across runs describes the lottery.

Three tests, each with its prediction committed before its runs, established what follows from that. Jittering each frame's send time, or letting every pair of nodes drift through all relative phases, changes the thirty-run *mean* by less than the 0.005 set in advance — none of sixteen comparisons exceeds 2.5 standard errors — while the spread between runs falls by a factor of ten to twenty when phases are redrawn every period. The runs reported here redraw the phase every period. Two of the predictions made along the way failed, and both failures are recorded with the data: a jitter of 1 ms was predicted to halve the spread and did so at two test points of four, and a phase sweep was predicted to bring it under 0.004 and reached 0.003–0.006.

Remark ("Within tolerance" is not "zero"). When all six cells had been run under both sources, the mean delivered fraction was higher with redrawn phases by 0.003 on average over 14 points, and every one of the six crossings was higher, by 1–9%. The points are close to independent — the simulator hands out random streams in creation order, so runs at different node counts draw different start offsets, and their per-seed results are uncorrelated — which puts the average at about two standard errors: weak evidence of a real difference of that size, and no more. If phases are uniform and a period's losses depend only on that period's phases, the expected difference is zero, so a real one would need a mechanism not yet identified. No reported capacity depends on the answer, since all of them use the redrawn source; what depends on it is how the strictly periodic figures of earlier work should be read.

Remark (It did not replicate). The question was put to seeds that had never been used: the same 14 points, 1 260 new runs, the statistic and the reading of each outcome committed first. On them the difference is 0.0004 with a standard error of 0.0009 — 0.5 standard errors, where the first sample had shown 2.2. The first result was sampling. Two sources with uniform phases have the same mean delivery, as the argument above said they should.

A second claim went with that registration and did not survive it. To explain why all six crossings had read lower under strictly periodic senders, the registration said that the crossing rule — the largest N that passes with every smaller N passing — reads low on a noisier curve. Resampling the periodic source's own run-to-run scatter around a common mean shows that it does not: the rule's median sits on the true crossing. What the scatter does is widen it, to between 4 and 13% for thirty runs; each of the six observed crossings lies inside that range; and their all being low is one fluctuation, not six, since they come from the same thirty seeds whose means were low. The statement that capacities read from strictly periodic senders "are 1–9% lower" is withdrawn. What is true is that those senders vary 6–23 times more from run to run, so thirty of their runs locate a capacity about ten times less precisely.

How regular a real autopilot's stream is. The two simulated sources are extremes, and which one a flight controller resembles had not been measured. PX4's companion-computer link streams the global position at 50 Hz. Run in software-in-the-loop and received with a timestamp per message, that stream holds a period of 20.0 ms on the autopilot's own clock, and the central 68% of its intervals lie within ±0.37 ms of the median — the same in a six-minute flight and in a second three-minute capture. About one message in a hundred arrives more than 4 ms late, on a desktop under load; the standard deviation, dominated by those, is 1.3 ms.

So a PX4 stream is nearer to the strictly periodic source than to the redrawn one: a node keeps its place in the period to within about one frame's airtime. That matters for the spread between runs and not for the mean. A jitter of 0.4 ms is a hundred times the 4 µs within which two frozen senders collide in every period, so the pathological runs — nodes

that deliver nothing — do not arise; but the same neighbours contend with one another period after period, which redrawn phases would mix. The capacities of this chapter are read from the mean over runs and are the same under both sources; their intervals are those of the redrawn source and are the tightest case.

Remark (A timing measurement spoiled by its clock). The statistic committed before the flight was the standard deviation of the intervals between receive timestamps. It came out at 54 ms and says nothing about PX4: the measuring host's wall clock was stepped back eight times during the flight, by about 2.3 s each, and ran some 6% fast in between. The figures above come from the interquantile spread, which a handful of clock steps cannot move, and from a second capture stamped with a monotonic clock. The prediction is recorded as not scored, not as held. A wall clock should never have been the only clock of a timing measurement; the recorder now reads both.

All of this is one autopilot, in simulation, with no radio and no serial link.

8.5.3 Why the crossing is where it is

The line of Equation (8.1) describes the crossings; it does not explain them, and its one constant was fitted. An explanation needs no more than the access rule.

What is known, and what is added. The first of the two mechanisms below is not new. Ma, Chen and Refai count the simultaneous transmission of stations within range among the losses of non-saturated vehicular broadcast [46]. Cao *et al.* analyse periodic IEEE 802.11p broadcast in a fully connected network with a perfect physical layer: a packet that finds the medium idle is sent at once, one that finds it busy backs off within a fixed window, and collisions arise only among packets that backed off; a fixed point gives the delivery ratio, which they check against a simulation of their own [47]. That is the tie of this section, in the setting of this chapter, and it was found only when this thesis was audited: the derivation below had been written without a search for prior work. Neither analysis has a counterpart of the second mechanism, the window in which a transmission cannot yet be sensed, which is between a sixth and a half of the loss at the crossings found here. What this section adds is that term, a comparison with a packet-level simulator at eighteen configurations instead of with a simulation that shares the model's assumptions, and predictions written down before their runs. Their fixed point was not evaluated at these configurations: it uses the mean number of packets in a collision, for which the paper gives no expression.

In this scenario — broadcast, so no acknowledgement, no retransmission and a contention window that never grows — a frame is lost in exactly two ways.

1. **A tie.** A frame that arrives while the medium is held, by another frame or by the DIFS after it, draws a counter uniformly from $\{0, \ldots, W-1\}$, counts idle slots and is

sent at zero. Two stations whose counters reach zero in the same slot send together. Any two stations that wait at the same moment tie with probability $1/W$, whether both have just drawn or one holds a counter frozen from an earlier round.

2. **The detection window.** A station cannot sense a transmission during its first $\delta = 4\,\mu s$, the simulator's preamble-detection time. Whoever decides to send inside that window — a frame arriving on an apparently idle medium, or a counter running out — sends as well.

A closed form with nothing fitted. Let $\rho = (N-1)\,f\,T$ be the share of time the other stations hold the medium. A frame defers with probability ρ. Treating the deferring frames as a queue with deterministic service, a deferring frame finds $\rho/(2(1-\rho))$ others waiting and as many again arrive while it waits, so it shares its wait with $K = \rho/(1-\rho)$ others; and another station starts within $\pm\delta$ of any frame at rate $(N-1)f$. Hence

$$\text{loss}(N) \;\approx\; \rho\left[1 - \left(1 - \tfrac{1}{W}\right)^{\rho/(1-\rho)}\right] \;+\; (N-1)\,f\,2\delta. \tag{8.2}$$

With $W = 16$ and no free constant this puts the 18 simulated crossings within 5.5% (mean 2.3%), its error ordered by frame length: the time counters take to run down, which the queue ignores, is a larger share of a short frame.

What the fitted slope was. To first order the tie term of Equation (8.2) is $\rho^2/(W(1-\rho))$, so the line's slope is $a = \rho/(W(1-\rho))$. The configurations reach 5% loss at occupancies of 0.48–0.60, 0.53 on average, and at that average the expression is 0.071 — the value the fit found. One constant served because the occupancy at the crossing varies little; it served only to a few percent because it does vary. The slope sat near $1/W$ because $\rho/(1-\rho)$ is near one there, not because it *is* $1/W$; and the line's constant c is 2δ exactly. So the line is a tangent to Equation (8.2) at one loss level, which is the reason the remark above confines it to that level.

The exact version of the same two mechanisms is an event simulation of the access rule and nothing else: arrival times, carrier sense, counters and the backoff a station performs after sending. It has no physical layer, no channel and no packets, shares no code with ns-3, and its constants are the standard's — a 9 µs slot, a 34 µs DIFS, $W = 16$ — and δ. Run at every point ns-3 has been run at, thirty seeds each, it reproduces all 18 crossings within 1.2% (mean 0.5%) and every one of the 153 delivered fractions within 0.002. Ties are 51–85% of the loss, more for long frames; the window is the rest.

Remark (A comparison is not a prediction). The model was written beside one of those configurations, and an omission in it — half of the detection window, the case of a frame arriving just *before* a counter runs out — was found from that configuration's numbers. Agreement with data one has looked at proves little. What the model was then asked for

were crossings in conditions no run had been made at: the contention window doubled, and the 2% and 10% loss levels. There the candidate explanations part — for the per-record frame with the window doubled, the fitted line with its slope scaled by $1/W$ gives 53 nodes, Equation (8.2) gives 43 and the event model 39. Those predictions were committed before their runs.

The predictions held. All six simulated crossings fell inside the ±3% registered for them, the worst 1.7% from the model's value. With the contention window doubled the per-record configuration reaches 39.2 nodes where the model said 39.1 and the design 139.6 where it said 139.0. Doubling the window therefore buys 11–12% more capacity, not twice: a station that counts down for twice as long stays waiting longer, so more stations wait together and tie, and the detection window is untouched. A reading of the fitted line as "slope = $1/W$" would have promised 53 nodes. The capacities of this chapter are thus explained by two named mechanisms and constants of the standard — within the simulator, and within one collision domain.

Remark (What this does not show)*.* The model and ns-3 implement one standard. Their agreement says the capacities of this chapter follow from that standard's access rule and a detection time of 4 µs. It does not say a radio behaves so: hidden terminals and a real preamble detector are outside both, and capture was outside both until Section 8.5.4. The question was put to two radios (Section 9.4): they keep the rule where contention can be isolated, and depart from the model below saturation.

8.5.4 Nodes spread out, and capture

Every capacity so far is of one scenario: all nodes at one point, equal received power, so that a frame which overlaps another is lost at every receiver. A review of the paper on 2026-10-09 ranked that first among its weak points. The part of "less idealised" that could be predicted was taken up at once: unequal received power, and with it capture.

The scenario. Two options were added to the simulated scenario, both absent by default so that the published one is invoked exactly as before; the rebuilt simulator reproduces stored runs bit for bit. One places the nodes uniformly in a disc. The other makes received power fall with distance as in free space.

The model. The access-rule model of Section 8.5.3, unchanged, with one question added for every group of frames that start together: does a receiver that is not itself sending get the strongest of them at least 4 dB above the sum of the rest? If so it receives that frame, and none of the others. The 4 dB is not chosen. It is the simulator's default for detecting a preamble, read in its source, where the rule is also to be found: of the preambles that arrive within one detection period the strongest is kept. Nothing is fitted. With power

falling as the inverse square of distance the ratios do not change when the disc is scaled, so the model gives one answer for every radius: 39.5 nodes for the baseline and 139.7 for the design, 12% above the equal-power crossings.

Two radii, two kinds of prediction. At 15 m the disc is 30 m across and the simulator's energy detection reaches 37 m: every station senses every other exactly as in the published scenario, and only the received powers differ. There the model should hold, and the prediction was a point with a band of ±5%. At 100 m, a swarm's scale, most pairs are sensed by their preamble and not by their energy, and a station that can lock onto neither of two colliding frames may take the medium for idle and send over them. The model has perfect sensing and cannot show that, so the prediction there was a bracket: no lower than the published equal-power crossing, no higher than the model's band. All four were committed before any run (Appendix B).

Table 8.7: The lean baseline (146 B at 50 frames/s) and the design (173 B at 12.5) with the nodes uniform in a disc and free-space loss: the crossing of 0.95 delivery as predicted and as simulated (30 seeds, each with its own placement; 95% interval).

	radius	predicted	simulated	from the model	above equal power
baseline	15 m	39.5, ±5%	39.6 [39.5, 39.7]	+0.2%	+12.3%
design	15 m	139.7, ±5%	140.4 [139.8, 140.8]	+0.5%	+12.7%
baseline	100 m	bracket	39.4 [39.3, 39.5]	−0.3%	+11.7%
design	100 m	bracket	139.3 [138.9, 139.8]	−0.2%	+11.9%

All four held, and the model is within 0.5% of every one (Table 8.7). Three things follow. **The published capacities are conservative for unequal received power**: with the nodes spread out a neighbourhood is 12–13% larger, not smaller. The ratio of design to baseline does not move: 3.5× at 15 m and 3.5× at 100 m, as with equal power, because capture rescues the same share of both. And imperfect sensing, the mechanism the model lacks, costs at most 0.7% at this load: the 100 m crossings sit just under the 15 m ones. The expectation written beside the predictions was "a few percent"; it was the right sign and too large.

Remark (What this does not show). Hidden stations. Both discs keep every pair within decoding range, so every station still hears every other. A swarm wider than about 370 m in this simulator's terms has stations that cannot hear each other at all; that lowers the capacity, and the next subsection simulates it. Nor does this test show fading, mobility or an antenna pattern, or that the simulator's capture rule is a radio's. What it removes from the list of doubts is one item: that equal power was the favourable assumption. It was the unfavourable one.

8.5.5 A spread, moving swarm

Hidden stations appear when a swarm is spread wider than one sensing range. The nodes were placed uniformly in discs of radius 100 m to 1 km, and in a grouped layout of two clusters with nodes between them, with log-distance path loss, Nakagami fading and the capture rule above, thirty seeds per point. The design keeps its advantage: the ratio of the crossings lies between 3.46 and 3.58 in every layout (Figure 8.4). At a radius of 1 km, 11% of the nodes able to spoil a frame cannot hear its sender.

What spreading changes is what a capacity counts. Counted in neighbours within reach, it falls, to 8.7 at 1 km, because a receiver contends with every node in its carrier-sensing range [48], not only with those it counts. Counted in nodes over the area, it rises to 218. Hidden stations are a minority of the spoilers: the registered prediction that spreading would cost capacity *because* of them overstated their part.

With every node on its own Gauss–Markov path at up to 15 m/s [3], delivery changes by at most 0.0005 and the ratio stays 3.57–3.63. The channel has no Doppler and no time-correlated fading, so motion changes who contends with whom and nothing else. A first test that moved the whole group rigidly could not tell its speeds apart — a rigid translation changes no distance — and was replaced by independent motion; both are registered (Table B.2).

8.5.6 A batch chosen at run time

The batch of four is fixed by the deadline at the operating point. A controller can instead choose b at run time from the share of time the channel is busy, as the node measures it: a small batch while the swarm is small, for fresher records, and a larger one as it grows. One was built into the simulator — b moves one step a period, and only after two periods agree — and tested on a swarm that grows while it flies, on two swarms that merge, and on a moving one, each registered (Table B.2).

As registered, with a margin of 0.01 on the target, it met $V \geq 0.95$ in 75% of one-second windows and failed at 76 nodes; with a margin of 0.02 it met it in 93% and failed at 116, against 99% and 127 with $b = 4$ fixed (Figure 8.5). It is fresher: by 21 ms of peak age at the design's load, and by 90 ms below thirty nodes. These ages include the instant at which the simulated source sends each frame, drawn over the next period (Section 8.5), and are on average half a frame period longer than those of Section 4.10.1. Choosing the batch at run time buys freshness at the cost of reliability margin; it does not raise capacity. The moving scenario could not tell the controllers apart: every threshold was exceeded from its first second.

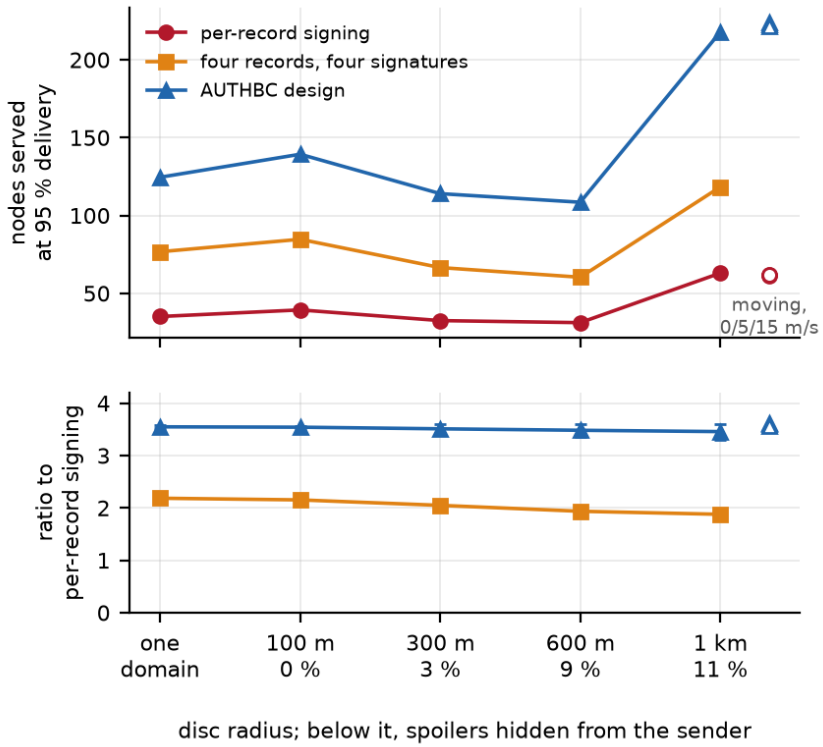


Figure 8.4: Nodes served at 95% delivery as the swarm spreads over a disc (top), and the design's ratio to per-record signing with its 95% interval (bottom). Under each radius, the share of spoilers hidden from the sender; hollow markers, the swarm at 1 km moving at 0, 5 and 15 m/s.

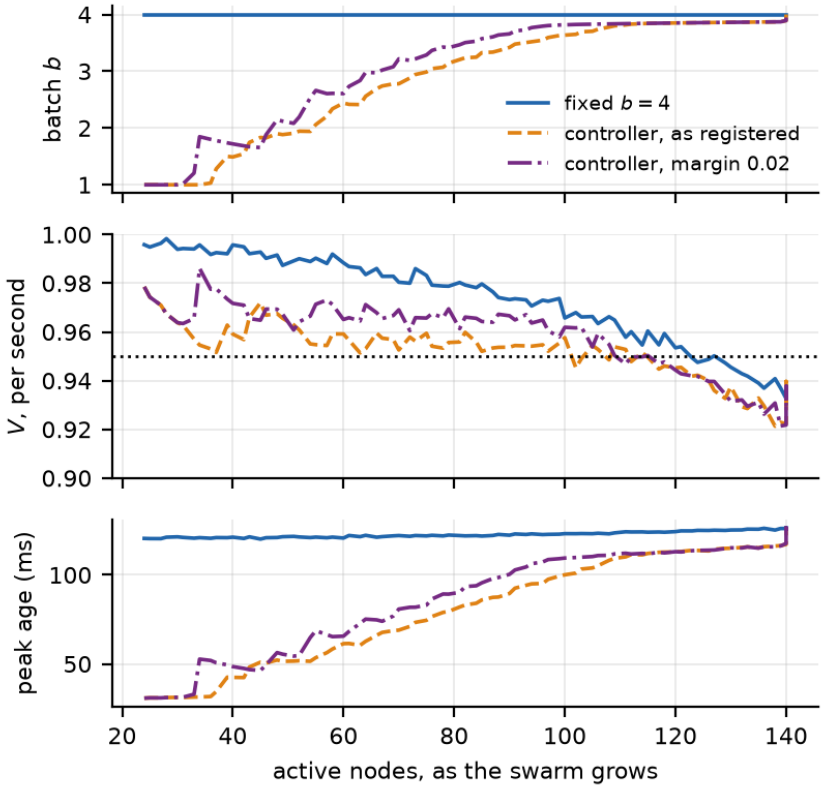


Figure 8.5: A swarm growing in flight, mean of ten seeds: the batch each policy chooses, the share of records verified in each second, and the peak age of information.

8.5.7 Other PHYs, and a post-quantum signature

The capacities above are of 802.11a at 6 Mb/s. The crossings were simulated again on 802.11a at 12 and 24 Mb/s, on the 10 MHz channel of 802.11p at 3, 6 and 12 Mb/s, and with a post-quantum signature (Table 8.8). The design serves 3.39–3.80 times the nodes of per-record signing, more at faster rates: a faster PHY shortens the payload but not the fixed cost of a frame, so the count of frames weighs more. The access-rule model of Section 8.5.3 stays within 1.5% except at the fastest rate of each channel width, where it is 2.9–4.6% optimistic; the registered ±3% failed there.

Falcon-512, with its padded signature of 666 B, fits a frame. Per-record signing then serves 12.4 nodes and the design 43.5, 3.5 times as many, as with Ed25519: the gain comes from sending a quarter as many frames, whatever the size of the signature, and the registered expectation that it would weigh more with a larger signature failed. On the board Falcon-512 signs in 638 µs and verifies in 88 µs, faster than Ed25519 verifies (259 µs). An ML-DSA signature of 2 420 B does not fit a frame at any batch.

Table 8.8: Nodes served at 95% delivery in ns-3 on other PHYs and with a post-quantum signature, and the access-rule model's error (negative: optimistic; the larger of the two rows). Frame: the design's, on air.

PHY and rate	frame (µs)	per record	design	ratio	model (%)
802.11a, 6 Mb/s	338	35.3	124.7	3.54	+0.5
802.11a, 12 Mb/s	198	50.5	184.2	3.65	−1.5
802.11a, 24 Mb/s	126	63.0	239.7	3.80	−4.6
802.11p, 3 Mb/s	666	20.4	69.1	3.39	+0.4
802.11p, 6 Mb/s	386	30.7	109.6	3.58	−1.1
802.11p, 12 Mb/s	242	41.2	151.6	3.68	−4.0
Falcon-512, 802.11a 6 Mb/s	1 142	12.4	43.5	3.5	+0.6

8.5.8 Flooding

A forged frame — a real sender, a newer sequence number and a well-formed signature by another key — costs the interpreted receiver 1.08 ms on the board, against 1.44 ms for an honest one, which also pays for its storage: one core takes 928 such frames a second. With 41 senders on one receiver core (Section 6.6), 247 forged frames a second push honest delivery below 0.95, and the cheap checks of Section 5.5 do no better (Figure 8.6). The group tag refuses an outsider's frame in 22.5 µs, 44 400 a second per core: at 3 000 forged frames a second, more than the 2 857 that one channel carries, honest delivery stays at 0.984.

A member holding the key is stopped neither by the tag nor by a per-sender rate cap: delivery falls below 0.95 from 129 forged frames a second behind the tag, and from 123 behind the cap when the forgeries name every sender. Against one forging sender the cap

keeps delivery at 0.96 or more up to the rate one channel carries. The tag costs 3.1% of capacity, eight bytes on every frame.

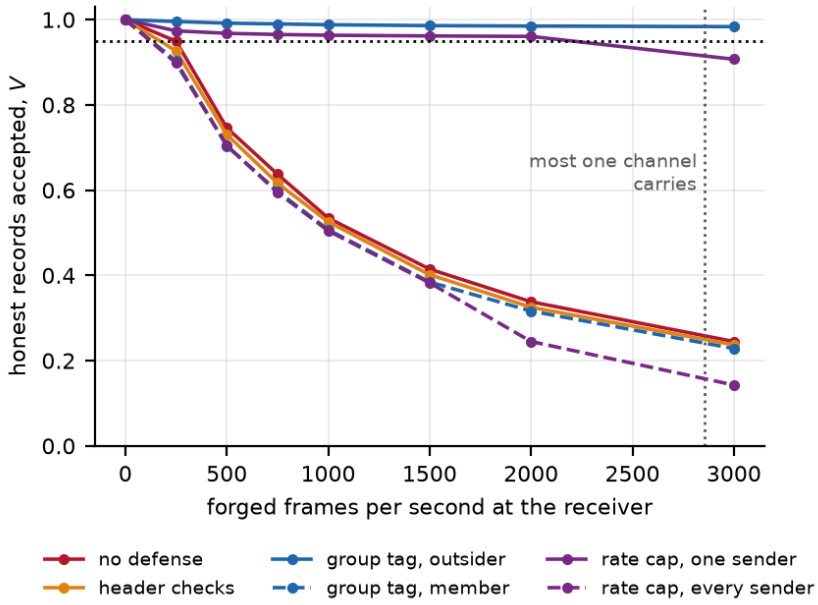


Figure 8.6: Honest delivery on one Raspberry Pi 4 receiver core with 41 senders, against the rate of forged frames, for each defence and attacker. The dotted vertical line is the most one 802.11a channel carries.

8.6 Freshness, with every term

Table 8.9: Delay of the oldest record in a frame, $b = 4$, 50 records/s. Channel delay is simulated at the heaviest load that still delivers 95% of frames.

	fill (ms)	channel (ms)	verify (ms)	total (ms)	margin (ms)
Ed25519, mean channel delay	80	0.76	0.26	81.0	19.0
Ed25519, 99th percentile	80	2.40	0.26	82.7	17.3
Ed25519, worst observed	80	3.49	0.26	83.8	16.2
BLS, worst observed	80	3.49	8.61	92.1	7.9

The freshness model of Chapter 4 was fill time plus airtime. Table 8.9 adds the two terms it left out: channel-access delay, which had been measured and never added, and verification time. Fill time dominates; the oldest record is verified within 83.8 ms of being generated. A fifth record would need a fill time of 100 ms by itself, which is why the batch stops at four.

Other rates and deadlines. The batch of four belongs to 50 records/s and 100 ms. Figure 8.7 evaluates Equation (4.2) over seven record rates and eight deadlines, with the

capacity each batch buys from the access-rule model. At 15 cells, chosen as distinct roles, the ns-3 crossings lie within 1.05% of the model; at 100 records/s and 100 ms, where the deadline admits a batch of 9, the design serves 6.63 times the nodes of per-record signing. Where $\Lambda_i D_{\max} \leq 2$ the batch is at most one record, and nothing is gained.

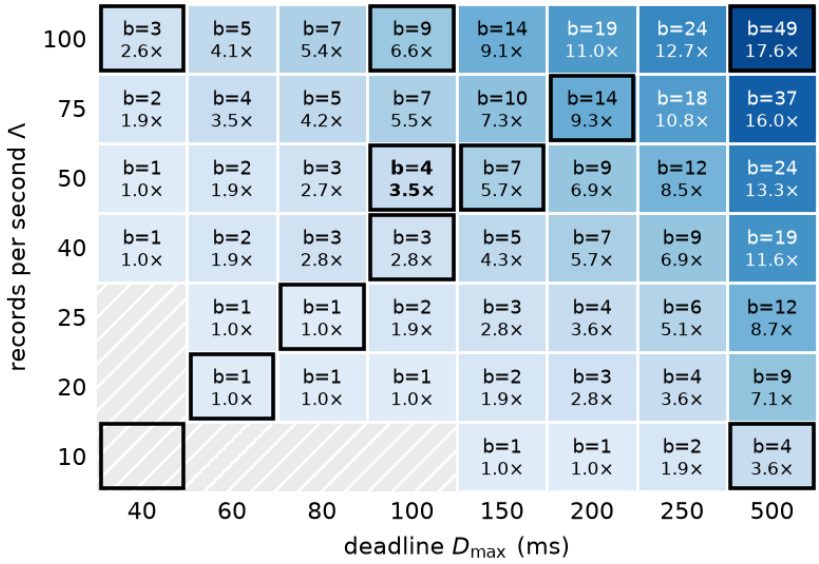


Figure 8.7: The design map: the batch b^* each record rate and deadline admit, and the capacity it buys over per-record signing by the access-rule model. Outlined: cells simulated in ns-3, for per-record signing, the design or both; hatched: no batch meets the deadline.

8.7 Certificates

The rows so far assume that receivers already hold the senders' public keys. Vehicular practice sends a full certificate in every fifth message and an eight-octet digest in the others [23]. Charging that to every frame — a 162 B certificate, hence 38.8 B per frame on average — gives the first "with cert." column of Table 8.1. Charged per frame, certificates are amortised by batching too, and both the baseline and the design are charged.

Remark (The per-frame charge favours the design). The source's "every fifth message" is of ten messages a second: one certificate in every 500 ms. Charged per frame, a baseline of 50 frames a second sends one every 100 ms, five times what the practice it copies sends, while the design at 12.5 frames a second sends one every 400 ms. The first column therefore charges the baseline 184.57 B per record and the design 52.95.

The second column charges one certificate per 500 ms at any frame rate, and the digest in every other frame: 159.93 B against 51.41. Under that reading a batch amortises the digest and not the certificate, and the baseline is about an eighth cheaper than the first column made it. At ten frames a second the two readings are one, which is why the first is a reading of the source and not an error; it is the one that leans this work's way, and it

stood alone in the table until a review of 2026-10-09.

These are *explicit* certificates: a verification key and the issuer's signature over it. IEEE 1609.2 also defines *implicit* certificates (ECQV), which carry only a reconstruction value from which the key is derived with the issuer's root certificate; the source this accounting follows gives a signed message of at most 226 B with an implicit certificate against 330 B with an explicit one [23]. Implicit certificates are not priced here, so both columns are upper bounds on what certificates cost either design. Pricing them is future work; it would lower both rows and leave the comparison between them where it is, since whichever certificate is used is paid once per frame.

Remark (Why the order of this work matters). The certificate term was added to the byte model *before* the published figures it is compared with in Chapter 2 were retrieved. Adding it afterwards would have been fitting the correction to the answer.

8.8 Energy

Table 8.10: Sender energy per record on a Raspberry Pi 4: metered (median and range of five repetitions) against time multiplied by power. The first-format rows are of July 2026; the lean rows are of October, on the second board through the second sensor, where a control that repeats the first row gives 116.2 µJ; the compiled rows are of 22 October, on that board, beside a control giving 118.0 µJ.

	metered (µJ)	model (µJ)	gap
first format, every record signed	118.8 [118.5, 119.9]	105.8	12.3%
first format, one signature per four	58.4 [58.1, 59.3]	54.3	7.5%
lean format, every record signed	273.1 [270.6, 276.9]	284	−3.9%
lean format, one signature per four	155.4 [155.1, 155.8]	163	−4.4%
compiled, every record signed	60.7 [60.2, 62.4]	70.2	−13.6%
compiled, one signature per four	17.1 [16.9, 17.1]	19.4	−11.6%

The batched sender spends 51% less energy per record in the first format and 43% less in the lean format. Only the sender is metered; receiver and radio energy are modelled.

The two pairs of rows are compared with two different models. For the first format the model composes separately measured times with one processor power, and that power is the median over the metered runs of July themselves (Section 6.6). That comparison tests the times, not the power: the model is 7.5–12.3% low, the gap being frame assembly in an interpreted prototype. For the lean format the model is the time of one whole frame on the board (0.87 ms for four records) at that same power, written down before the meter was read; the meter is within 5% of it, on the low side. One busy core adds 0.72 to 0.74 W on the second board where the constant taken from the first is 0.749, and with four records to a frame the loop that is metered runs a frame 3% faster than the loop that was timed.

Remark (What the lean rows are, and what they say). The lean rows were metered in

October 2026 on the second board through the second sensor: the first board had been re-installed since July, and the rig had first to pass a test it failed three times that morning (Section 6.6). A control that repeats the first row of the table on that rig gives 116.2 µJ [114.8, 120.7] per record and 0.737 W where July gave 118.8 and 0.755: 2.2% apart, inside the range written down for it, so the four rows can share a table. The range of that control is wider than the others' because the author's own logins to the board fell in one of its windows.

The lean sender costs 2.7 times the energy per record of the first format's. The lean codec was written for the size of the frame and is slower to run in this interpreted prototype. The saving in bytes and the capacity reported in this thesis are the lean format's, and so now is its energy: it is the larger of the two, and the table says by how much. The absolute values still carry the sensors' gain, which no reference load has checked; that two sensors on two boards agree within 2.2% on one workload bounds their difference and not their common error.

Compiled, the order follows the bytes. The sender of Section 5.6 was metered in October 2026 on the same board, with the rig check passed that day and the sensor's gain taken as calibrated. It spends 17.1 µJ per record with four records to a frame, 3.4 times less than the first format with batching, and 60.7 with every record signed. The reversal above was the interpreter's. Against the model the meter reads −11.6 and −13.6%, low because a compiled loop adds 0.66 W above idle where the model's one power is 0.749 W: the registered ranges, set with that power, missed low (Table B.2). A control that repeats the first row that day gives 118.0 µJ, inside the range registered for it.

Chapter 9

Model Validation and Hardware

9.1 Three independent implementations of the same physics

The broadcast channel model gives the saturation bound N_{sat} of Chapter 8 and was the input to the envelope that chapter used to report, so it is validated against two independent sources and not one.

Table 9.1: Saturation throughput (Mb/s), 1400 B broadcast frames. The slot-exact simulator was written *before* the published model was located and shares none of its assumptions.

N	published model	slot-exact sim	Δ	NS-3	Δ
5	4.3329	4.3305	−0.06%	4.3318	−0.03%
10	3.1319	3.1321	+0.00%	3.1288	−0.10%
20	1.7823	1.7830	+0.04%	1.7866	+0.24%
35	1.2407	1.2407	−0.00%	1.2343	−0.51%
50	1.2595	1.2585	−0.08%	1.2541	−0.43%

Agreement is $\leq 0.08\%$ against the independent simulator and $\leq 0.51\%$ against NS-3. Unicast agrees with the acknowledged DCF model to $-0.40/+1.29\%$ across $N = 5$–50, with fixed-point residuals below 10^{-12}.

9.2 The mechanism, confirmed by removing it

The reason broadcast needs its own model is a specific asymmetry. A station that has just transmitted redraws its backoff and may draw zero, taking the medium immediately after the interframe space, while every station that deferred necessarily holds a counter of at least one. In unicast the acknowledgement timeout blocks colliders from doing this; in broadcast nothing does.

The slot-exact simulator can disable *exactly* that asymmetry and nothing else. Doing so collapses it onto the naive reduction — a 16.9× gap at $N = 50$. **The mechanism**

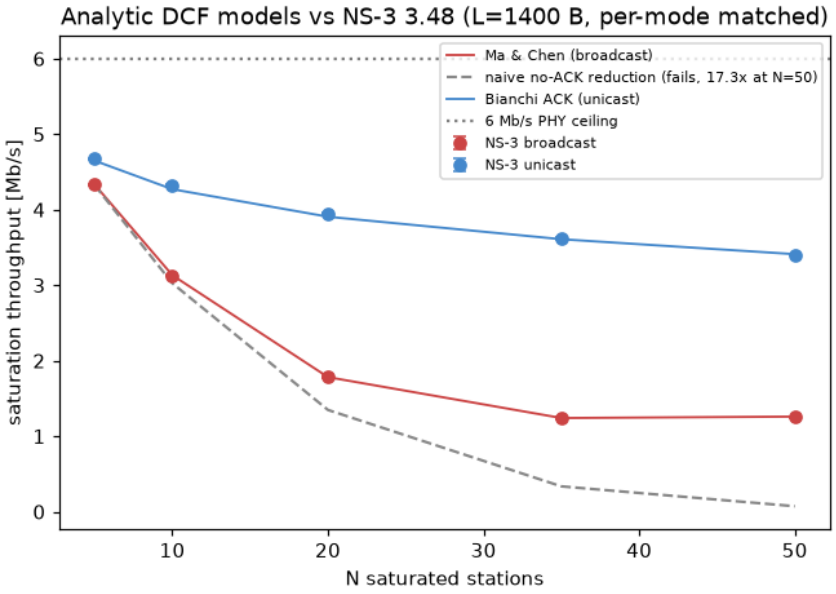


Figure 9.1: Analytic models against simulation, both modes.

attribution is therefore not an inference from agreement; it is a controlled intervention.

Remark (An expensive lesson). This project used the naive reduction — the unicast model with the acknowledgement removed and the transmission probability fixed — for several phases. It under-predicts by 16× at $N = 50$. The published broadcast model's own authors open by warning that unicast models cannot simply be reduced for broadcast analysis. *The failure was reading a model without reading its preamble.*

9.3 A non-monotonicity that is real

Saturation throughput is **not monotone decreasing** in N: it dips near $N \approx 35$ and recovers. All three implementations reproduce it, so it is a property of the freeze-process series rather than a bug — each freeze stage contributes a hump peaking at a station count set by the contention window.

This matters operationally. The capacity search breaks on the first violating N, which is valid only if utilisation is monotone. It is — because the explicit factor N in the numerator outruns the recovery — but **that is an arithmetic accident, not a structural guarantee**, and it is now verified exhaustively rather than assumed.

9.4 Hardware

Two Raspberry Pi 4 boards in ad hoc mode on a 5 GHz channel, broadcasting 1400 B frames at 6 Mb/s. Four things were measured on them: the loss of the link, how one radio

spaces its own frames, the loss when both send at a set load, and the loss when both always have a frame to send. The second and the fourth were not planned. They are what the third made necessary.

9.4.1 The link

With one transmitter at 100 frames/s, 4 frames of 17 600 were lost in August 2026 ($p = 2.3 \times 10^{-4}$), and none of 17 600 when the same session was repeated in October. This is the loss of a link with no contention.

Remark (A figure withdrawn). Earlier versions of this chapter and of the paper reported "airtime 1.995 ms per frame against 1.988 ms predicted, a gap of 0.36%", and called it an independent check on the timing constants. Both figures were wrong, in opposite senses. The measured one was the rate at which the sender's operating system accepted frames, which counts those still queued when the sender stops; on air the frames were 2.019 ms apart. The predicted one left out the 64 B of headers every frame carries and assumed the standard's backoff; the standard's figure is 2.078 ms. An agreement of 0.36% did not invite a second look, and for two months it did not get one.

Remark (A warning earned earlier). An initial 2.4 GHz sweep produced a tidy 97.45% delivery figure that was **saturation at the 802.11b 1 Mb/s broadcast basic rate**, not channel loss. It was caught by a pre-stated prediction plus a load sweep, and is kept in the record labelled as what it was. Repeatability did not protect against it — the standard deviation across windows was small. *A precisely reproducible measurement of the wrong thing looks exactly like a good measurement.*

9.4.2 How one radio spaces its own frames

A saturated station that keeps to the standard sends a frame, waits DIFS, counts a number of slots drawn from 0 to 15, and sends the next: for this frame 2.078 ms from one frame to the next, of which 101.5 µs is the gap. A receiver can measure that time without any help from the sender: the sequence numbers it spans, over the time from the first frame it hears to the last (Table 9.2).

Table 9.2: Time from one frame of a saturated sender to its next, read at the receiver. The frame itself is 1976 µs on air.

	per frame (ms)	gap (µs)
the standard's rule	2.078	101.5
this radio, the driver's settings (August and October)	2.019	43
this radio, the vendor's frame-burst mode off	2.050	74

The radio does not keep the standard's rule between its own consecutive frames. The Linux driver switches the chip's *frame-burst* mode on whenever it configures

it [57]; with that mode off the spacing moves toward the standard's and stops short of it. This is not peculiar to one chip: Bianchi *et al.* measured the same quantity on six commercial cards, with an instrument built for the purpose, and found that none of them kept to the standard [58]. The firmware is closed. What it does is inferred here from a mean spacing; no single frame was timed.

9.4.3 Two radios at a set load: the registered experiment

What two boards can test is the mechanism the capacities are made of: the loss among stations that all send, at a known offered load. Every board sends one frame per period at a redrawn instant and counts what it receives from the other. The model of Section 8.5.3 was asked for its predictions in advance: with two boards filling half, seven tenths and 85% of the medium, losses of 0.29, 0.64 and 1.40%; with three, 0.66, 1.52 and 3.00% (Appendix B). The band registered for two boards was 0.6 to 1.4 times the model's loss. The two-board case is sharp in one respect: the only receiver of a frame is the other transmitter, so no third radio can capture one of two colliding frames. ns-3 was run at the same points first: it agrees with the model within 5% at three to five nodes and is 9–18% below it at two, a small bias of the model at the smallest network that nothing so small had exposed.

The session was run with two boards on the day it was registered: twelve windows, all of them usable. **The prediction held at the first two loads and failed at the third**, where the radios lost 0.46 of what the model predicts (Table 9.3). Loss rises with load, as predicted, and the two directions lose nearly the same number of frames in every window, which is what collisions between two half-duplex radios do and what a lossy link does not.

Table 9.3: Frames lost between two radios, in percent. Three sessions of twelve windows on one night; the last column is all 36 windows over the model's prediction.

medium filled	predicted	registered band	registered session	frame burst off	repeat	all windows	over the model
0.50	0.29	0.19–0.40	0.29	0.32	0.38	0.33	1.13
0.70	0.64	0.39–0.89	0.51	0.52	0.48	0.50	0.79
0.85	1.40	0.85–1.95	**0.64**	**0.84**	0.86	0.78	**0.56**

A follow-up in which all four predictions failed. The spacing of Section 9.4.2 suggested a cause: a radio that sends its queued frames one behind another, without contending again, would be described well at low load and lose less than the model at high load. Four predictions were registered before any run with frame burst off (Appendix B). One saturated sender would then take 2.070–2.095 ms per frame: it took 2.050. Two boards would then fall inside the original bands: at the highest load they lost 0.84%, under a band that begins at 0.848%. A repeat of the first session, frame burst on, would

agree with it: at the highest load it gave 0.86% where the first had given 0.64. And the loss with frame burst off would be at least 1.5 times the loss with it on: against the repeat it is no larger at all.

The repeat is what this follow-up was worth. After the run with frame burst off the author was ready to write that the vendor mode explained a third of the shortfall. **Frame burst changes one sender's spacing and has no demonstrated effect on the loss between two boards**; the difference that would have been reported was the scatter between sessions. That scatter is itself larger than was assumed: 0.20 percentage points between windows at the highest load, where frames lost in independent pairs would give 0.13.

What the design could not separate. Two boards can fill the medium only if each sends often. At 85% each board sends a frame every 4.7 ms and its frame lasts 2 ms, so a board frequently holds two frames of its own, and how a radio sends its own queued frames matters as much as how two radios contend. At the operating point of this thesis — 124 stations, each sending 12.5 frames a second — a station almost never holds two. Below saturation, then, a two-board experiment mixes two things, and the capacities depend on one of them.

9.4.4 Two saturated radios

The case that isolates the contention is the one every treatment of the access rule begins with: both stations always have a frame to send. It needs nothing from a traffic generator. It was registered before its runs, together with the author's own expectation, which was that the shortfall belonged to the contention and that about 7% of frames would be lost.

Table 9.4: Two radios, each offered 400 frames/s where the medium carries about 490. Frames on air are read at the receiver. Eight values per column.

	the standard's rule	frame burst off	frame burst on
frames on air that are lost (%)	11.8	12.4	12.1
frames on air per sender (1/s)	260	263	265

The standard's figure follows from two lines of arithmetic: with a contention window of 16 that never doubles, each of two stations attempts in a slot with probability 2/17, and a collision costs both frames. **The radios lose 12.4% where the rule gives 11.8%**, inside the registered band of 7.1–16.5% and about a twentieth above the figure itself; the expectation of 7% was wrong. Frame burst makes no difference here either.

9.4.5 What the hardware supports, and what it does not

- **Where contention can be isolated, the rule the simulated capacities use was observed on two radios, within** 6%**.** This is the first measurement of that mechanism in this work that did not run inside a simulator.
- **Below saturation the same radios depart from the model**, from 1.13 times its loss with the medium half full to 0.56 times at 85%. The registered prediction failed there. The cause that the saturated case leaves standing is the radio's handling of its own queued frames, which is measurably not the standard's; that it is the cause has not been shown. Why the loss is *above* the model at the lowest load is not explained.
- **No capacity was measured.** Two saturated stations are the opposite corner from 124 stations at 12.5 frames/s. Every capacity in this thesis remains a simulated one, for radios that keep the standard's rule.
- One chip family, one room, two boards. The predictions for three to five boards are registered and have not been run.

9.5 The utilisation ceiling, and a flaw in the author's own pre-registration

The capacity envelope applies one measured utilisation ceiling to configurations whose frames span 153–299 B, but that ceiling had only ever been measured at 288 B. A second sweep at 174 B — the compression-only baseline frame, which produces the denominators the capacity ratios divide by — tested it, with the prediction committed data-free beforehand.

	288 B	174 B
$V \geq 0.95$ crossing	2.435	2.367
combined standard error	± 0.151	
significance of the shift	0.45σ	

Statistically indistinguishable across a 1.66× **change in frame size.** At the time this was read as justifying the universal ceiling. It justified half of it, at one N; the other half is below.

Remark (The prediction that should not have been made). The pre-registration also predicted a *direction* — that the crossing would drift higher — and offered a mechanism. It drifted lower, at 0.45σ. The experiment has no power to resolve the sign, so had the noise fallen the other way a confirmation would have been recorded that had not been earned, and the mechanism offered is neither supported nor refuted.

This is recorded rather than quietly dropped because it is the same defect class the rest of this thesis catalogues: **a number that looks like evidence and is not**. A directional prediction needs a power estimate, or must be stated as a band. The load-bearing prediction did carry one, which is why the run answers its question at all.

9.6 The other half: invariance in N, tested, and it does not hold

Both sweeps above are at $N = 50$, while the envelope applied the ceiling from $N = 2$ to $N = 213$. For two months that half of the assumption was listed as stated and not measured. An external review asked for confidence intervals on the 802.11 capacities, "as for LoRa", and the LoRa figure is a direct search in N; so the 802.11 figure was made one.

The prediction, committed before any run: in each of six configurations, the directly simulated $N_{\max}$ lies within $\pm 10\%$ of the ceiling's value. No direction was predicted, and a power estimate was given.

The outcome. It fails. With the scenario exactly as published (1080 runs):

configuration	frame, rate	ceiling	direct (interpolated)	deviation
first, sign every record	174 B, 50/s	31	31.0	−0.1%
first, batched design	300 B, 12.5/s	97	87.8	−9.5%
lean, sign every record	146 B, 50/s	34	32.4	−4.7%
lean, batched design	173 B, 12.5/s	142	118.3	**−16.7**%
first, shared frame	565 B, 12.5/s	49	54.7	+11.7%
lean, shared frame	372 B, 12.5/s	80	74.8	−6.6%

The ceiling overstates the capacity of short frames sent by many nodes and understates that of long frames sent by few. At the crossings U runs from 1.98 to 2.61 where the ceiling says 2.435. It is not a constant, and Chapter 8 no longer uses it.

Remark (Three defects in the measurement itself, each of which could have hidden the result). The driver first reported the *median of the bootstrap replicates* as $N_{\max}$ where the registered estimate is the sample's own crossing; they differed in one configuration (31 against 29). The grid was coarse — eleven nodes between points in the configuration that decided the outcome — so an interpolated crossing was added afterwards and is labelled as an addition. And the power estimate promised ± 2 nodes where the interval is ± 5. None changes the verdict; all three are in the record.

What follows from the failure — the traffic-source experiments, and the capacities now reported — is in Section 8.5.

9.7 The access rule away from its operating point

The access-rule model of Section 8.5.3 was fitted to nothing and predicted capacities at the operating point. In October 2026 it was put to cases it had not seen. At 15 cells of the design map — record rates from 10 to 100 a second, deadlines from 40 to 500 ms, frames from one record to 49 — the ns-3 crossings lie within 1.05% of it (Figure 8.7). On five further PHYs and rates, and with Falcon-512's frames of 776 B, they lie within 1.5% except at the fastest rate of each channel width, where the model is 2.9–4.6% optimistic (Table 8.8). The registered tolerance was ±3%, and it failed there. The cause is not established: the error is largest where frames are shortest on air, but the few microseconds in which a transmission cannot yet be sensed do not account for it alone.

The loss the simulator makes was traced as well: at the capacity crossing consecutive frames of one link are lost independently (Section 7.4), so the model of V for independent loss is the one that applies there.

Chapter 10

The Low-Rate Regime and the Exclusion Bound

10.1 Why the low-rate arm is a different regime, not a slower one

The 802.11 numbers do not transfer. They differ by two to four orders of magnitude, and the binding constraint differs *in kind*: a regulatory airtime quota instead of a frame size or a latency budget. This is the third regime in the taxonomy of Theorem 4.3:

regime	what caps the batch	where
MTU-limited	frame size	—
freshness-limited	$\Lambda_i D_{\max}$	**802.11**
duty-cycle-limited	regulatory airtime quota	**low-rate**

On this arm the arrival rate and the freshness are *derived*, not configured: a frame of airtime T may repeat only every T/duty, so the sustainable rate and the staleness of the oldest record are the same interval. There is no independent deadline to trade against bytes.

This chapter is *analysis and simulation, not measurement.* No LoRa hardware was instrumented and no LoRa energy figure is reported: the only metered radio power in this work is a Wi-Fi figure, and reusing it for a different transceiver would be fabrication. Every LoRa constant is transcribed from the SX1276 datasheet [52] and from LoRaWAN RP002-1.0.3 [53] (Tables 12 and 13; EU863-870 duty cycle below 1%).

10.2 The exclusion

Theorem 4.8 says that a link of maximum payload M admits a frame that verifies alone only if

$$M \;\geq\; H_f + L + g_a + s_k,$$

a header, one chain link, an authentication object and one record that decodes without its predecessor. An exclusion is claimed against the *smallest frame the format can emit at all*, not against the frames one telemetry stream happened to produce.

10.2.1 The floor of the format

Take the lean format of Chapter 3 with sender 0, sequence number 0 and one record in which every field is zero. Every integer then occupies its minimum width:

$$\underbrace{16}_{H_f} + \underbrace{35}_{L} + \underbrace{64}_{g_a} + \underbrace{9}_{s_k} = 124 \text{ bytes}.$$

That frame was emitted, signed and verified; it is not an estimate. **Header, link and signature alone are 115 B.** The first format's floor is higher still, because its record carries its own 32 B link and its keys are text.

10.2.2 All twelve data rates

RP002-1.0.3 defines *twelve* EU863-870 data rates: DR0–DR6 are LoRa, DR7 is FSK and DR8–DR11 are LR-FHSS. (Earlier versions of this chapter considered seven.)

Table 10.1: EU863-870 data rates against one self-contained, signed, chained frame (g_a = 64 B). The verdict is the same in both frame formats and under both of the standard's payload tables.

data rates	payload M	verdict	what stops it
DR0, DR1, DR2, DR8, DR10	50 or 51 B	**excluded**	the signature alone: $64 > 51$
DR3, DR9, DR11	115 B	**excluded**	header, link and signature fill the payload
DR4, DR5, DR6, DR7	222 or 242 B	feasible	—

Eight of twelve data rates cannot carry one signed, hash-chained frame. None of the eight depends on the telemetry: each is excluded against the floor. They fall into two groups that are excluded for different reasons, and the difference decides what could ever rescue them.

Five rates: the signature. At 50 or 51 B a 64 B signature does not fit. This holds with a zero-byte header, no chain link and no record. Only a smaller authentication object could change it.

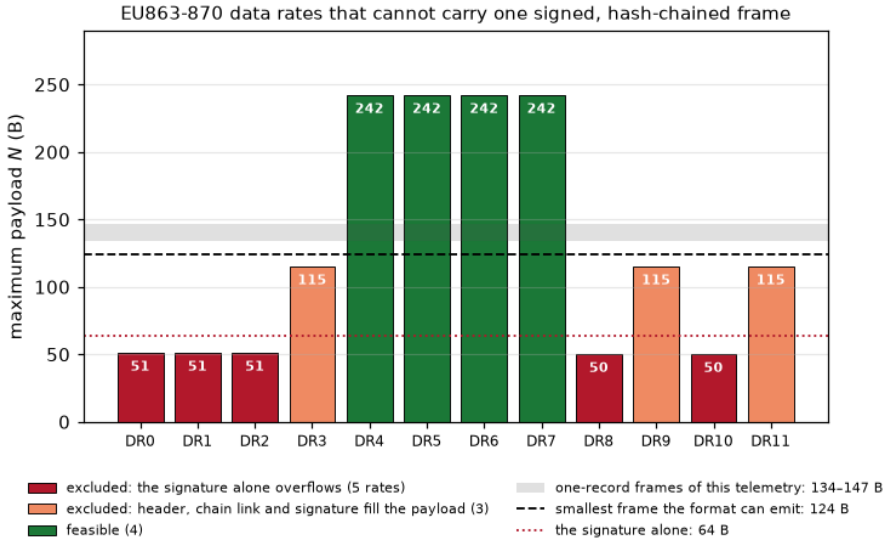


Figure 10.1: Payload limit of each data rate against the floor of the lean format and the frames this telemetry produces.

Three rates: what a frame must contain. At 115 B the signature fits, and so does the header. What does not fit is the chain link together with a record: $16 + 35 + 64 = 115$, the whole payload, before a single byte of telemetry.

Remark (Which payload column). Published EU868 tables differ. Klimiashvili *et al.* list DR3 at 123 B where this work uses 115 B [59]. Both read the standard correctly: 123 is the maximum *MACPayload* and 115 the maximum *application* payload, the two differing by the 8 B of `FHDR` and `FPort`. The application figure is the applicable one, since the frame travels inside `FRMPayload`. Under the other reading the floor of 124 B still does not fit, by one byte — too thin a margin to lean on, which is why it is recorded here and not offered as a second argument.

10.2.3 Scope

The exclusion is a statement about a particular kind of system. It holds under five conditions, and each is necessary.

1. **The limits are LoRaWAN's.** They come from the regional parameters. The LoRa physical layer itself carries 255 B at any spreading factor and is not excluded; a private LoRa network that ignores the regional payload table is outside this result.

2. **Every frame carries a public-key signature.** A symmetric tag is far smaller, and authenticates a link instead of an originator.

3. **There is no loss recovery.** The Remote ID authentication standard takes the other road — it pages a signature over several small frames and adds a parity page

(Section 2.4.3) — and so lies outside this result by construction. Retransmission or coding across frames changes what it means for a frame to arrive.

4. $\varepsilon \leq p$**.** This is what forbids spreading a signature over several frames: a unit spanning n frames verifies with probability $(1-p)^n$, so $V \geq 1-\varepsilon$ needs $n \leq \lfloor \ln(1-\varepsilon)/\ln(1-p) \rfloor$, which is 1 exactly when $\varepsilon \leq p$. At $p = 0.01$ and $\varepsilon = 0.05$ a signed unit may span five frames, and the exclusion no longer holds as stated.

5. **The frame is this thesis's format.** Five of the eight rates are excluded by the signature alone and would be in any format. The other three are not. A 115 B payload less a 64 B signature and a 32 B hash leaves 19 B, and the lean header at its floor with the framing of the link field is exactly that many. A hand-packed layout would leave part of those bytes for a record: fewer than the 24.0 B this telemetry's keyframe averages, so the three rates stay out of reach for this telemetry, but no longer whatever the telemetry. This condition was added after a review of the paper on 2026-10-09. Until then the count was stated as if it belonged to the problem, where three eighths of it belong to the format.

10.2.4 What each relaxation buys

Table 10.2: The lean format under four authentication choices. *Floor*: every field zero. *Range*: frames of this telemetry over a flight.

	floor	range	50/51 B	115 B	222/242 B
64 B signature, chain link on air	124	134–147	no	no	yes
... chain link not sent on air	89	99–112	no	**yes**	yes
48 B BLS signature	108	118–131	no	no*	yes
13 B symmetric tag	72	82–95	no	yes	yes

* a frame of zeros would fit; no frame of this telemetry does.

Three readings of Table 10.2 are worth stating.

The 115 B rates are excluded by the chain link, not by the signature. Remove the link from the air and they fit with ordinary 64 B signatures. That row is arithmetic, not an implemented format: without the link a receiver that has missed a frame cannot rebuild the next one, so removing it trades the exclusion for the dependence that Theorem 4.6 counts.

A three-byte miss is not an exclusion. The smallest standardised signature is a 48 B BLS12-381 point in the minimal-signature-size variant [60], which targets 126-bit security. It fits a 51 B payload and leaves three bytes. At 115 B a frame of zeros would fit (108 B) while the smallest frame of this telemetry is 118 B. That margin is within reach of a format change or a different payload, so the artifact grades it separately and it is *not* counted among the excluded rates.

At 50 or 51 B, hash chaining itself does not fit. Header and link are 51–58 B before any authenticator or record. Even a 13 B symmetric tag, as MAVLink 2 uses, leaves no room. The exclusion there rests on two independent facts.

10.2.5 How this result reached its present form

This section exists because the result was stated wrongly twice, and the second error was made *while correcting the first.*

First statement (July 2026). $s_{\max} = M - H_f - g_a \geq s_{\min}$, with $H_f = 44\,\mathrm{B}$, $g_a = 64\,\mathrm{B}$ and $s_{\min} = 13\,\mathrm{B}$, applied to seven data rates. DR0–DR2 were excluded by the signature and DR3 "by six bytes" (7 B of room for a 13 B record): *four of seven.*

First correction (August 2026). The header is 29 B of CBOR text key names; the same seven keys as small integers cost 7 B. An integer-keyed header of 22 B gave DR3 29 B of room, and the headline was changed to *three of seven*, with DR3 "rescued" by a header redesign. The correction was presented, in this chapter and in the paper's abstract, as an example of composing two facts that had been recorded separately.

Second correction (October 2026). An external review asked what a frame that verifies alone must actually carry. Two things had never been charged in either statement:

- the **chain link**. Per-frame chaining (Section 10.3) moves the 32 B link out of each record and into the frame, where it still has to travel. The test had subtracted it from the record and added it nowhere.
- a **record that decodes alone**. 13 B was the *mean delta* record less its link. A frame that verifies without its predecessor starts with a keyframe.

With both charged, the "29 B of room" is $29 - 35 < 0$. DR3 is excluded under either header, whatever the telemetry. And the count had been of seven data rates where the standard defines twelve.

Remark (What the second error teaches that the first did not). The first correction ended with the sentence: *a register that stores facts separately does not compose them; only re-derivation does.* Re-derivation was not enough either. Every number in the first correction was re-derived and every one was right. What was wrong was that no frame of the proposed kind had ever been *built*: the design was a sum of sizes measured in three places. The lean format now exists as code (Chapter 3), the floor above is the length of a frame it emitted, and the question "does it fit" is answered by trying.

The half that was right throughout: DR0–DR2 are excluded by the signature alone.

10.3 Per-frame chaining, and what it is worth

On 802.11 freshness binds, so the batch is fixed by $\Lambda_i D_{\max}$ and saving payload bytes buys no extra records. Here the regional payload limit binds, so bytes convert directly into records per frame, and the duty cycle converts records per frame into sustainable rate. Carrying the chain link once per frame instead of once per record is therefore worth far more on this arm.

Remark (Correction, October 2026). The factor of 3.0 first reported for this change (Figure C.3, kept in Appendix C) sized every record from a 13 B body — the delta record measured with records 50 ms apart. At the rates this arm sustains, records are seconds apart, and a difference grows with the time it spans:

spacing	50 ms	0.1 s	1 s	5.5 s	16.5 s
lean delta record (B)	9.0	10.1	12.5	13.6	16.2

The first figures also carried no keyframe, so no frame decoded without the one before it. Sized at the spacing the batch itself implies, with every frame self-contained, DR5 at 242 B gives:

	records per frame	frame (B)	records/s per node
as first reported	7	231	0.182
first format, corrected	5	240	0.13
lean format	8	241	0.20

Per-frame chaining is worth about 2.1× in the first format, not 3.0×; the lean format recovers the larger figure because it also shortens the header and the record. The decision to chain per frame on this arm is unaffected: its argument is that the payload limit binds, and that has not changed.

In the first format the 802.11 arm keeps a link in every record, because freshness binds there and the same change buys no extra records. The lean format carries one link per frame on both arms. What that gives up is stated in Chapter 3: inside a frame, order and content rest on the frame signature alone instead of on a signature and an independent hash.

10.4 Capacity

Time on air and the duty cycle are deterministic, so simulating them would be circular. One quantity is not derivable that way — how many nodes can share the channel — because LoRaWAN uplinks are pure ALOHA, with no carrier sense and no acknowledgement.

Simulating LoRaWAN scalability in ns-3 is established practice [61], including proposals to replace the ALOHA MAC with carrier sense [62] and extensions to non-terrestrial links [63].

10.4.1 A topology caveat

LoRaWAN is by specification a star of stars with no peer-to-peer mode: a FANET survey notes that it "presents some limitations regarding its star topology, its medium access control (MAC) layer and its lack of routing procedures" [64], and published peer-to-peer LoRa meshes are built by discarding the LoRaWAN MAC entirely, "not relying on LoRaWAN... without the use of gateways" [65]. A LoRa FANET that was actually flown reports the same constraint, and hybrid LoRa/802.11s UAV meshes reach for a second radio for the same reason [66]; having built one, Zirak *et al.* converge on a single channel and on dedicated transmit and receive radios to work around half-duplex [51], which is the configuration modelled here. A review of mesh proposals finds that even those pursuing flexible topologies keep "a predominant nodes → gateway/sink data flow" [67]; concrete multi-hop deployments keep the same sink-directed shape [68, 69], and a 2025 ns-3 LoRa mesh study states that "there is currently no standardised and commercialised multi-hop LoRa-based network" [70].

The scenario is therefore N end devices reporting to one receiver, where the 802.11 arm is a broadcast among peers. What transfers is the part that matters: collision statistics at a receiver depend on how many transmitters are concurrently in range, not on what the receiver is attached to. It is also why a single demodulator is the right provisioning here and not, as it would be for a gateway, an artificially harsh one: a UAV peer has one radio. Two ad hoc effects remain outside the model. A half-duplex radio cannot receive during its own 364 ms frame, costing a peer about 2% of every other peer's frames. And V is a per-receiver measure, as on the 802.11 arm: a record reaching *every* peer in one broadcast has probability about $(1-p)^{N-1}$, which is not what V measures and would need gossip or relay.

10.4.2 The result, and how it must be quoted

Simulating the frame at DR5 under the 0.95 threshold gives $N_{\max} = 3$, with a sharp cliff: delivery is 0.960 at N=3 and 0.917 at N=5. There is no backoff to absorb contention.

Remark (Never quote this bare). The bootstrap interval is $[2, 3]$ — a knife edge — and at the certified $N = 3$, **nine of thirty seeds fail the very criterion being certified**. Under a per-realisation reading, where at least 95% of runs must individually meet V, the answer is **1**, not 3.

Both are computed and emitted. The decision to headline the mean is recorded as a decision, and it was safe to take only because the comparative result survives either

reading.

An earlier value of $N_{\max} = 5$ came from a three-seed sample against a bimodal distribution and is retracted; the artifact that produced it is retained, renamed and superseded.

The capacity runs sent a 218 B frame of six records on the 222 B payload table, which is the table the simulator enforces. The corrected frames on that table are 214.0 B (lean, six records) and 221.7 B (first format, four records), within 2% of it in size and airtime, so the runs were not repeated.

10.4.3 Why DR5 and not DR6

Table 10.1 admits DR4, DR5 and DR6 equally, and DR6 (SF7 at 250 kHz) runs at twice DR5's bit rate, so the choice needs stating. *Capacity does not change.* At a fixed duty cycle d the transmissions falling inside one $2T_{\text{air}}$ vulnerability window number $N \cdot 2T_{\text{air}}/T_{\text{period}} = 2Nd$: **the airtime cancels**, so $P(\text{deliver}) = e^{-2(N-1)d}$ is rate-independent and $N_{\max}$ is the same at either rate. That closed form reproduces the measured DR5 run to within 0.9 percentage points across N=2–10 and returns $N_{\max} = 3$, matching the simulation. DR6 therefore doubles the aggregate at the same three nodes, and pays for it in link budget: -120 against -123 dBm is 3 dB, or 1.41× less range under free-space spreading.

This is derived and not simulated, and the reason is a limitation worth recording. The module's sensitivity vectors are indexed by *spreading factor alone* — there is no bandwidth dimension in `EndDeviceLoraPhy::SENSITIVITY` or its gateway counterpart, and none in the interference helper. Forcing SF7 onto a 250 kHz channel would have returned DR6's doubled bit rate while silently keeping DR5's noise floor, so the 3 dB penalty — DR6's entire cost — would have been invisible in a run that completed cleanly and repeated across all 30 seeds.

10.4.4 What the configuration models, and what it does not

$N_{\max} = 3$ is far below the node counts usually quoted for LoRaWAN, and the difference is configuration. The simulation uses the LoRa physical layer on its harshest MAC provisioning: a single logical channel, a single demodulation path and one forced spreading factor, since the data rate is a design variable here and each run fixes it. The interference matrix treats *any* same-spreading-factor overlap as fatal, so there is no capture, where a real receiver recovers the stronger of two colliding frames. Enabling a 6 dB capture threshold raises delivery by 2.7 points at N=8 and by 1.29× at N=50 without moving $N_{\max}$. The small margin over the Poisson curve e^{-2G} at low load is traffic regularity, not capture: sources are periodic, for which the escape probability is $(1 - 2\tau/T)^{N-1} = 0.868$ at N=8 against 0.871 measured.

10.4.5 The result is range-limited, and that limit binds first

The channel model attributes all loss to collisions. Enabling correlated shadowing changes the outcome on no row at either 500 or 1000 m, and that is a property of the link budget: with 14 dBm transmit power and −130 dBm SF7 gateway sensitivity, log-distance path loss leaves 28.8 dB of margin at 500 m and 17.5 dB at 1000 m, which a shadowing standard deviation of about 8 dB cannot erode. *The propagation parameters describe a considerably better link than a real drone-to-drone channel.* Field measurements of air-to-air LoRa on real drones [51] show a link losing frames to range alone: delivery falls from 0.9711 at 200 m to 0.9550 at 500 m and 0.9045 at 1000 m, measured with two drones and a base station, where contention is negligible. The two loss mechanisms are independent, so delivery is $P_{\text{link}}(d) \times P_{\text{no-coll}}(N)$:

V target	200 m	500 m	1000 m
0.95	1	1	0
0.90	3	3	1
0.85	5	5	3
0.80	10	8	5

At $V \geq 0.95$ the regime admits no multi-node network at all. The link alone delivers 0.9711 at 200 m, leaving 0.0039 of headroom, while the least-loaded measured point (N=2) already spends 0.0283 on collisions. Relaxing to $V \geq 0.90$ restores $N_{\max} = 3$, and to 0.85 restores 5. The curve is reported instead of a point because the choice of threshold, not the physics, decides whether this regime is usable, and that choice belongs to the deployment. $N_{\max} = 3$ **is therefore a worst-case bound at short range — one channel, one demodulator, one spreading factor,** $d \leq 500$ **m — and not a LoRaWAN network capacity.**

10.4.6 Mobility

The capacity runs keep the nodes still. Whether that matters was tested: twenty nodes sending the 218 B frame every 36.6 s within 1000 m, static and under three mobility models, thirty seeds each. The runs use the interference matrix that allows capture, because without capture position cannot enter a collision's outcome at all, and the senders' random streams are pinned so that installing a mobility model does not change what the senders draw.

	mean delivery	s.d. across runs	against static	mean displacement
static	0.7454	0.063	—	0 m
Gauss–Markov, 5 m/s	0.7429	0.066	−0.24 pp	965 m
Gauss–Markov, 20 m/s	0.7451	0.066	−0.03 pp	967 m
Random Waypoint, 20 m/s	0.7490	0.065	+0.36 pp	816 m

No arm differs from the static one by more than 0.06 of the standard deviation between runs ($|t| \leq 0.22$), although nodes move most of a kilometre. Without capture the four arms are identical run for run, as they must be. The reason is the link margin of the previous subsection: motion inside the simulated area never brings a link near the sensitivity threshold, so the only route left is the power ratio of two colliding frames, and over thirty seeds that averages out. The two mobility models are indistinguishable here, so the choice between them carries no weight in this result.

This says that mobility does not change *collision-limited* delivery in this configuration. It does not say mobility is harmless to a link: Doppler, antenna pattern and a node that leaves range are outside the model. And the comparison is only as good as its control: before the random streams were pinned it showed a five-point penalty for motion that was produced entirely by the simulator handing the senders different streams once a mobility model had been installed (class C4 of Chapter 11).

10.4.7 What infrastructure collection would buy

The complementary question — what if a ground gateway collected the ledger — is a configuration change: three channels and eight demodulation paths, with the same forced data rate. Delivery improves substantially and the improvement grows with load, reaching 1.90× at N=50 (0.3755 → 0.7134) and still 0.5071 at N=100. **Yet $N_{\max}$ moves only from 3 to 8** (95% interval $[5, 8]$; the bare value must not be quoted). The $V \geq 0.95$ criterion sits on a near-vertical part of the delivery curve — the gateway passes at N=8 with 0.9526 and fails at N=10 with 0.9398 — so a threefold reduction in offered load per channel moves the threshold barely at all.

10.4.8 External corroboration, quoted as a curve

Haxhibeqiri *et al.* [50] publish a capacity result in closed form — a quintic fitted at $R^2 = 0.997$ to total packet loss measured on SX1301 hardware — so it can be evaluated at this work's operating point instead of quoted at theirs. It was implemented, checked against the four loss figures their own text states, and given the same criterion.

Remark (Two cautions about that comparison). Their $N_{\max}$ is decided at $N = 4$ and $N = 5$, both inside the region their own fit is intercept-dominated: the polynomial predicts 1.78% loss at *zero* transmitters, roughly 40% of the entire loss budget the criterion allows.

Table 10.3: The low-rate capacity by two independent methods: frame loss at N transmitters.

N	this work (ns-3)	Haxhibeqiri *et al.*
2	2.8%	3.1%
3	4.0%	3.8%
5	8.3%	5.1%
8	12.9%	7.0%
30	43.0%	19.9%
50	62.5%	29.9%
$N_{\max}$ at $V \geq 0.95$	**3**	**4**

Forcing the fit through the origin would give them $N_{\max} = 5$, *widening* the gap — so quoting 4 is the conservative choice.

And the pessimism ratio is not a single number. It runs 0.91× at $N = 2$, where *this work is the more optimistic model*, through 1.07× at $N = 3$, to 2.1× from $N = 10$ upward. The sign changes in exactly the region where $N_{\max}$ is decided. An earlier finding in this project quoted a single ratio in the wrong direction and had to be retracted.

Remark (A correction of attribution). Until October 2026 this model was cited throughout the project — in this chapter, the paper, the code and eight documents — under the names of the authors of a *different* paper. The source had been read and its figures checked; nobody had compared the author list with the title page. Every bibliography entry is now compared with its registry record by a script (Chapter 11).

10.5 The same accounting on other broadcast links

The exclusion is an inequality about a payload and a frame, and nothing in it belongs to LoRa. It was applied in October 2026 to three broadcast systems that aircraft and vehicles already use, each with its payload and message rate taken from its standard, and with this thesis's lean frame as the record (Table 10.4). ADS-B 1090ES leaves 56 bits of each message for data and Bluetooth 4 Remote ID 25 B: neither can carry a 64 B signature at all, the case of the five signature-bound LoRaWAN rates. Bluetooth 5 and Wi-Fi Remote ID fit up to seven records in a frame, but at one message a second a record waits a whole message period under the convention of Section 4.10, so no frame meets any deadline up to 1 s. V2X CAM at 10 Hz admits no frame at 100 ms, one record at 200 ms and two at 300 ms; CAM's standard bounds how often a message is generated, not when it must arrive, so the deadlines are swept, not quoted. This is consistent with these standards signing each message; it does not explain why they do.

Table 10.4: Records per frame each broadcast system admits with one signature per frame, by deadline, under the freshness bound; —: no signature fits. The last row is this thesis's design, by the same rule.

	payload	rate	$D_{\max}$			
system	(B)	(/s)	100 ms	200 ms	300 ms	1 s
ADS-B 1090ES	7	2	—	—	—	—
Remote ID, Bluetooth 4	25	1	—	—	—	—
Remote ID, Bluetooth 5 / Wi-Fi	225	1	none	none	none	none
V2X CAM, 10 Hz	1500	10	none	1	2	9
V2X CAM, 1 Hz	1500	1	none	none	none	none
this design, 802.11a	1500	50	4	9	14	49

10.6 What the regime is for

Three nodes at about 0.17 records per second each are 0.5 records per second in all, where the 802.11 design carries some 6 200. The low-rate link is not a slow 802.11. Its defensible role for a ledger is long-range *provenance* — a tamper-evident trickle — and not live telemetry. An independent ns-3 comparison of the two technologies reaches the same structural conclusion from a different direction: Klimiashvili *et al.* find neither uniformly better and recommend selecting between them instead of tuning one [59].

10.7 Scope of this chapter

This is simulation and analysis, not measurement. No radio was metered, and there is no energy column. Airtime per record is reported instead: it is what the regulator rations, and it is computed, not assumed.

Chapter 11

Reproducibility, Defects and Research Integrity

11.1 Why this chapter exists

Most theses report what was found. This one also reports *what was found to be wrong, and how.* The justification is not confession but transferability: the defects catalogued below are not peculiar to this work, and several of them survived safeguards that are widely considered sufficient — version control, a green test suite, seeded runs, committed raw data.

By August 2026 ten results in this project had moved after they were first recorded. Only four were catchable by running more seeds. Two were *protected by passing tests.* Three were contradictions between a paper and the artifact that generated it.

In October an external review of the paper found five more, and they were of a kind the first ten had not prepared the project for (Section 11.3). Every one of them had survived two full audits and a green suite of thirteen hundred tests, and one of them was a "correction" the second audit had itself introduced.

11.2 The defect taxonomy

class	mechanism	instance in this work
C1	small-sample mean lands the wrong side of a threshold	four headline numbers; a capacity value $5 \to 3$
C2	an unverified constant on the measurement path	the 2.4 GHz basic-rate saturation; the frame header treated as a constant
C3	a threshold applied to a mean rather than a distribution	the capacity criterion; the utilisation crossing
C4	a configuration change that perturbs the random realisation	simulator stream displacement under mobility
C5	a claim wider than the experiment supports	two-node hardware described as validating contention; a load ceiling measured at one N applied at all
C6	a correction made once, at its source, that does not reach the copies	five constants derived from a re-measured header; a retracted finding still printed in a table
C7	a composite that was never built: a result summed from parts measured separately	the headline design; its verifiability; the exclusion test; the low-rate batch
C8	a simulated source more regular than any real one	phase-locked senders, on both arms
C9	a source taken on trust	a paper cited for two months under another paper's authors; figures quoted from a paper that was not held

Remark (The load-bearing observation). **More seeds fixes only C1.** C2–C9 are immune to sample size. "We ran thirty seeds" is therefore not a statement of correctness, and the discipline installed after the C1 failures gave no protection at all against the C2 failures found later — nor did the discipline installed after those protect against C7.

11.3 October 2026: five errors that survived two audits

The design had never been built. The configuration the results headlined — delta records, one signature per four — was a *sum of sizes*: a record size from one module, a header measured on frames of a different encoding from another, and a signature

length. No frame carrying delta records had been encoded, sent or decoded, and the receiver that would decode one did not exist. Each number was individually correct, so re-deriving any one of them reproduced it.

Its verifiability was wrong. The byte model amortised one keyframe over four frames; the verifiability model treated the four as independent. At its own loss budget the design verified 0.881 of its records, not 0.95 (Theorem 4.6).

The exclusion test left out two things a frame must carry — the chain link and a record that decodes alone — and counted seven data rates where the standard defines twelve. The August audit had "corrected" the exclusion from four rates to three on the strength of a leaner header; with the link charged, that correction is itself wrong (Section 10.2.5).

A record size was used at the wrong time scale. The delta record measured with records 50 ms apart was used on the low-rate arm, where they are seconds apart and the same record is half as large again (Section 10.3).

A source was cited under the wrong authors, in the paper, the thesis, the code and eight documents, for two months. Each citation had been "verified": the DOI resolved and the claim was in the PDF. Nobody had compared the author list with the title page — and the paper carried a paragraph asserting that every citation had been checked.

The first four are one error seen four times. What fixed them was not a better audit but an object: the frame format was implemented, with a sender and a receiver, and the byte, loss and exclusion results were recomputed *from one definition of a frame*, whose arithmetic a test holds equal to the frames the code emits. The fifth was fixed by a script that compares every bibliography entry with its registry record, and by deleting the paragraph: a claim of that kind belongs in a check that runs, not in prose.

11.3.1 The capacity ceiling, and a prediction that failed

The same review asked for confidence intervals on the 802.11 capacities. Producing them meant simulating each configuration directly instead of applying one load ceiling, and the prediction that the two would agree within ±10% — committed before the runs — failed (Chapter 9). The ratios this thesis had described as "protected by construction" moved with it.

Chasing the per-run spread of those simulations produced the 802.11 counterpart of an artifact already known on the low-rate arm (below): strictly periodic senders keep their relative phases for a whole run, so a run is one phase configuration, and in an unlucky one a few nodes lose every frame while the rest lose none. Three follow-up experiments were registered before they were run. Two of their predictions failed and are reported as

failures; the third, that the choice of source moves the mean by less than a tolerance set in advance, held; with all six cells run, the difference was small and of one sign, and it did not replicate on fresh seeds (Section 8.5).

11.4 A simulator artifact large enough to change what a reader should believe

The ns-3 LoRaWAN module's periodic sender fires on an exact interval, and LoRaWAN ALOHA has no backoff, so relative transmission phases are frozen for an entire run: a pair that collides once collides on every cycle. Delivery becomes *bimodal* instead of noisy about a mean. Over 30 seeds at N=5, 22 runs deliver exactly 1.000 and the rest fall to 0.20–0.84. The coefficient of variation is inflated **2–8**× relative to a sender with one-second transmission jitter, the inflation growing with node count (2.9× at N=5, 7.9× at N=100). The LoRaWAN specification makes it a rule that an end-device "pseudo-randomly changes its transmit periodicity to prevent systematic synchronization of populations of end-device transmissions" [54]; the module's sender does not.

The measurement is claimed, not the observation. Durand and Booysen report the same effect in their own results, attributing bimodal delivery to nodes that "always transmit on a specific SF, time, and channel", producing "certain packet collisions being repeated for every transmission cycle" [70]. What is added here is the quantification.

On 802.11 carrier sense softens the effect and does not remove it. The simulator declares the medium busy only at the end of a 4 µs preamble-detection period; two strictly periodic nodes whose phases fall inside that window collide in every period. At $N = 29$ the per-run standard deviation of delivery is 0.034 with strictly periodic sources and 0.0016 when phases are redrawn every period. The means are the same: a difference of 0.003 seen in the first thirty seeds did not replicate on fresh ones (0.0004 ± 0.0009).

11.4.1 A claim that was withdrawn

It had been intended to report that this dispersion is undetectable in the ns-3 LoRa literature because that literature does not state replication information, and a survey was pre-registered to test it — inclusion criteria, keyword set and a 25% falsification threshold, all committed before the corpus was assembled. At the achieved n=23 the result is 21.7% reporting with a 95% interval of $[7.5, 43.7]$%. **That interval contains the threshold, so the survey cannot answer its own question, and no claim about the literature is made.** The protocol, corpus and adjudications are in the repository as a null result.

11.5 An automated staleness gate, and what it does not cover

Every model-derived artifact is re-derived from current code on every run and compared row by row; a single differing value fails the build. This makes silent staleness of *derived* data impossible.

It does not make staleness impossible. Two limits were found the hard way:

- **Scope.** Every drift guard originally parsed the paper. The specification documents were outside that scope, so corrections propagated into the paper and not into the documents for three weeks — including a table built verbatim on data the project had already purged and superseded.
- **Direction.** Retracting a finding in a register does not remove it from the prose. One retracted claim was found alive in the paper months later, one hundred lines below a sentence stating the opposite, and later again in the open-items register. **When you retract, grep the whole repository.**

11.6 Practices that earned their place

Pre-registration, committed data-free. Predictions are committed as their own commit before data exist, so the ordering is third-party checkable. This cost the project a claim once: an assertion of pre-registration was *withdrawn* rather than reconstructed, because writing an expectations file after results are known manufactures evidence.

Stating the expected value before measuring. This is what caught the 2.4 GHz basic-rate artifact, which repeatability would never have caught.

Retaining a discarded model as a labelled failure curve. The naive broadcast reduction remains in the codebase, clearly marked, so the 16× error it produces can be shown rather than described.

Keeping per-seed raw data, not summaries. This allowed validation bands to be re-derived at thirty seeds without re-simulating.

An independent second implementation. The slot-exact simulator, written before the published model was found, shares none of its assumptions and is what turns agreement into evidence.

No typed results. Since October 2026 every result in the paper, and every corrected result in this thesis, is a macro in a file that a script writes from the artifacts. A sentence that states a verdict instead of a number — “eight of twelve”, “does not

meet the target" — is held by the same script, which refuses to run if the artifact no longer says so.

A check on the bibliography that runs. Every entry is compared with its Crossref, DataCite or arXiv record — family names in order, initials, title, year — and entries with no registry record must name the held document they were checked against.

11.7 Two defects worth stating individually

11.7.1 A test that asserted a bug

A utilisation function returned exactly zero for a single node, with a unit test asserting that a lone sender never contends. The function conflated *contention* with *utilisation*: a lone sender does not collide but still occupies airtime, and returning zero declared a single node able to carry unbounded traffic. The defect was harmless until another result began reporting single-node cases — **and it was protected by a passing test that had encoded the wrong behaviour as intended behaviour**.

11.7.2 A criterion that could not fail

The project pre-registered a success criterion requiring at least a 40% reduction in authentication bytes, and committed it to version control two days before the result existed. The ordering is genuine and verifiable.

The quantity it tests, however, reduces exactly to $1 - 1/b$. The threshold is therefore $b \geq 2$ — independent of encoding, scheme, placement, header size and signature size. The companion condition on verifiability is satisfied by construction with zero margin. **No configuration that batches at all could have failed the test.**

The pre-registration is real and worth keeping; what had to go was the implication that it constituted a risk the design might not have survived.

11.8 What a reader should take from this

Three times before October 2026 a claim in this project survived because two facts were each individually recorded and never placed side by side, and this chapter used to end: a register that stores facts separately does not compose them; *only re-derivation does.*

That was not enough. The errors of Section 11.3 were re-derived, repeatedly, and reproduced every time, because each part was right. **What composes facts is building the thing they describe and making it run.** A design that exists only as a sum has no receiver to refuse a frame it cannot decode, and no encoder to refuse a frame that does not fit.

11.9 How citations are checked

No figure or number is quoted from a source that has not been read in full, and where a source establishes prior art for something this thesis might otherwise have claimed as novel, that is said explicitly. Every bibliography entry is compared by a script with its Crossref, DataCite or arXiv record — family names in order, initials, title and year — and an entry with no registry record must name the held document it was checked against.

Those rules exist because each was broken once. An early finding claimed this work was the *more optimistic* of two LoRa capacity models; it was drawn from the wrong figure of a paper that reported pure-ALOHA results beside LoRa results, and was retracted: **quoting the paper is not enough — quote the figure**. And until October 2026 the background chapter said that every reference had been confirmed against its Crossref record, while four entries carried wrong metadata and one paper was cited under the authors of a different one. The confirmation had checked that each DOI resolved; it had not compared the author list. **A claim that citations were checked belongs in a check that runs**, which is why the sentence is gone and the script exists.

11.10 Where this sits

Credibility in network simulation has been studied mainly as a problem of *reporting*. Kurkowski, Camp and Colagrosso read the 151 papers of six years of one conference on mobile ad hoc networks against four criteria — repeatable, unbiased, rigorous, statistically sound. Of the 114 that used simulation, none said its code was available, three in ten did not name the simulator, and 87.5% of those with plots showed no confidence intervals; fewer than 15%, they concluded, could be repeated [10]. Cavalcanti *et al.* found the vehicular literature of the following decade little better [11]. The prescription drawn from such findings is familiar: publish the code, state the seeds, report intervals.

This project followed that prescription from its first commit — code and raw data public, seeds in every result file, an interval on every simulated quantity — and the record of this chapter is what the prescription did not prevent. Of the first ten results that moved, four were of the kind that more seeds catch. The rest, and all five found in October, were invisible to any rule about what to report, because everything reported was right: a constant nobody had checked, a threshold applied to the wrong statistic, a correction that stopped at its source, a design that existed only as a sum. One of the classes had been seen before by others: Durand and Booysen describe in their own results the frozen collisions of class C8 [70].

What this chapter adds to that conversation is three practices that act on something other than the report.

Test the text. The unit that went wrong most often was not a result file but the bound-

ary between a result file and a sentence. Numbers in the documents are therefore written by a script, and a sentence that states a verdict is held to the data by a test (Section 5.7).

Write the prediction first, and keep the failures. An interval says how well a quantity is known. A prediction committed before the data says something an interval cannot: it can be wrong. Of the twenty-two registrations in Appendix B, eight failed in whole or in part, and each failure changed what this thesis reports.

Build the composite. A result that is a sum of parts measured separately should be doubted until the object it describes has been built and run. No audit of the parts finds this class, because the parts are correct.

Two cautions bound the claim. This is a self-audit of one project: the classes are observed, their frequencies in the literature are not, and the one attempt made here to measure a frequency could not answer its question (Section 11.4.1). And a taxonomy of the errors that were found says nothing about those that were not.

Chapter 12

Conclusions, Limitations and Future Work

12.1 What this thesis establishes

1. After compression, authentication — not data — is the dominant on-air cost of a tamper-evident UAV telemetry ledger. At the smallest encoding studied, the signature is larger than the record it signs.

2. Frame-level self-batching is the correct placement, and it Pareto-dominates block-level aggregation under *any* stationary loss process, not merely under the independent-loss assumption the emulator implements.

3. On IEEE 802.11 the freshness deadline binds before the frame size does, so the amplification law that motivates aggressive compression is never operative: compression pays exactly 1×. On low-rate links the frame size binds and the leverage is real. **The regimes are different in kind.**

4. A delta-coded frame that depends on its predecessor does not meet a verifiability target equal to the loss rate. Every frame of the design therefore carries a self-contained record, and the cost of that is small.

5. Eight of the twelve EU863-870 data rates cannot carry one signed, hash-chained frame: five because a 64 B signature does not fit a 51 B payload, three because header, chain link and signature fill a 115 B payload. The result holds under four stated conditions.

6. Built as one frame format and simulated directly, the design carries 43.25 B per record where per-record signing carries 145.77 B (70.3% fewer), and sustains 124 nodes at 95% delivery where per-record signing sustains 35, at an operating point compliant with 3GPP TS 22.125.

7. Of the four design axes, placement and batching interact exactly, by $g_a(1 - 1/b)$, and in the frame as built delta coding is conditional on batching as well: both gains are zero at $b = 1$. The scheme axis is byte-neutral. The four-axis co-design claim, as originally stated, was overstated; what is true is that batching is the axis the other gains depend on.

8. The gain does not rest on the conditions it was first computed under. Spread over a 1 km disc with hidden stations, moving at up to 15 m/s, carried by six 802.11 PHYs and rates, or signed with Falcon-512, the design serves 3.39–3.80 times the nodes of per-record signing (Chapter 8). What changes is the batch a deadline admits, which the design map gives for any record rate and deadline.

9. The protocol's properties are proven in Tamarin, in three models, and two defences close two of the gaps the first threat model left: a window on GNSS time refuses frames an attacker holds back, and an 8 B group tag stops floods from outside the swarm. A flood from a member is not stopped, and is stated (Chapters 3 and 8).

10. The prototype's two anomalies were the interpreter's. Compiled, one receiver core serves 289 nodes, so the channel binds again, and the design's sender spends 17.1 µJ per record, 3.4 times less than the first format.

12.2 The research questions, answered

RQ1 — how large is the overhead, and where does it dominate? The authentication fraction is $g_a/(s+g_a)$ and passes one half exactly when the record is no larger than the signature (Theorem 4.1). With 64 B signatures that is where compact encodings put a telemetry record: 49% for CBOR and 59% for delta coding, and 96 of 109 B once the chain link inside the record is counted as authentication material (Chapter 7).

RQ2 — which placement? One signature over the records of one frame. It verifies with probability $1 - p$ whatever the batch, where a signed unit spread over n frames verifies with $(1 - p)^n$ and is infeasible beyond one frame once $\varepsilon \leq p$ (Theorems 4.4 and 4.5); and the frame must also decode alone (Theorem 4.6). Self-batching needs one ordinary signature. Cross-signer aggregation is needed only for relayed records, which this thesis models and does not evaluate.

RQ3 — when does an aggregatable scheme win? Not in any of the eighty cells evaluated at 6 Mb/s. For a node's own records BLS costs bytes; for relayed ones it pays only where the radio draws more than thirty times the power of the CPU, against a measured ratio of 0.29, and one core cannot verify it above 200 records per second (Chapter 8). Its regime is links on which airtime is orders of magnitude dearer than computation.

RQ4 — the end-to-end configuration, and do the models survive? Delta coding, Ed25519, one signature and one chain link per frame, four records, a self-contained record at the head of every frame: 43.25 B per record against 145.77 B, and 124 nodes against 35. The saturation model of the broadcast channel survives simulation, within 0.51%. The single utilisation ceiling built on it did not, and was replaced by direct simulation; a model of the access rule alone then reproduced the simulated capacities within 1.2% and predicted six unseen ones within 1.7%. On two radios the access rule was observed within 6% where contention can be isolated; a registered prediction of their loss below saturation held at two loads and failed at the third; no capacity was measured (Chapters 8 and 9).

RQ5 — where can the frame not be sent at all? On eight of the twelve EU863-870 LoRaWAN data rates, under five conditions, the fifth being this frame format for three of them; a 48 B signature, a 13 B tag or a chain link kept off the air each recover some of them, and Table 10.2 says which (Chapter 10).

12.3 Limitations

12.3.1 Modelling scope

The capacities are computed in one collision domain of static nodes. A separate scenario spread the nodes over discs of up to 1 km, with capture, Nakagami fading and hidden stations, and moved them on independent paths at up to 15 m/s (Section 8.5.5): the gain held, and what a capacity counts changed. That scenario has one propagation model, no Doppler and no time-correlated fading. On the low-rate arm mobility moves collision-limited delivery by nothing measurable (Section 10.4.6); that is not a statement that mobility is harmless to a link. Loss in the verifiability experiments is generated at the frame level — independent, bursty, or growing with frame length — and collision loss was traced in the simulator; no loss was measured on air. Falcon-512 was timed and simulated in the frame; signatures that do not fit one frame remain projections.

12.3.2 Validation scope

On hardware, two radios gave the loss of a link, the access rule between two saturated stations, and a failed prediction of the loss between two busy ones (Section 9.4); a figure for airtime that earlier versions carried is withdrawn there. No capacity was measured, and the capacities of Chapter 8 rest on simulation, of one collision domain in which every station hears every other. Unequal received power and capture were simulated and raise them by about an eighth (Section 8.5.4); hidden stations and motion were simulated separately (Section 8.5.5). Record sizes come from a synthetic generator, checked against twelve public flight logs at 0.2 s spacing and above and, at the 20 ms of the operating

point, against flights of an autopilot in simulation, whose navigation-quality fields do not vary (Table 7.4); no flight log recorded at 50 Hz was found. Only the sender's energy is metered, with a sensor never checked against a reference load: interpreted, the lean sender costs 2.7 times the first format's energy per record; compiled, 3.4 times less. The first format's composed energy is a lower bound by roughly 7–14%, the residual being uncharged prototype framing overhead. The receiver-CPU ceilings charge cryptography only; the interpreted decoder costs several times more and the compiled one does not, timed without a radio or a dispatcher in the path (Chapter 8). Capacity was simulated on six 802.11 PHYs and rates and on no other family of link, and for batches other than one record and four only as a test of the capacity rule (Table 8.6): nothing else was evaluated at them.

12.3.3 The low-rate arm

Analysis and simulation only: no radio metered, no energy column, one spreading factor per run. The capacity figure is a within-one-spreading-factor bound, not a network capacity, and its confidence interval is a knife edge.

12.3.4 The boundary itself

The exclusion is conditional: on LoRaWAN's regional payload limits, on a public-key signature in every frame, on no loss recovery, on a verifiability target no looser than the loss rate, and, for three of its eight data rates, on the frame format of this thesis. It was also stated wrongly twice before reaching its present form (Section 10.2.5), and narrowed a third time when a review of 2026-10-09 separated what belongs to the problem from what belongs to the format; a reader is entitled to weigh that. What has not moved in any version is the part that rests on one inequality: $64 > 51$.

12.3.5 Security scope

Confidentiality, availability and key management are outside the design (Section 3.4). A member of the swarm holding the group key can still flood, and a delayed frame is refused only while GNSS time is held. The properties are proven in three models, not one, with canonical encoding assumed injective (Section 3.4.3). With one chain link per frame, continuity inside a frame rests on the frame signature alone, and showing one record to a third party means showing its frame. The receiver keeps every accepted frame for that purpose, which it did not do until a review of 2026-10-09 (Section 3.4).

12.4 Future work

12.4.1 Small

Pricing implicit certificates. The certificate accounting charges an explicit certificate periodically. Implicit (ECQV) certificates are the standard smaller alternative and were never modelled. The current charge is an upper bound, which cuts against this work's own comparison — so the gap is safe but should be closed.

Telemetry at the operating rate. Public flight logs carry position at 5 Hz, and the delta record at 20 ms spacing rests on the generator and on simulated flights of one autopilot. A public log recorded at 50 Hz on a real vehicle, if one appears, would settle it.

12.4.2 Substantial

An analytical channel-access delay. The constraint that *sets* the batch counts fill time and airtime. Table 8.9 adds the channel-access delay as simulated at the capacity point — under a millisecond on average and 3.49 ms at worst, against a fill time of 80 ms — so the batch is not affected; but the figure exists only where a simulation was run. An access-delay term from the model of Section 8.5.3 would let the constraint be evaluated at any load.

Multi-spreading-factor capacity on the low-rate arm. Each run fixes one data rate, forfeiting the quasi-orthogonality that supplies much of a real deployment's capacity. That answers a different question — deployment capacity rather than per-configuration feasibility — and it is the question an operator would ask next.

The 5G NR sidelink. Only 802.11 was simulated. The direct link between devices of the cellular standard has a medium access of its own, which the access-rule model does not describe; whether the frame count dominates there as it does here needs its own simulation.

Post-quantum signatures that do not fit one frame. Falcon-512 fits a frame and the gain holds with it (Section 8.5.7). An ML-DSA signature does not fit a frame at any batch, and the deadline caps the batch, so it cannot be amortised either; it needs a variant of the design built around fragmentation.

12.4.3 Speculative

The exclusion bound is link-agnostic, and three broadcast systems already in use have been put through it (Section 10.5). Instantiating it on other constrained bands — and on the signature sizes the post-quantum standards actually mandate — would say which of

tomorrow's radio and cryptographic combinations are feasible before either is deployed. That is the natural generalisation of this work.

12.5 Closing

This thesis introduces no new cryptographic primitive and makes no claim to one. Its contribution is a reproducible co-design of existing, standardised components; the quantified trade-offs that co-design entails; the boundary beyond which no batch size helps, with its conditions; and an account — unusually explicit for a thesis — of the errors made in establishing that boundary and the classes of defect that survived the safeguards intended to prevent them.

The most durable result here is also the smallest one that survived scrutiny: on five EU863-870 data rates, a 64-byte signature does not fit a 51-byte payload, and no batch size, encoding or placement changes that. A shorter authenticator does, and Chapter 10 says what it buys.

Appendix A

Reproducing the Results

Everything this thesis reports can be regenerated from the repository `https://github.com/besco1998/fanet-authbc`. This appendix says what is needed for each part and what each command produces. The design rule behind it is stated in Chapter 5: work that depends on a machine — timings, simulations, radio and energy measurements — is run once and its raw output committed; everything derived from that output is recomputed and compared on every run.

A.1 Four paths, by cost

path	reproduces	needs	time
A, analytical	every byte, loss, exclusion, energy-model and optimiser result; every table and figure, from the committed raw data	Python 3.12 or later, about 1 GB of disk	minutes
B, 802.11 simulation	the channel-model validation and the capacity search	ns-3.48 built from source, about 8 GB of disk	hours
C, low-rate simulation	the LoRaWAN capacity and mobility runs	the ns-3 LoRaWAN module	tens of minutes
D, hardware	primitive timings, powers, sender energy, two-radio loss, frame spacing and contention	two Raspberry Pi 4, an INA219 current sensor, a host to read it	half a day

Path A alone reproduces every number in the results chapters, because the outputs of paths B to D are committed and the analytical layer reads them.

A.2 Path A

```
git clone https://github.com/besco1998/fanet-authbc.git
cd fanet-authbc
make setup     # a virtual environment with the pinned dependencies
make all       # lint, type check, the test suite, the reproduction gate
make paper thesis
```

`make all` ends with the reproduction gate: every derived result file is recomputed from the current code and the committed raw data, and a single differing row fails the run. `make paper` and `make thesis` first run `analysis/paper_numbers.py`, which writes every number the two documents print into `numbers.tex` and stops if a result file no longer supports a sentence that states a verdict. A number is changed by re-running its experiment, never by editing the text.

A.3 What is where

`src/authbc/`	the library: encodings, signature schemes, ledger, the two frame formats with sender and receiver, the analytical models, the two simulators written for this work
`analysis/`	one script per group of results; `paper_numbers.py`; figure scripts
`ns3/`	simulation scenarios (C++) and the drivers that run them over seeds
`hw/`	what runs on the boards: timing, the energy harness, the capture and reduction of current samples, the two-radio link test
`experiments/`	configurations and run plans, including the lists of seeds
`results/raw/`, `results/hw/`	raw output, one file per experiment, each with a header that records the environment and a hash of the configuration
`results/figures/`	figures, regenerated from the raw output
`tests/`	unit tests, published test vectors, the reproduction gate, and tests that hold the text of the paper and of this thesis to the result files
`docs/`	specifications, the findings register, the expectation files of Appendix B, the logbook

A.4 Paths B to D

The simulation scenarios are single source files copied into the simulator's `scratch` directory; each driver names the scenario it runs, the seeds, and the file it writes. A run is identified by its configuration and seed, and a stored run can be repeated bit for bit, which is how a rebuilt simulator is checked before new runs are trusted. The hardware procedures — wiring, sampling, the order of a metered run and what makes a repetition unusable — are in `hw/`, written to be followed without the author.

A.5 Software

Python 3.12; `cbor2` pinned at 5.8.0, because a later release changed canonical output; `cryptography` for Ed25519 and ECDSA and `blspy` for BLS12-381, each checked against published test vectors before any timing; ns-3.48 with the `signetlabdei/lorawan` module; PX4 v1.17.0 for the simulated flight of Chapter 7. The measured platform is a Raspberry Pi 4 (Cortex-A72) under a 64-bit Linux with the `performance` governor.

Appendix B

Predictions Registered Before Their Data

Where a run tests a prediction, the prediction was committed to the repository before the data existed, as a commit of its own. The table lists every such registration, including one whose experiment has not yet been run: the commit that holds it, what it said, and what happened. Commits are those of the public repository history; each registration is an ancestor of the commit that holds its data, which a reader can check with `git log`.

commit, date	what was registered	outcome
`3354ec1` 3 Jul	success criterion: at least 40% fewer authentication bytes than the compression-only baseline at $V \geq 0.95$	met (75%) — and it could not have failed: the quantity is $1 - 1/b$ (Chapter 11)
`eb3eda5` 7 Aug	survey protocol: fewer than 25% of ns-3 LoRa papers report replication information	21.7%, interval $[7.5, 43.7]$% at $n = 23$: contains the threshold; no claim made
`ca8fd1c` 7 Aug	at DR6: six records per frame, half the airtime, $N_{\max}$ unchanged at three	batch and airtime confirmed; $N_{\max}$ derived, not simulated — the simulator has no bandwidth dimension — and not claimed as confirmed
`84adb92` 28 Aug	the utilisation crossing at a 174 B frame lies in 2.1–2.8; it drifts higher than at 288 B	2.367: held. Direction: lower, at 0.45σ — the run had no power to resolve a sign (Chapter 9)
`6f82599` 6 Oct	directly simulated $N_{\max}$ within ±10% of the single ceiling, six configurations	**failed**: −16.7% and +11.7% in two of six
`47ebbbb` 6 Oct	freezing the senders' phases moves the mean by less than 0.005; a jitter of 1 ms halves the spread between runs	mean: held. Spread: halved at two test points of four — **failed**

commit, date	what was registered	outcome
`47ebbbb` 6 Oct	seven configurations not yet simulated lie within ±6% of the airtime line (slope fixed in `fad28e0`, before any of the seven was run)	held: all within 3.4%
`fbcb646` 6 Oct	sweeping every pair of nodes through all phases gives the same mean and a spread under 0.004	mean: held. Spread 0.003–0.006 — **failed**
`e4e3cdf` 6 Oct	on public flight logs chosen by a fixed rule: mean delta record at most 13 B and saving at least 68% at 0.1 s; keyframe 22–25 B	the logs carry 5 Hz: applied at 0.2 s, held (11.1 B, 69.1%); untested at 0.1 s; keyframe 21.98 B, under by 0.02 B
`f72db7f` 7 Oct	five one-record frames of the stream-signing schemes within ±6% of the line	held: 3.5–4.2% above it
`a65ad6c` 8 Oct	on unused seeds the difference between the two traffic sources does not replicate ($z < 2$)	held: 0.0004 ± 0.0009. A second claim registered with it — that the crossing rule reads low on a noisy curve — was **refuted**
`dbe4ee9` 8 Oct	one simulated PX4 flight: five ranges for record sizes; mean and standard deviation of the 50 Hz stream's period	four ranges held; the 0.2 s range missed by 0.04 B; mean period held; standard deviation **not scored** (the host's clock was stepped)
`9e9c096` 8 Oct	six crossings in conditions never simulated — contention window doubled; 2% and 10% loss — within ±3% of the access-rule model	held: all six, worst 1.7%
`b69e08e` 8 Oct	on the Raspberry Pi 4, per four-record frame: lean sender 1.0–2.5 ms, lean receiver 2–5 ms; Ed25519 in a batch of 64 at 0.45–0.60 of the cost of one at a time	both times **failed**, on the fast side (0.87 and 1.43 ms): the desktop figure they were scaled from was three to four times too high. The batch held (0.46)
`456a4e7` 9 Oct	contention on real radios: loss among two to five boards at three loads; with two boards, within 0.6–1.4 times the access-rule model's	run with two boards: held at two loads, **failed** at the third (0.46 times the model's loss). Three to five boards: not run (Section 9.4.3)
`06f1bfa` 9 Oct	a vendor mode of the radio explains that miss: with frame burst off, one sender takes 2.070–2.095 ms per frame, two boards fall inside the bands and lose at least 1.5 times what they lose with it on, and a repeat with it on agrees with the first run	all four **failed**, two of them at the highest load only: 2.050 ms per frame; the highest load still under its band; the repeat gave 0.86% where the first run gave 0.64. No effect of frame burst on the loss is shown

commit, date	what was registered	outcome
`9a85afa` 9 Oct	two saturated radios lose 11.8% of the frames they put on air, band 7.1–16.5%; the author's expectation, recorded with it, was about 7%	held: 12.4% with frame burst off, 12.1% with it on. The expectation was wrong
`a38994f` 9 Oct	four energy runs on the second board and sensor, in µJ per record: a control repeating July's baseline row at 113–125; the lean sender within 10% of its timed frame at the metered power, with four records and with one; the JSON row at 115–127	held, all four: 116.2; 155.4 and 273.1; 120.9 (Table 8.10)
`64322fc` 9 Oct	the lean design with two, three and eight records to a frame: each crossing within 6% of the airtime line, slope unchanged, and within 3% of the access-rule model	held, all six: 66.7, 96.5 and 222.3 nodes, at most 1.6% from the line and 0.8% from the model (Table 8.6)
`61a4b4f` 9 Oct	the receiver, once it keeps every frame, timed on the board against a control of the receiver as it was: the control within 2% of the morning's figure, the new receiver between 1.00 and 1.02 times the control	held: 0.7% more, 10 µs on a four-record frame
`eb2751b` 9 Oct	the receiver on one to four cores of the board at once: one core 680–708 frames/s; two, three and four cores 1.90–2.00, 2.75–3.00 and 3.50–4.00 times one; two cores below 124 nodes and three at or above	one and two cores held (692 frames/s; 1.90); three and four **failed**, low (2.70 and 3.36); the line on nodes held (106 and 150)
`db0bca1` 9 Oct	baseline and design with the nodes spread over a disc and capture at work: at 15 m within 5% of the access-rule model with the simulator's capture rule; at 100 m between the equal-power crossing and that model's band	held, all four: 39.6 and 140.4 at 15 m, 39.4 and 139.3 at 100 m, within 0.5% of the model (Table 8.7)

The experiments of October 2026

The experiments added in October 2026 test the objections a revision plan expected reviewers to raise. Each was registered as above, with the plan's expected outcomes quoted and finer predictions added in the same commit (in italics below); each registration is tagged in the repository as `prereg-` followed by its name. Table B.2 lists them in the

order of the chapters that report them.

Table B.2: Registrations of October 2026 and their outcomes.

commit, date	what was registered	outcome
`d4bf273` 19 Oct, `c7f05c7` 20 Oct	spread swarm: ratio 3–4 in every layout; capacity counted in neighbours falls at 1 km; motion moves V by under 0.01; *at* 1 *km,* 10–40 *neighbours at the crossing; hidden share rising with radius*	held, except *the neighbour count*, 8.7; the group's speed proved untestable in that scenario (a rigid translation)
`13c406e` 21 Oct	every node moving on its own at 0, 5 and 15 m/s: $\lvert\Delta V\rvert < 0.01$, ratio 3–4; *a square layout's control* 0–5% *above the disc's*	held, except *the square's baseline*, just below its band with overlapping intervals
`b2b6ae6` 19 Oct	design map: ratio non-decreasing in b^*; above 5 at 100/s and 100 ms; every simulated role within 5% of the model; mean peak age within $D_{\max}$ at $p = 0.05$, its 99th percentile above	all held (6.63; within 1.05%); the stated cause of the age tail, two lost frames, was wrong: one suffices
`527ba59` 19 Oct	loss margin: $\varepsilon \geq p + 0.02$ meets $1 - \varepsilon$, interval included; a tolerable loss under bit errors; traced V between independent and Gilbert-8 loss; *theory inside every interval*	tolerable loss held; the rule failed by its interval in 5 of 192 cells, all bursty; traced V below the band in one of four, by 0.0002; *theory outside* 5 *of* 100 *intervals*, what 95% intervals give by chance
`1469c2b` 20 Oct	batch controller: about 30 ms fresher than fixed $b = 4$ at low load; failure within 5% of fixed $b = 4$'s; the target met while two swarms merge	merge and low-load age held; fresher by 90 ms, not 30; failure at 76 nodes against 127 — **failed**
`a36489a` 21 Oct	two further controllers: with a margin of 0.02, failure at 115 nodes or more and the target in 95% of windows; with a predicted load, failure at 105 or more	both failure points held on the mean (116); windows **missed** (93%); neither within 5% of fixed $b = 4$
`04658f3` 20 Oct	time window: no honest frame refused at clock errors up to 10 ms; every frame held 250 ms refused; a 250 ms window survives about two hours without GNSS; *it survives* 75 *min*	the first two held; about two hours **failed** (75 min), *as the finer prediction said*

commit, date	what was registered	outcome
`3b17a90` 20 Oct	formal model: properties (i)–(iv) proven; (v) shown as an attack, then its fix	held across three models, two made after registration; the registered model settled 3 of its 9 lemmas: 15 *of* 19 *predicted verdicts missed*, the lemmas not settling
`6dc683f` 20 Oct	flooding: about 700 forged frames/s saturate a core; an outsider needs over 100 000 a second and is removed by the tag; a member remains a threat; the tag costs about 2%	the outsider removed and the member's threat held; the rates **missed** (928 and 44 400); the tag costs 3.1%
`3a9e85a` 20 Oct, `26f6f08` 22 Oct	compiled code: receive within 0.35 ms; at least 230 nodes per core; energy per record follows the bytes	held (277 µs, 289 nodes, 17.1 µJ); *the energy ranges missed low*
`3b06582` 20 Oct	lazy verification: over 500 nodes per core; evidence within 100–200 ms; *within* 170 *ms of sampling*	held (628); 170 *ms missed by* 1 *ms*
`aa3b86d` 20 Oct, `89cfd6f` 22 Oct	airframes, 25 fields: delta 20–30 B, saving 60–70%, ratio 2.6–3.4; keyframe 50–70 B; fixed-wing delta 10–13 B	delta, saving and ratio held; keyframe (45.5 B) and fixed-wing delta (9.2 B) **missed**, the latter at the planned speeds as well
`b7e1ae3` 20 Oct	stream schemes run against the deadline: every delivery capacity inside its earlier interval; schemes that cannot verify in time never cross	held; *three finer predictions missed*: in-time delivery equal to delivery for two schemes, and crossings for TESLA with $d = 1$ and EMSS with $k = 4$
`b7e1ae3` 20 Oct	other PHYs: ratios spanning 1.7–4.3, rising with the rate; every crossing within ±3% of the model	the rise held; the span **failed** (3.39–3.80); ±3% **failed** at the fastest rate of each width
`50e715b` 20 Oct	Falcon-512: crossings within 10% of 12 and 45 nodes; a ratio near 3.9, larger than with Ed25519	the crossings held; the ratio **failed**: 3.5, as with Ed25519
`baeb1d2` 20 Oct	ledger audit: both planted faults found; about 1.3 GB stored per hour for 124 neighbours; *no other finding;* 7–10 *minutes of a core*	detection held, no false finding; the storage figure failed, less being stored, since frames are kept, not records; *the time failed, faster*: 6.4 min
`fd128fd` 20 Oct	other broadcast systems: the accounting gives no reason to batch at their rates and deadlines	held

All dates are 2026. Two entries of the project's history are absent on purpose. A prediction for the first hardware loss measurement was stated before the measurement and is what exposed the basic-rate artifact of Chapter 9, but it was not a commit of its own. And one

early assertion that a result had been pre-registered was withdrawn, not reconstructed, because its expectation had been written after the data.

Appendix C

Results That Were Corrected

Three results of this thesis were first reported in a form that was later corrected. The chapters give the corrected results. What was first reported is kept here, with what was wrong with it, because the comparison between the two is part of the record (Chapter 11) and a reader who meets an earlier version of this work should be able to see what changed.

C.1 The configuration as first reported

The search over encoding, scheme, placement and batch was first reported as returning 72.0 B per record against 174.3 B for the compression-only baseline, a reduction of 58.7% (Figure C.1). That frame amortised one self-contained record over four frames and so verified 0.881 of its records at the loss rate it was specified for, not 0.95. The corrected figures are in Table 8.1.

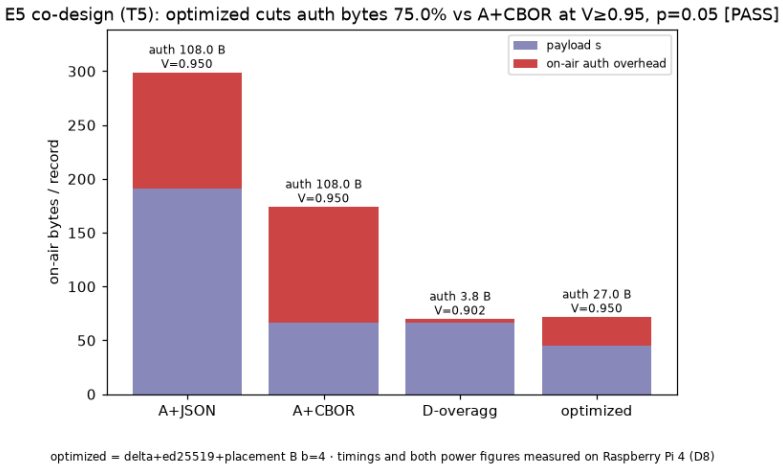


Figure C.1: Per-record cost of the configuration as first reported, against the two baselines (first format). Superseded by Table 8.1.

C.2 The capacity envelope from one load ceiling

Until October 2026 Chapter 8 presented a "capacity envelope" computed, not simulated: the $V \geq 0.95$ crossing had been measured once, at $N = 50$ with a 288 B frame, as a utilisation $U = 2.435$ (offered load over saturation throughput), and that one ceiling was applied to every configuration.

Table C.1: The envelope as previously reported, from the single load ceiling. **Superseded** by Tables 8.1 and 8.4; kept because the comparison is the finding.

configuration	Λ_i	$D_{\max}$	b	$N_{\max}$ (U<1)	$N_{\max}$ (V mean)	(V per run)
optimized delta/B	50	100 ms	4	35	100	64–91
A+CBOR (compression-only)	50	100 ms	1	18	31	24–29
optimized delta/B	20	250 ms	4	103	213	163–201
A+CBOR	20	250 ms	1	32	88	54–80

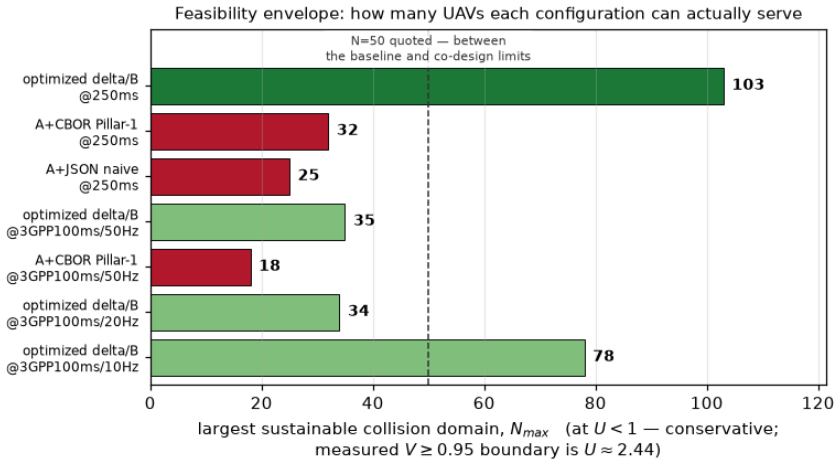


Figure C.2: The envelope as previously reported (saturation threshold). Superseded for the $V \geq 0.95$ threshold by direct simulation.

An earlier experiment had tested half of the assumption behind that ceiling — that it does not depend on frame size — at $N = 50$ (Chapter 9). The other half, that it does not depend on N, was predicted to hold within ±10% in six configurations, and the prediction was committed before the runs. **It failed.** The lean design crosses near 118–124 nodes where the ceiling says 142; the first format's shared-frame configuration crosses *above* the ceiling's value. The signs differ between configurations, so the ceiling cannot be re-tuned. The reason is not subtle in hindsight: U divides the offered load by the saturation throughput of N stations, which falls as N grows, while the loss of a lightly loaded network depends on how much airtime the traffic occupies and not on how many stations produce it.

The ratios of the old envelope had been called "protected by construction", because numerator and denominator used the same ceiling. They were protected against an error

in the ceiling's *value*. They were not protected against the ceiling being the wrong *kind* of quantity, and they moved.

C.3 Per-frame chaining as first computed

On the low-rate arm, carrying the chain link once per frame was first reported to raise the sustainable record rate by a factor of 3.0 (Figure C.3). The figure sized every record from a delta measured with records 50 ms apart and carried no self-contained record. At the spacing the batch itself implies, and with every frame decoding alone, the factor is about 2.1 in the first format (Section 10.3).

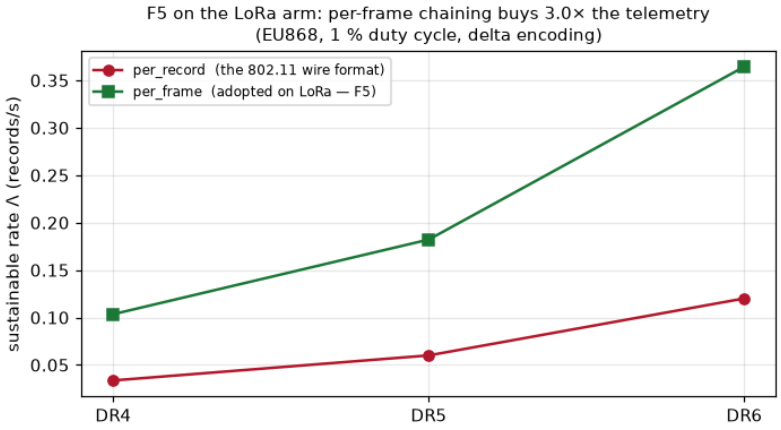


Figure C.3: Per-frame versus per-record chaining across data rates, as first computed. Superseded by the table of Section 10.3.

Bibliography

[1] X. Chen, J. Tang, and S. Lao, "Review of unmanned aerial vehicle swarm communication architectures and routing protocols," *Applied Sciences*, vol. 10, no. 10, p. 3661, 2020.

[2] L. Gupta, R. Jain, and G. Vaszkun, "Survey of important issues in UAV communication networks," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1123–1152, 2016.

[3] A. Bujari, C. T. Calafate, J.-C. Cano, P. Manzoni, C. E. Palazzi, and D. Ronzani, "Flying ad-hoc network application scenarios and mobility models," *International Journal of Distributed Sensor Networks*, vol. 13, no. 10, 2017.

[4] X. Huang, A. Liu, H. Zhou, K. Yu, W. Wang, and X. Shen, "FMAC: A self-adaptive MAC protocol for flocking of flying ad hoc network," *IEEE Internet of Things Journal*, vol. 8, no. 1, pp. 610–625, 2021.

[5] PX4 Development Team, "PX4-Autopilot: `mavlink_main.cpp`, `configure_streams_to_default`," Source code, release v1.17.0, https://github.com/PX4/PX4-Autopilot, 2026.

[6] ArduPilot Development Team, "ArduPilot: `GCS_MAVLink_Parameters.cpp`, `default_rates[]`," Source code, https://github.com/ArduPilot/ardupilot, 2026.

[7] 3GPP, "Unmanned aerial system (UAS) support in 3GPP," 3rd Generation Partnership Project, Technical Specification TS 22.125, Apr. 2022.

[8] R. Rotta and P. Mykytyn, "Secure multi-hop telemetry broadcasts for UAV swarm communication," arXiv:2401.11915, 2024.

[9] D. Bhatt, S. Penumatsa, and V. Kumar, "Hybrid MAC protocol with integrated multi-layered security for resource-constrained UAV swarm communications," arXiv:2510.10236, 2025.

[10] S. Kurkowski, T. Camp, and M. Colagrosso, "MANET simulation studies: The incredibles," *ACM SIGMOBILE Mobile Computing and Communications Review*, vol. 9, no. 4, pp. 50–61, 2005.

[11] E. R. Cavalcanti, J. A. R. de Souza, M. A. Spohn, R. C. d. M. Gomes, and A. F. B. F. d. Costa, "VANETs' research over the past decade: Overview, credibility, and trends," *ACM SIGCOMM Computer Communication Review*, vol. 48, no. 2, pp. 31–39, 2018.

[12] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.

[13] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," https://bitcoin.org/bitcoin.pdf, 2008.

[14] P. Mehta, R. Gupta, and S. Tanwar, "Blockchain envisioned UAV networks: Challenges, solutions, and comparisons," *Computer Communications*, vol. 151, pp. 518–538, 2020.

[15] D. Basin, C. Cremers, J. Dreier, S. Meier, R. Sasse, and B. Schmidt, "The Tamarin prover," version 1.12.0, https://tamarin-prover.com, 2023.

[16] National Institute of Standards and Technology, "Digital signature standard (DSS)," NIST, FIPS PUB 186-5, 2023.

[17] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," in *Cryptographic Hardware and Embedded Systems — CHES 2011*, ser. LNCS, vol. 6917. Springer, 2011, pp. 124–142.

[18] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," IETF, RFC 8032, 2017.

[19] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the Weil pairing," in *Advances in Cryptology — ASIACRYPT 2001*, ser. LNCS, vol. 2248. Springer, 2001, pp. 514–532.

[20] D. Boneh, C. Gentry, B. Lynn, and H. Shacham, "Aggregate and verifiably encrypted signatures from bilinear maps," in *Advances in Cryptology — EUROCRYPT 2003*, ser. LNCS, vol. 2656. Springer, 2003, pp. 416–432.

[21] MAVLink Development Team, "MAVLink developer guide: Message signing (authentication)," https://mavlink.io/en/guide/message_signing.html, 2026.

[22] ETSI, "Intelligent transport systems (ITS); security; security header and certificate formats," European Telecommunications Standards Institute, Technical Specification TS 103 097, Jun. 2015.

[23] G. Twardokus, N. Bindel, H. Rahbari, and S. McCarthy, "When cryptography needs a hand: Practical post-quantum authentication for V2V communications," in *Network and Distributed System Security Symposium (NDSS)*, 2024.

[24] R. Gennaro and P. Rohatgi, "How to sign digital streams," in *Advances in Cryptology — CRYPTO '97*, ser. LNCS, vol. 1294. Springer, 1997, pp. 180–197.

[25] C. K. Wong and S. S. Lam, "Digital signatures for flows and multicasts," *IEEE/ACM Transactions on Networking*, vol. 7, no. 4, pp. 502–513, 1999.

[26] A. Perrig, R. Canetti, J. D. Tygar, and D. Song, "Efficient authentication and signing of multicast streams over lossy channels," in *Proc. 2000 IEEE Symposium on Security and Privacy (S&P)*, 2000, pp. 56–73.

[27] ——, "The TESLA broadcast authentication protocol," *RSA CryptoBytes*, vol. 5, no. 2, pp. 2–13, 2002.

[28] J. Veara, M. Jain, K. Moy, and A. Ranganathan, "TBRD: TESLA authenticated UAS broadcast remote ID," arXiv:2510.11343, 2025.

[29] A. Pannetrat and R. Molva, "Efficient multicast packet authentication," in *Network and Distributed System Security Symposium (NDSS)*, 2003.

[30] S. Card and R. Moskowitz, "DRIP entity tag (DET) authentication formats and protocols for broadcast remote identification (RID)," IETF, RFC 9575, 2024, A. Wiethuechter, Ed.

[31] M. Strohmeier, V. Lenders, and I. Martinovic, "On the security of the automatic dependent surveillance-broadcast protocol," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 2, pp. 1066–1087, 2015.

[32] C. Zhang, R. Lu, X. Lin, P.-H. Ho, and X. Shen, "An efficient identity-based batch verification scheme for vehicular sensor networks," in *IEEE INFOCOM 2008 — The 27th Conference on Computer Communications*, 2008, pp. 246–250.

[33] A. S. Rajasekaran, A. Maria, F. Al-Turjman, C. Altrjman, and L. Mostarda, "Anonymous mutual and batch authentication with location privacy of UAV in FANET," *Drones*, vol. 6, no. 1, p. 14, 2022.

[34] H. Li, C. Shen, H. Huang, and C. Wu, "A certificateless aggregate signature scheme for VANETs with privacy protection properties," *PLOS ONE*, vol. 20, no. 2, p. e0317047, 2025.

[35] E. F. Cahyadi, T.-W. Su, C.-C. Yang, and M.-S. Hwang, "A certificateless aggregate signature scheme for security and privacy protection in VANET," *International Journal of Distributed Sensor Networks*, vol. 18, no. 5, 2022.

[36] Y. Liang and Y. Liu, "Analysis and improvement of an efficient certificateless aggregate signature with conditional privacy preservation in VANETs," *IEEE Systems Journal*, vol. 17, no. 1, pp. 664–672, 2023.

[37] Z. Xu, D. He, N. Kumar, and K.-K. R. Choo, "Efficient certificateless aggregate signature scheme for performing secure routing in VANETs," *Security and Communication Networks*, vol. 2020, p. 5276813, 2020.

[38] H. Wang, L. Wang, K. Zhang, J. Li, and Y. Luo, "A conditional privacy-preserving certificateless aggregate signature scheme in the standard model for VANETs," *IEEE Access*, vol. 10, pp. 15 605–15 618, 2022.

[39] C. Bormann and P. Hoffman, "Concise binary object representation (CBOR)," IETF, RFC 8949, 2020.

[40] S. Furuhashi, "MessagePack: Efficient binary serialization format," https://msgpack.org, 2008.

[41] T. Pelkonen, S. Franklin, J. Teller, P. Cavallaro, Q. Huang, J. Meza, and K. Veeraraghavan, "Gorilla: A fast, scalable, in-memory time series database," *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1816–1827, 2015.

[42] G. Bianchi, "Performance analysis of the IEEE 802.11 distributed coordination function," *IEEE Journal on Selected Areas in Communications*, vol. 18, no. 3, pp. 535–547, 2000.

[43] X. Ma and X. Chen, "Saturation performance of IEEE 802.11 broadcast networks," *IEEE Communications Letters*, vol. 11, no. 8, pp. 686–688, Aug. 2007.

[44] ——, "Performance analysis of IEEE 802.11 broadcast scheme in ad hoc wireless LANs," *IEEE Transactions on Vehicular Technology*, vol. 57, no. 6, pp. 3757–3768, Nov. 2008.

[45] I. Tinnirello, G. Bianchi, and Y. Xiao, "Refinements on IEEE 802.11 distributed coordination function modeling approaches," *IEEE Transactions on Vehicular Technology*, vol. 59, no. 3, pp. 1055–1067, Mar. 2010.

[46] X. Ma, X. Chen, and H. H. Refai, "Performance and reliability of DSRC vehicular safety communication: A formal analysis," *EURASIP Journal on Wireless Communications and Networking*, vol. 2009, p. 969164, 2009.

[47] L. Cao, H. Yin, J. Hu, and L. Zhang, "Performance analysis and improvement on DSRC application for V2V communication," arXiv:2102.07023, 2021.

[48] L. Hu and Z. Dai, "Performance and reliability analysis of prioritized safety messages broadcasting in DSRC with hidden terminals," *IEEE Access*, vol. 8, pp. 177 112–177 124, 2020.

[49] D. Skordoulis, Q. Ni, H.-H. Chen, A. P. Stephens, C. Liu, and A. Jamalipour, "IEEE 802.11n MAC frame aggregation mechanisms for next-generation high-throughput WLANs," *IEEE Wireless Communications*, vol. 15, no. 1, pp. 40–47, 2008.

[50] J. Haxhibeqiri, F. Van den Abeele, I. Moerman, and J. Hoebeke, "LoRa scalability: A simulation model based on interference measurements," *Sensors*, vol. 17, no. 6, p. 1193, 2017.

[51] Q. Zirak, D. Shashev, and S. Shidlovskiy, "Swarm of drones using LoRa flying ad-hoc network," in *2021 International Conference on Information Technology (ICIT)*, 2021, pp. 400–405.

[52] Semtech Corporation, "SX1276/77/78/79 137–1020 MHz low power long range transceiver," Semtech Corporation, Datasheet, Mar. 2015.

[53] LoRa Alliance, "RP002-1.0.3 LoRaWAN regional parameters," LoRa Alliance, Technical Specification RP002-1.0.3, 2021.

[54] ——, "TS001-1.0.4 LoRaWAN L2 1.0.4 specification," LoRa Alliance, Technical Specification TS001-1.0.4, 2020.

[55] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, "Age of information: An introduction and survey," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1183–1210, 2021.

[56] T. C. Hesterberg, "What teachers should know about the bootstrap: Resampling in the undergraduate statistics curriculum," *The American Statistician*, vol. 69, no. 4, pp. 371–386, 2015.

[57] W. Feng, "brcmfmac: enable frameburst mode in default firmware setting," Linux kernel, commit a3bdc6deb60b, 2018.

[58] G. Bianchi, A. Di Stefano, C. Giaconia, L. Scalia, G. Terrazzino, and I. Tinnirello, "Experimental assessment of the backoff behavior of commercial IEEE 802.11b network cards," in *IEEE INFOCOM 2007 - 26th IEEE International Conference on Computer Communications*, 2007, pp. 1181–1189.

[59] G. Klimiashvili, C. Tapparello, and W. Heinzelman, "LoRa vs. WiFi ad hoc: A performance analysis and comparison," in *2020 International Conference on Computing, Networking and Communications (ICNC)*, 2020, pp. 654–660.

[60] D. Boneh, S. Gorbunov, R. S. Wahby, H. Wee, C. A. Wood, and Z. Zhang, "BLS signatures," IRTF Crypto Forum Research Group, Internet-Draft draft-irtf-cfrg-bls-signature-05, Jun. 2022.

[61] F. Van den Abeele, J. Haxhibeqiri, I. Moerman, and J. Hoebeke, "Scalability analysis of large-scale LoRaWAN networks in ns-3," *IEEE Internet of Things Journal*, vol. 4, no. 6, pp. 2186–2198, 2017.

[62] T.-H. To and A. Duda, "Simulation of LoRa in NS-3: Improving LoRa performance with CSMA," in *2018 IEEE International Conference on Communications (ICC)*, 2018, pp. 1–7.

[63] A. Traspadini, M. Zorzi, and M. Giordani, "Performance evaluation of LoRa for IoT applications in non-terrestrial networks via ns-3," arXiv:2509.02811, 2025.

[64] W. D. Paredes, H. Kaushal, I. Vakilinia, and Z. Prodanoff, "LoRa technology in flying ad hoc networks: A survey of challenges and open issues," *Sensors*, vol. 23, no. 5, p. 2403, 2023.

[65] R. Berto, P. Napoletano, and M. Savi, "A LoRa-based mesh network for peer-to-peer long-range communication," *Sensors*, vol. 21, no. 13, p. 4314, 2021.

[66] L. Davoli, E. Pagliari, and G. Ferrari, "Hybrid LoRa-IEEE 802.11s opportunistic mesh networking for flexible UAV swarming," *Drones*, vol. 5, no. 2, p. 26, 2021.

[67] R. P. Centelles, F. Freitag, R. Meseguer, and L. Navarro, "Beyond the star of stars: An introduction to multihop and mesh for LoRa and LoRaWAN," *IEEE Pervasive Computing*, vol. 20, no. 2, pp. 63–72, 2021.

[68] A. Abrardo and A. Pozzebon, "A multi-hop LoRa linear sensor network for the monitoring of underground environments: The case of the medieval aqueducts in Siena, Italy," *Sensors*, vol. 19, no. 2, p. 402, 2019.

[69] X. Jiang, H. Zhang, E. A. Barsallo Yi, N. Raghunathan, C. Mousoulis, S. Chaterji, D. Peroulis, A. Shakouri, and S. Bagchi, "Hybrid low-power wide-area mesh network for IoT applications," *IEEE Internet of Things Journal*, vol. 8, no. 2, pp. 901–915, 2021.

[70] T. G. Durand and M. J. Booysen, "Performance evaluation of a mesh-topology LoRa network," *Sensors*, vol. 25, no. 5, p. 1602, 2025.